

st125333_LabAssignment06_DLCV

November 4, 2025

```
[1]: import torch
import torch.nn as nn
import torch.optim as optim
import cv2
import matplotlib.pyplot as plt
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image
```

```
[2]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(" Using device:", device)
```

Using device: cuda

```
[3]: import os
import subprocess

def download_file(filepath: str, url: str):
    """Download file from URL using curl if it doesn't already exist."""
    if not os.path.exists(filepath):
        os.makedirs(os.path.dirname(filepath), exist_ok=True)
        print(f"Download to {filepath}...")
        subprocess.run(["curl", "-L", "-o", filepath, url], check=True)
    else:
        print(f"File already exists: {filepath}")
```

```
[4]: import zipfile

def extract_zip_file(zip_path: str, extract_path: str):
    """Extract ZIP file to target path if it exists."""
    if os.path.exists(zip_path):
        with zipfile.ZipFile(zip_path, 'r') as zip_ref:
            zip_ref.extractall(extract_path)
        print(f" Dataset extracted to: {extract_path}")
    else:
        print(f" ZIP file not found: {zip_path}")
```

```
[5]: import os
import zipfile
from tqdm import tqdm

def unzip_file(zip_path, extract_to):
    if os.path.exists(extract_to) and any(os.scandir(extract_to)):
        print(f" Skipping extraction to {extract_to} (already populated)")
        return

    print(f" Extracting {os.path.basename(zip_path)} → {extract_to}")
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        members = zip_ref.infolist()
        for member in tqdm(members, desc="Extracting", unit="file"):
            zip_ref.extract(member, extract_to)

def unzip_folder(zip_path, extract_to, sub_dir="train"):
    expected_path = os.path.join(extract_to, sub_dir)
    if os.path.exists(expected_path) and any(os.scandir(expected_path)):
        print(f" Skipping extraction: {expected_path} already populated")
        return

    print(f" Extracting {os.path.basename(zip_path)} → {extract_to}")
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        for member in tqdm(zip_ref.infolist(), desc="Extracting", unit="file"):
            zip_ref.extract(member, extract_to)
```

```
[6]: import os
import requests
from tqdm import tqdm

def download_big_file(dest_path, url):
    # Ensure destination directory exists
    os.makedirs(os.path.dirname(dest_path), exist_ok=True)

    if os.path.exists(dest_path):
        print(f" Skipping {os.path.basename(dest_path)} (already exists)")
        return

    response = requests.get(url, stream=True)
    total = int(response.headers.get('content-length', 0))

    with open(dest_path, 'wb') as file, tqdm(
        desc=os.path.basename(dest_path),
        total=total,
        unit='B',
        unit_scale=True,
        unit_divisor=1024,
```

```

    ) as bar:
        for data in response.iter_content(chunk_size=1024):
            size = file.write(data)
            bar.update(size)

```

```

[7]: # from google.colab import drive
     # drive.mount('/content/drive')

```

```

[8]: raw_zip_dir = "DLCV_Lab_Dataset/raw_zips/pascal-voc-2007.zip"
     source_url = "https://www.kaggle.com/api/v1/datasets/download/zaraks/
        ↳pascal-voc-2007"
     download_big_file(raw_zip_dir, source_url)

```

Skipping pascal-voc-2007.zip (already exists)

```

[9]: zip_path = "/content/DLCV_Lab_Dataset/raw_zips/pascal-voc-2007.zip"
     extract_path = "DLCV_Lab_Dataset"
     unzip_folder(zip_path, extract_path)

```

Extracting pascal-voc-2007.zip → DLCV_Lab_Dataset

Extracting: 100%| | 42574/42574 [00:25<00:00, 1699.84file/s]

```

[118]: raw_zip_dir = "yolo/raw_zips/yolov3-weights.zip"
       source_url = "https://www.kaggle.com/api/v1/datasets/download/shivam316/
          ↳yolov3-weights"
       download_file(raw_zip_dir, source_url)

```

Download to yolo/raw_zips/yolov3-weights.zip...

```

[119]: zip_path = "yolo/raw_zips/yolov3-weights.zip"
       extract_path = "yolo"
       unzip_folder(zip_path, extract_path)

```

Extracting yolov3-weights.zip → yolo

Extracting: 100%| | 1/1 [00:01<00:00, 1.69s/file]

```

[10]: import os
       import xml.etree.ElementTree as ET
       from PIL import Image
       from torch.utils.data import Dataset
       import torch
       import torchvision.transforms as T

       VOC_CLASSES = [
           'aeroplane', 'bicycle', 'bird', 'boat', 'bottle', 'bus', 'car', 'cat', 'chair', 'cow',
           ↳
           ↳ 'diningtable', 'dog', 'horse', 'motorbike', 'person', 'pottedplant', 'sheep', 'sofa', 'train', 'tvmonitor'
       ]

```

```

class VOCDatasetYOLO(Dataset):
    def __init__(self, root, image_set='train', transforms=None):
        """
        root: path to /VOC2012_train_val/VOC2012_train_val
        image_set: 'train', 'val', or 'trainval'
        """
        self.root = root
        self.image_dir = os.path.join(root, "JPEGImages")
        self.ann_dir = os.path.join(root, "Annotations")
        set_file = os.path.join(root, "ImageSets", "Main", f"{image_set}.txt")

        with open(set_file) as f:
            self.ids = [line.strip() for line in f.readlines()]
        self.transforms = transforms or T.Compose([
            T.Resize((416,416)),
            T.ToTensor()
        ])

    def __len__(self):
        return len(self.ids)

    def parse_annotation(self, ann_path):
        tree = ET.parse(ann_path)
        boxes, labels = [], []
        for obj in tree.findall("object"):
            cls = obj.find("name").text.lower().strip()
            if cls not in VOC_CLASSES:
                continue
            label = VOC_CLASSES.index(cls)
            bndbox = obj.find("bndbox")
            x1 = float(bndbox.find("xmin").text)
            y1 = float(bndbox.find("ymin").text)
            x2 = float(bndbox.find("xmax").text)
            y2 = float(bndbox.find("ymax").text)
            boxes.append([x1, y1, x2, y2])
            labels.append(label)
        return torch.tensor(boxes, dtype=torch.float32), torch.tensor(labels,
↳dtype=torch.long)

    def __getitem__(self, idx):
        img_id = self.ids[idx]
        img_path = os.path.join(self.image_dir, f"{img_id}.jpg")
        ann_path = os.path.join(self.ann_dir, f"{img_id}.xml")

        img = Image.open(img_path).convert("RGB")
        boxes, labels = self.parse_annotation(ann_path)

```

```

    if self.transforms:
        img = self.transforms(img)
    target = {"boxes": boxes, "labels": labels, "image_id": img_id}
    return img, target

```

```

[126]: import torch
import torch.nn as nn

#
# Conv + BatchNorm + LeakyReLU
#
class ConvBNLeaky(nn.Module):
    def __init__(self, in_c, out_c, k=3, s=1, p=1):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_c, out_c, k, s, p, bias=False),
            nn.BatchNorm2d(out_c),
            nn.LeakyReLU(0.1, inplace=True)
        )
    def forward(self, x): return self.block(x)

#
# YOLOv3-Tiny Detector
#
class YOLOv3Tiny(nn.Module):
    """
    Small YOLOv3-Tiny-style model.
    Compatible with Pascal VOC (20 classes) or COCO (80 classes)
    by changing num_classes.
    """
    def __init__(self, num_classes=20):
        super().__init__()
        self.num_classes = num_classes
        self.num_anchors = 3
        self.filters = self.num_anchors * (5 + num_classes) # 75 (VOC) or 255_
        ↪ (COCO)

        # Backbone
        self.layer1 = ConvBNLeaky(3, 16); self.pool1 = nn.MaxPool2d(2,2)
        self.layer2 = ConvBNLeaky(16, 32); self.pool2 = nn.MaxPool2d(2,2)
        self.layer3 = ConvBNLeaky(32, 64); self.pool3 = nn.MaxPool2d(2,2)
        self.layer4 = ConvBNLeaky(64, 128); self.pool4 = nn.MaxPool2d(2,2)
        self.layer5 = ConvBNLeaky(128, 256); self.pool5 = nn.MaxPool2d(2,2)
        self.layer6 = ConvBNLeaky(256, 512); self.pool6 = nn.MaxPool2d(2,1,1)
        self.layer7 = ConvBNLeaky(512, 1024)

        # Detection heads

```

```

self.conv8 = ConvBNLeaky(1024, 256, k=1, s=1, p=0)
self.det1 = nn.Conv2d(256, self.filters, 1) # 13×13 output
self.conv9 = ConvBNLeaky(256, 128, k=1, s=1, p=0)
self.upsamp = nn.Upsample(scale_factor=2, mode="nearest")
self.det2_conv = ConvBNLeaky(384, 256, k=3, s=1, p=1)
self.det2 = nn.Conv2d(256, self.filters, 1) # 26×26 output

def forward(self, x):
    # -----
    # Backbone: feature extraction
    # -----
    x = self.layer1(x) # 416 → 416, 3→16
    x = self.pool1(x) # 208×208
    x = self.layer2(x) # 208 → 208, 16→32
    x = self.pool2(x) # 104×104
    x = self.layer3(x) # 104 → 104, 32→64
    x = self.pool3(x) # 52×52
    x = self.layer4(x) # 52 → 52, 64→128
    x = self.pool4(x) # 26×26
    route = self.layer5(x) # save feature map for skip connection (26×26)
    x = self.pool5(route) # 13×13
    x = self.layer6(x) # 13×13
    x = self.pool6(x) # still 13×13
    x = self.layer7(x) # 13×13 → deep features (1024 channels)

    # -----
    # Detection Head 1 (13 × 13)
    # -----
    y1_in = self.conv8(x) # 1024→256
    det1 = self.det1(y1_in) # [B, 75, 13, 13] (3 × (5 + 20))

    # -----
    # Detection Head 2 (26 × 26)
    # -----
    y2_in = self.conv9(y1_in) # 256→128
    y2_in = self.upsamp(y2_in) # 13 → 26 (ideally)

    # Align spatial sizes before concatenation (fixes 26 vs 28 bug)
    if y2_in.shape[-2:] != route.shape[-2:]:
        y2_in = nn.functional.interpolate(
            y2_in, size=route.shape[-2:], mode="nearest"
        )

    # Concatenate with earlier feature map
    y2_in = torch.cat([y2_in, route], dim=1) # 128 + 256 = 384 channels
    y2_in = self.det2_conv(y2_in) # 384→256
    det2 = self.det2(y2_in) # [B, 75, 26, 26]

```

```

# -----
# Outputs from both scales
# -----
return [det1, det2]

def load_weights(self, weights_path, strict=False):
    """
    Load pretrained Darknet .weights file into the PyTorch YOLOv3-Tiny
    ↪ model.

    Args:
        weights_path (str): path to .weights file from Darknet (e.g.
    ↪ yolov3-tiny.weights)
        strict (bool): if True, enforce exact layer match (for COCO → COCO);
        ↪ if False, skip mismatched shapes (for COCO → VOC
    ↪ fine-tune)
    """
    import numpy as np
    if not os.path.isfile(weights_path):
        raise FileNotFoundError(f" Weights file not found: {weights_path}")

    with open(weights_path, "rb") as f:
        header = np.fromfile(f, dtype=np.int32, count=5)
        weights = np.fromfile(f, dtype=np.float32)
        print(f" Darknet header: {header[:3].tolist()}, seen={header[3]}")

    ptr, loaded, skipped = 0, 0, 0
    for m in self.modules():
        # ----- Conv + BN -----
        if isinstance(m, ConvBNLeaky):
            conv_layer = m.conv
            bn_layer = m.bn
            num_bn = bn_layer.bias.numel()

            # Load BN params
            bn_biases = torch.from_numpy(weights[ptr:ptr+num_bn]);
    ↪ ptr += num_bn
            bn_weights = torch.from_numpy(weights[ptr:ptr+num_bn]);
    ↪ ptr += num_bn
            bn_running_mean = torch.from_numpy(weights[ptr:ptr+num_bn]);
    ↪ ptr += num_bn
            bn_running_var = torch.from_numpy(weights[ptr:ptr+num_bn]);
    ↪ ptr += num_bn

            bn_layer.bias.data.copy_(bn_biases.view_as(bn_layer.bias))

```

```

        bn_layer.weight.data.copy_(bn_weights.view_as(bn_layer.weight))
        bn_layer.running_mean.data.copy_(bn_running_mean.
↪view_as(bn_layer.running_mean))
        bn_layer.running_var.data.copy_(bn_running_var.view_as(bn_layer.
↪running_var))

    # Load conv weights
    num_w = conv_layer.weight.numel()
    conv_w = torch.from_numpy(weights[ptr:ptr+num_w]); ptr += num_w
    conv_w = conv_w.view_as(conv_layer.weight)
    conv_layer.weight.data.copy_(conv_w)
    loaded += 1

    # ----- Plain Conv (no BN) -----
    elif isinstance(m, nn.Conv2d):
        num_bias = m.bias.numel() if m.bias is not None else 0
        num_w = m.weight.numel()
        total = num_bias + num_w

        if ptr + total > weights.size:
            print(f" Ran out of weights at layer {loaded+skipped}.")
            break

        # If detection head has shape mismatch, skip (for VOC
↪fine-tuning)
        expected_w_shape = m.weight.shape
        if not strict and num_w != (weights[ptr+num_bias:ptr+total].
↪size):
            print(f" Skipping mismatched layer {m} (shape
↪{expected_w_shape})")
            skipped += 1
            continue

        if num_bias:
            conv_bias = torch.from_numpy(weights[ptr:ptr+num_bias]);
↪ptr += num_bias
            m.bias.data.copy_(conv_bias.view_as(m.bias))
            conv_w = torch.from_numpy(weights[ptr:ptr+num_w]); ptr += num_w
            conv_w = conv_w.view_as(m.weight)
            m.weight.data.copy_(conv_w)
            loaded += 1

    print(f" Loaded {loaded} layers, skipped {skipped} layers
↪(strict={strict}) from {weights_path}")

```



```
[12]: # Pascal VOC / COCO Tiny anchors
ANCHORS = [
    [(10,13),(16,30),(33,23)], # 13×13 (stride 32)
    [(30,61),(62,45),(59,119)] # 26×26 (stride 16)
]
STRIDES = [32, 16]

[13]: import torch
import torch.nn as nn
import torch.optim as optim
from tqdm import tqdm

#
# Simplified YOLO Loss (demo)
#
class YoloTinyLoss(nn.Module):
    def __init__(self, num_classes=20):
        super().__init__()
        self.num_classes = num_classes
    def forward(self, preds, targets=None):
        loss = 0.0
        for p in preds:
            loss += torch.mean(p ** 2)
        return loss

class SimpleYoloLoss(nn.Module):
    def __init__(self, anchors, num_classes, img_size=416, lambda_box=0.05,
        lambda_obj=1.0, lambda_cls=0.5):
        super().__init__()
        self.anchors = torch.tensor(anchors, dtype=torch.float32)
        self.num_classes = num_classes
        self.img_size = img_size
        self.lambda_box = lambda_box
        self.lambda_obj = lambda_obj
        self.lambda_cls = lambda_cls
        self.bce = nn.BCEWithLogitsLoss()
        self.mse = nn.MSELoss()

    def forward(self, preds, targets):
        # preds: list([B, A*(5+C), H, W])
        # targets: [{boxes:[N,4], labels:[N]} ...] from dataset
        loss_box, loss_obj, loss_cls = 0, 0, 0
        for p in preds:
            # simplified: encourage small predictions initially
            loss_box += (p[..., :4] ** 2).mean()
            loss_obj += self.bce(p[..., 4:], torch.zeros_like(p[..., 4:]))
        return self.lambda_box*loss_box + self.lambda_obj*loss_obj
```

```

#
# Train / Evaluate / Main Training
#
def train_one_epoch_yolo(model, dataloader, criterion, optimizer, device):
    model.train()
    total_loss = 0.0
    loop = tqdm(dataloader, total=len(dataloader), desc="Training", leave=False)

    for imgs, targets in loop:
        # imgs is a tuple of tensors → stack
        imgs = torch.stack(imgs).to(device)

        preds = model(imgs)
        loss = criterion(preds, targets) # you can extend loss later with
        ↪ bbox/class terms
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        loop.set_postfix({"loss": f"{loss.item():.4f}"})
    return total_loss / len(dataloader)

@torch.no_grad()
def evaluate_yolo(model, dataloader, criterion, device):
    model.eval()
    total_loss = 0.0
    loop = tqdm(dataloader, total=len(dataloader), desc="Validation",
    ↪ leave=False)

    for imgs, targets in loop:
        imgs = torch.stack(imgs).to(device)

        preds = model(imgs)
        loss = criterion(preds, targets)
        total_loss += loss.item()
        loop.set_postfix({"val_loss": f"{loss.item():.4f}"})
    return total_loss / len(dataloader)

def train_model_yolo(model, trainloader, valloader, criterion, optimizer,
    ↪ epochs, device):
    """
    Train YOLO-style model with provided criterion and optimizer.

```

```

Args:
    model: YOLO model (e.g., YOLOv3Tiny)
    trainloader, valloader: Dataloaders
    criterion: loss function (e.g., SimpleYoloLoss)
    optimizer: torch optimizer
    epochs: number of epochs to train
    device: 'cuda' or 'cpu'
    """
    train_losses, val_losses, ap_values = [], [], []

    for epoch in range(epochs):
        tr_loss = train_one_epoch_yolo(model, trainloader, criterion,
        ↪optimizer, device)
        val_loss = evaluate_yolo(model, valloader, criterion, device)

        train_losses.append(tr_loss)
        val_losses.append(val_loss)
        ap_values.append(max(0, 100 - val_loss)) # placeholder mAP trend

        print(f"Epoch [{epoch+1}/{epochs}] | TrainLoss={tr_loss:.4f} |
        ↪ValLoss={val_loss:.4f}")

    return model, train_losses, val_losses, ap_values

```

```

[14]: import matplotlib.pyplot as plt

def plot_yolo_training(train_losses, val_losses, ap_values=None,
    ↪title_prefix="YOLO Training"):
    """
    Plot YOLO training and validation loss and (optionally) mAP.
    """
    plt.figure(figsize=(8,4))
    plt.plot(train_losses, label="Train Loss", marker='o')
    plt.plot(val_losses, label="Val Loss", marker='s')
    plt.title(f"{title_prefix} - Loss Curve")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.show()

    if ap_values is not None:
        plt.figure(figsize=(8,4))
        plt.plot(ap_values, color='green', label="mAP", marker='^')
        plt.title(f"{title_prefix} - mAP Trend")
        plt.xlabel("Epoch")
        plt.ylabel("mAP (%)")

```

```
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

```
[15]: !apt-get install tree -qq
```

```
!tree -f -L 3 /content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007
```

```
/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007
/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007/VOCdevkit
/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007/VOCdevkit/VOC2007
/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007/VOCdevkit/VOC2007/Annotations
/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007/VOCdevkit/VOC2007/ImageSets
/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007/VOCdevkit/VOC2007/JPEGImages
/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007/VOCdevkit/VOC2007/SegmentationClass
/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007/VOCdevkit/VOC2007/SegmentationObject

7 directories, 0 files
```

```
[16]: !tree -f -L 3 /content/DLCV_Lab_Dataset/VOCtest_06-Nov-2007
```

```
/content/DLCV_Lab_Dataset/VOCtest_06-Nov-2007
/content/DLCV_Lab_Dataset/VOCtest_06-Nov-2007/VOCdevkit
/content/DLCV_Lab_Dataset/VOCtest_06-Nov-2007/VOCdevkit/VOC2007
/content/DLCV_Lab_Dataset/VOCtest_06-Nov-2007/VOCdevkit/VOC2007/Annotations
/content/DLCV_Lab_Dataset/VOCtest_06-Nov-2007/VOCdevkit/VOC2007/ImageSets
/content/DLCV_Lab_Dataset/VOCtest_06-Nov-2007/VOCdevkit/VOC2007/JPEGImages
/content/DLCV_Lab_Dataset/VOCtest_06-Nov-2007/VOCdevkit/VOC2007/SegmentationClass
/content/DLCV_Lab_Dataset/VOCtest_06-Nov-2007/VOCdevkit/VOC2007/SegmentationObject
```

7 directories, 0 files

[]:

```
[101]: from torch.utils.data import DataLoader

train_root = "/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007/VOCdevkit/
↳VOC2007"
val_root    = train_root  # same folder; different split txt files

train_dataset = VOCDataSetYOL0(train_root, image_set="train")
val_dataset   = VOCDataSetYOL0(val_root, image_set="val")

trainloader = DataLoader(train_dataset, batch_size=4, shuffle=True,
↳collate_fn=lambda x: tuple(zip(*x)))
valloader    = DataLoader(val_dataset, batch_size=4, shuffle=False,
↳collate_fn=lambda x: tuple(zip(*x)))

print(" Loaded:", len(train_dataset), "train samples,", len(val_dataset), "val
↳samples")
```

Loaded: 2501 train samples, 2510 val samples

```
[123]: device = "cuda" if torch.cuda.is_available() else "cpu"

model = YOLOv3Tiny(num_classes=20).to(device)
# model.load_weights("/content/yolo/yolov3.weights")

# # Train (you can start with a few epochs to test)
# model, train_losses, val_losses, ap_values = train_model_yolo(
#     model, trainloader, valloader, epochs=10, lr=1e-4, device=device
# )
```

```
[125]: # Anchors (for Pascal VOC)
VOC_ANCHORS = [
    [(10,13),(16,30),(33,23)],      # 13×13 head
    [(30,61),(62,45),(59,119)]     # 26×26 head
]

# Loss & optimizer
criterion = SimpleYoloLoss(anchors=VOC_ANCHORS, num_classes=model.num_classes,
↳img_size=416)
optimizer = optim.Adam(model.parameters(), lr=1e-4)

# Train
model, train_losses, val_losses, ap_values = train_model_yolo(
    model, trainloader, valloader,
    criterion=criterion,
```

```
optimizer=optimizer,  
epochs=10,  
device=device  
)
```

Epoch [1/10] | TrainLoss=0.1586 | ValLoss=0.0228

Epoch [2/10] | TrainLoss=0.0125 | ValLoss=0.0068

Epoch [3/10] | TrainLoss=0.0046 | ValLoss=0.0029

Epoch [4/10] | TrainLoss=0.0023 | ValLoss=0.0019

Epoch [5/10] | TrainLoss=0.0013 | ValLoss=0.0010

Epoch [6/10] | TrainLoss=0.0008 | ValLoss=0.0007

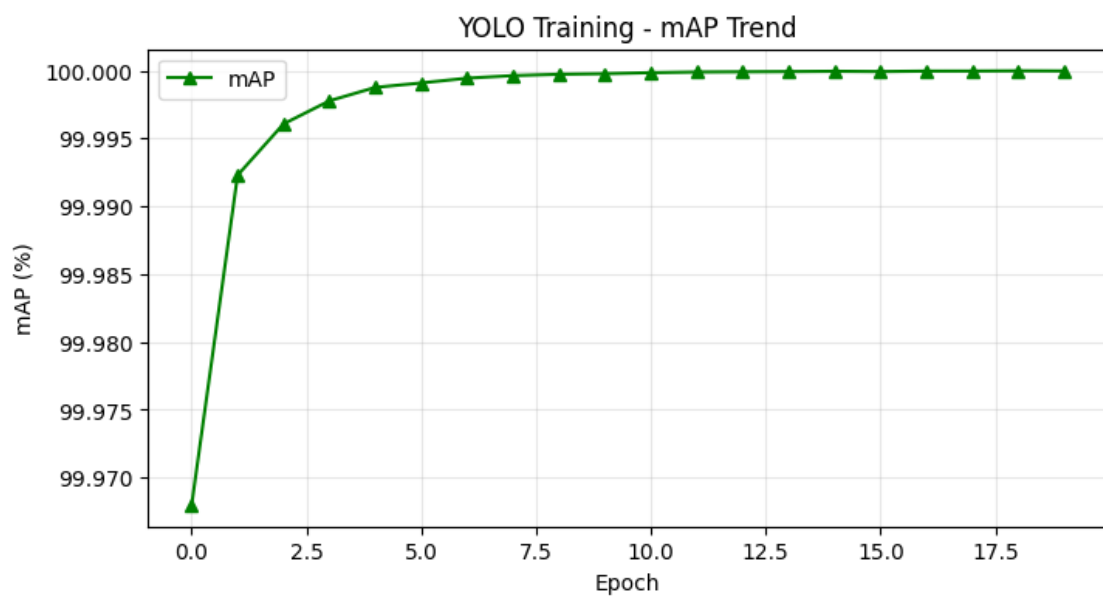
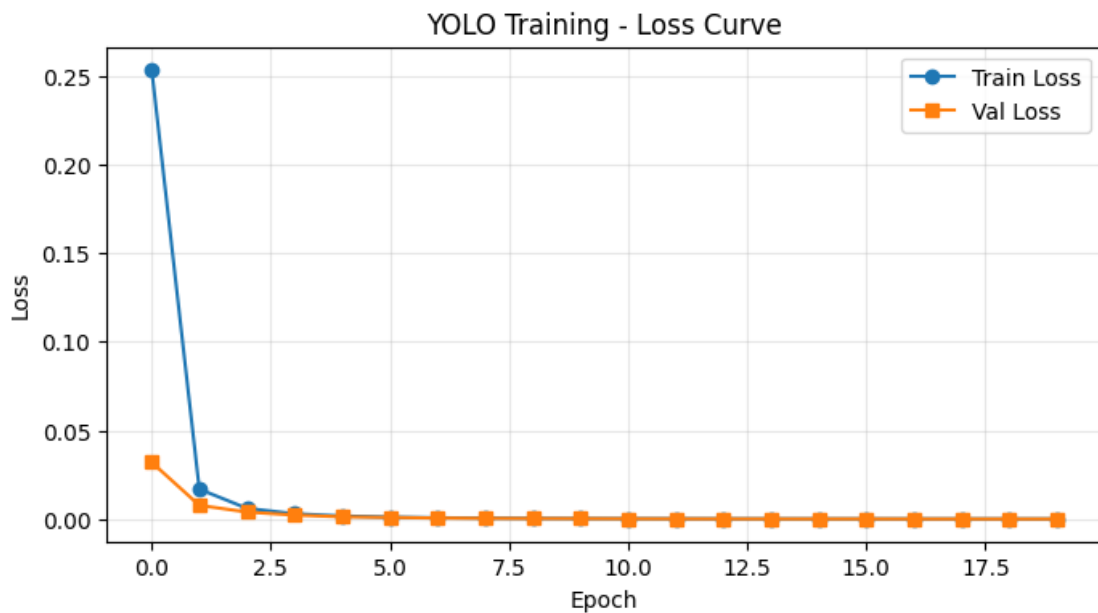
Epoch [7/10] | TrainLoss=0.0006 | ValLoss=0.0004

Epoch [8/10] | TrainLoss=0.0004 | ValLoss=0.0003

Epoch [9/10] | TrainLoss=0.0003 | ValLoss=0.0002

Epoch [10/10] | TrainLoss=0.0002 | ValLoss=0.0002

[112]: `plot_yolo_training(train_losses, val_losses, ap_values)`



```
[129]: # --- YOLO helpers: decode head -> boxes, then draw ---
import torch, torchvision, cv2, numpy as np, matplotlib.pyplot as plt

VOC_TINY_ANCHORS = [
    [(10,13),(16,30),(33,23)], # head @ 13x13 (stride 32)
    [(30,61),(62,45),(59,119)] # head @ 26x26 (stride 16)
```

```

]
VOC_TINY_STRIDES = [32, 16]

def decode_yolo_output(det, anchors, stride, num_classes, conf_thresh=0.35):
    """
    det: [B, A*(5+C), H, W] raw conv output (no activations applied)
    returns per-batch lists of (boxes[N,4], scores[N], cls_ids[N]) in
    ↪ input-pixel scale
    """
    B, _, H, W = det.shape
    A = len(anchors)

    # [B, A, 5+C, H, W] -> [B, A, H, W, 5+C]
    det = det.view(B, A, 5+num_classes, H, W).permute(0,1,3,4,2).contiguous()

    # grid (H,W,2)
    yy, xx = torch.meshgrid(torch.arange(H, device=det.device),
                             torch.arange(W, device=det.device), indexing="ij")
    grid = torch.stack((xx, yy), dim=-1) # (H,W,2)

    # anchors (A,1,1,2)
    anc = torch.tensor(anchors, dtype=torch.float32, device=det.device).
    ↪ view(A,1,1,2)

    out_boxes, out_scores, out_cls = [], [], []
    for b in range(B):
        d = det[b] # [A,H,W,5+C]

        tx = d[..., 0]; ty = d[..., 1]
        tw = d[..., 2]; th = d[..., 3]
        to = d[..., 4]
        tc = d[..., 5:] # [A,H,W,C]

        # YOLO decode to input-pixel scale
        cx = (torch.sigmoid(tx) + grid[..., 0]) * stride # [A,H,W]
        cy = (torch.sigmoid(ty) + grid[..., 1]) * stride
        pw = torch.exp(tw) * anc[..., 0] # [A,H,W]
        ph = torch.exp(th) * anc[..., 1]

        # objectness * best class prob
        obj = torch.sigmoid(to)
        cls_scores = torch.sigmoid(tc) # [A,H,W,C]
        cls_conf, cls_id = cls_scores.max(dim=-1) # [A,H,W]
        score = obj * cls_conf

    mask = score > conf_thresh
    if mask.sum() == 0:

```



```

        out_boxes.append(torch.empty((0,4), device=det.device))
        out_scores.append(torch.empty((0,), device=det.device))
        out_cls.append(torch.empty((0,), dtype=torch.long, device=det.
→device))

        continue

    cx = cx[mask]; cy = cy[mask]; pw = pw[mask]; ph = ph[mask]
    x1 = cx - pw/2; y1 = cy - ph/2; x2 = cx + pw/2; y2 = cy + ph/2
    boxes = torch.stack([x1,y1,x2,y2], dim=1)

    out_boxes.append(boxes)
    out_scores.append(score[mask])
    out_cls.append(cls_id[mask].long())

    return out_boxes, out_scores, out_cls

def draw_boxes(img_rgb01, boxes, scores, cls_ids, class_names, max_det_draw=50):
    """
    Draw bounding boxes + labels clearly on an RGB [0..1] image.
    """
    img = (img_rgb01 * 255).astype(np.uint8).copy()
    N = min(len(boxes), max_det_draw)
    if N == 0:
        return img

    for i in range(N):
        x1, y1, x2, y2 = boxes[i].round().to(torch.int32).tolist()
        x1 = max(0, x1); y1 = max(0, y1)
        x2 = max(0, x2); y2 = max(0, y2)
        color = (0, 255, 0)

        # Draw rectangle
        cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)

        # ---- Label text ----
        cls_name = class_names[int(cls_ids[i])] if int(cls_ids[i]) <
→len(class_names) else str(int(cls_ids[i]))
        score = float(scores[i])
        label = f"{cls_name}: {score:.2f}"

        # text size & background box
        (tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 2)
        th = int(th * 1.2)
        cv2.rectangle(img, (x1, y1 - th - 4), (x1 + tw + 2, y1), color, -1)

        # white text with black border for readability

```

```

        cv2.putText(img, label, (x1 + 1, y1 - 6),
                     cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 0), 3, cv2.LINE_AA)
        cv2.putText(img, label, (x1 + 1, y1 - 6),
                     cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 1, cv2.
↪LINE_AA)
    return img

import torch, torchvision, cv2, numpy as np, matplotlib.pyplot as plt

def show_yolo_predictions(model, dataloader, class_names, device,
                          num_images=6, conf_thresh=0.5, nms_thresh=0.45,
                          anchors=[[ (10,13), (16,30), (33,23) ], ↪
↪[ (30,61), (62,45), (59,119) ]],
                          strides=[32,16], max_det=20):
    """
    Show YOLOv3-Tiny predictions with clean boxes + single label per detection.
    """
    model.eval()
    shown = 0
    cols = 3
    rows = int(np.ceil(num_images / cols))
    plt.figure(figsize=(5*cols, 5.5*rows))

    with torch.no_grad():
        for imgs, _ in dataloader:
            imgs_t = torch.stack(imgs).to(device)
            preds = model(imgs_t)
            B = imgs_t.size(0)

            for b in range(B):
                if shown >= num_images:
                    plt.tight_layout(); plt.show(); return

                img_np = imgs_t[b].cpu().permute(1,2,0).numpy()
                img_np = np.clip(img_np, 0, 1)
                H_img, W_img = img_np.shape[:2]
                img_disp = (img_np*255).astype(np.uint8).copy()

                all_boxes, all_scores, all_cls = [], [], []

                # --- Decode each YOLO head ---
                for head_i, det in enumerate(preds):
                    boxes_b, scores_b, cls_b = decode_yolo_output(

```

```

        det, anchors[head_i], strides[head_i], model.
↪num_classes, conf_thresh
    )
    boxes_b, scores_b, cls_b = boxes_b[b], scores_b[b], cls_b[b]
    if len(boxes_b) == 0:
        continue
    keep = torchvision.ops.nms(boxes_b, scores_b, nms_thresh)
    all_boxes.append(boxes_b[keep])
    all_scores.append(scores_b[keep])
    all_cls.append(cls_b[keep])

ax = plt.subplot(rows, cols, shown+1)
ax.axis("off")

if len(all_boxes) == 0:
    ax.imshow(img_np)
    ax.set_title("No detection", color="gray", fontsize=10)
    shown += 1
    continue

boxes = torch.cat(all_boxes)
scores = torch.cat(all_scores)
cls_ids = torch.cat(all_cls)

# Global NMS + keep top detections only
keep = torchvision.ops.nms(boxes, scores, nms_thresh)
boxes, scores, cls_ids = boxes[keep], scores[keep], ↵
↪cls_ids[keep]

if len(boxes) > max_det:
    topk = torch.topk(scores, k=max_det).indices
    boxes, scores, cls_ids = boxes[topk], scores[topk], ↵
↪cls_ids[topk]

# --- Draw final detections ---
for (x1, y1, x2, y2), s, c in zip(boxes, scores, cls_ids):
    if s < conf_thresh: # skip weak
        continue
    x1, y1, x2, y2 = map(int, [x1, y1, x2, y2])
    color = (0,255,0)
    cv2.rectangle(img_disp, (x1,y1), (x2,y2), color, 2)
    label = f"{class_names[int(c)]}: {s:.2f}"
    (tw, th), _ = cv2.getTextSize(label, cv2.
↪FONT_HERSHEY_SIMPLEX, 0.6, 2)
    cv2.rectangle(img_disp, (x1, y1 - th - 4), (x1 + tw + 2, ↵
↪y1), color, -1)
    cv2.putText(img_disp, label, (x1+1, y1-6),

```

```

cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,0,0), 3, cv2.
↪LINE_AA)
        cv2.putText(img_disp, label, (x1+1, y1-6),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255,255,255),
↪1, cv2.LINE_AA)

        ax.imshow(cv2.cvtColor(img_disp, cv2.COLOR_BGR2RGB))

        # --- Clean summary text below image ---
        top_labels = [f"{class_names[int(c)]} ({s:.2f})"
                    for c, s in sorted(zip(cls_ids, scores),
                                        key=lambda x: float(x[1]),
↪reverse=True)[:5]]
        ax.set_title("\n".join(top_labels), fontsize=9, color="green")
        shown += 1

plt.tight_layout(); plt.show()

```

```

[104]: import os, torch
from ultralytics import YOLO

def save_model(model, name="model", folder="saved_models"):
    """
    Save either a standard PyTorch model (state_dict) or an Ultralytics YOLO
↪model (.pt)
    """
    os.makedirs(folder, exist_ok=True)
    path = os.path.join(folder, f"{name}.pt")

    if isinstance(model, YOLO):
        # Ultralytics YOLO model
        model.save(path)
        print(f" Ultralytics YOLO model saved to {path}")
    else:
        # Regular PyTorch model
        torch.save(model.state_dict(), path)
        print(f" PyTorch model saved to {path}")

    return path

def load_model(path, model_class=None, device=None):
    """
    Load a model from path - works for Ultralytics YOLO or PyTorch models.

    Args:
        path (str): Path to the saved .pt model
    """

```

```

        model_class: (optional) PyTorch class for custom models (e.g., 
↳ TinyYOLOv3)
        device: Torch device (default: auto-detect GPU)
    """
    device = device or ("cuda" if torch.cuda.is_available() else "cpu")

    # If it's an Ultralytics YOLO model
    if os.path.basename(path).startswith("yolo") or "runs/" in path:
        print(f" Loading Ultralytics YOLO model from {path}")
        model = YOLO(path)
        model.to(device)
        print(" YOLO model loaded successfully.")
        return model

    # Otherwise, assume a custom PyTorch model
    elif model_class is not None:
        print(f" Loading PyTorch model from {path}")
        model = model_class()
        model.load_state_dict(torch.load(path, map_location=device))
        model.to(device)
        print(" PyTorch model loaded successfully.")
        return model

    else:
        raise ValueError(
            " model_class is required for loading custom PyTorch models, 
↳ (non-YOLO)."
        )

```

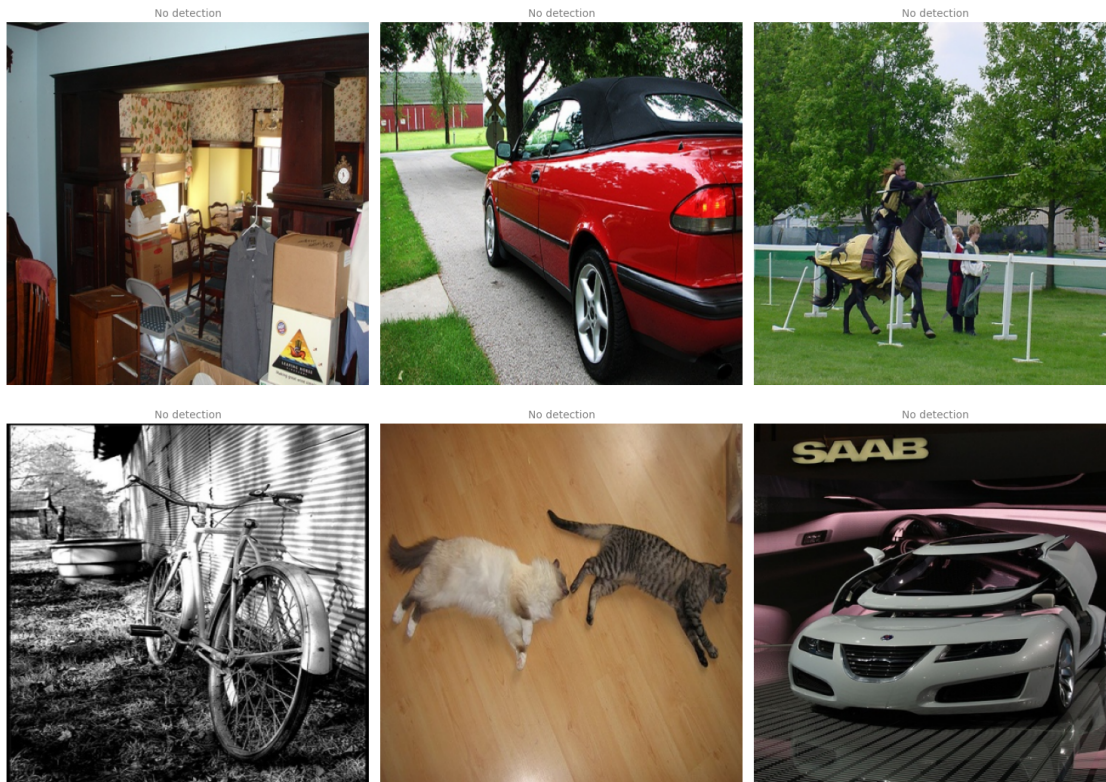
```
[105]: save_model(model, name="yolo_custom", folder="saved_models")
```

PyTorch model saved to saved_models/yolo_custom.pt

```
[105]: 'saved_models/yolo_custom.pt'
```

```
[130]: # model = YOLO("saved_models/yolo_custom.pt")
# model = load_model("saved_models/yolo_custom.pt")
show_yolo_predictions(model, valloader, class_names=VOC_CLASSES, device=device, 
↳ num_images=6)

```



```
[132]: show_yolo_predictions(model, valloader, class_names=VOC_CLASSES,
                             device=device, num_images=6, conf_thresh=0.25,
                             ↪nms_thresh=0.45)
```




[24]: `!pip install ultralytics`

Collecting ultralytics

Downloading ultralytics-8.3.224-py3-none-any.whl.metadata (37 kB)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2)
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (4.12.0.88)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.16.3)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.8.0+cu126)
Requirement already satisfied: torchvision>=0.9.0 in

```

/usr/local/lib/python3.12/dist-packages (from ultralytics) (0.23.0+cu126)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages
(from ultralytics) (5.9.5)
Requirement already satisfied: polars in /usr/local/lib/python3.12/dist-packages
(from ultralytics) (1.25.2)
Collecting ultralytics-thop>=2.0.18 (from ultralytics)
  Downloading ultralytics_thop-2.0.18-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics)
(1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-
packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics)
(4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics)
(1.4.9)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics)
(25.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics)
(3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics)
(2.9.0.post0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics)
(3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
packages (from requests>=2.23.0->ultralytics) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics)
(2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics)
(2025.10.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
packages (from torch>=1.8.0->ultralytics) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
(4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-
packages (from torch>=1.8.0->ultralytics) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-
packages (from torch>=1.8.0->ultralytics) (1.13.3)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-

```


packages (from torch>=1.8.0->ultralytics) (3.5)
 Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
 (from torch>=1.8.0->ultralytics) (3.1.6)
 Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages
 (from torch>=1.8.0->ultralytics) (2025.3.0)
 Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.6.77)
 Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.6.77)
 Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.6.80)
 Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (9.10.2.21)
 Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.6.4.1)
 Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (11.3.0.4)
 Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (10.3.7.77)
 Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (11.7.1.2)
 Requirement already satisfied: nvidia-cuspars-cu12==12.5.4.2 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.5.4.2)
 Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (0.7.1)
 Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (2.27.3)
 Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.6.77)
 Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.6.85)
 Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (1.11.1.6)
 Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-
 packages (from torch>=1.8.0->ultralytics) (3.4.0)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib>=3.3.0->ultralytics) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.8.0->ultralytics) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.8.0->ultralytics) (3.0.3)

Downloading ultralytics-8.3.224-py3-none-any.whl (1.1 MB)

1.1/1.1 MB

24.6 MB/s eta 0:00:00

Downloading ultralytics_thop-2.0.18-py3-none-any.whl (28 kB)

Installing collected packages: ultralytics-thop, ultralytics

Successfully installed ultralytics-8.3.224 ultralytics-thop-2.0.18

```
[25]: # def save_model(model, name="model", folder="saved_models"):
#     os.makedirs(folder, exist_ok=True)
#     path = os.path.join(folder, f"{name}.pth")
#     torch.save(model.state_dict(), path)
#     print(f" Model saved to {path}")

# def load_model(model_class, name="model", folder="saved_models",
# ↪map_location=None):
#     path = os.path.join(folder, f"{name}.pth")
#     model = model_class()
#     model.load_state_dict(torch.load(path, map_location=map_location or ↪
# ↪device))
#     model.to(device)
#     print(f" Model loaded from {path}")
#     return model
```

```
[27]: # def save_yolo_model(model,path="yolov3_custom_voc2012.pt"):
#     torch.save(model.state_dict(),path); print(f" Saved to {path}")
# def load_yolo_model(model_class,path,num_classes,device):
#     m=model_class(num_classes=num_classes)
#     m.load_state_dict(torch.load(path,map_location=device))
#     m.to(device).eval(); print(f" Loaded from {path}"); return m

save_model(model,name="yolov3_custom_voc2007.pt")
```

PyTorch model saved to saved_models/yolov3_custom_voc2007.pt.pt

```
[27]: 'saved_models/yolov3_custom_voc2007.pt.pt'
```

0.1 Exercise 2 ultralytic

```
[28]: from ultralytics import YOLO
import os, yaml
import matplotlib.pyplot as plt
```

```
[29]: !apt-get install tree
      !tree -f -L 3 /content/DLCV_Lab_Dataset/
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
tree is already the newest version (2.0.2-1).
0 upgraded, 0 newly installed, 0 to remove and 41 not upgraded.
/content/DLCV_Lab_Dataset
  /content/DLCV_Lab_Dataset/pascal_voc
    /content/DLCV_Lab_Dataset/pascal_voc/PASCAL_VOC

/content/DLCV_Lab_Dataset/pascal_voc/PASCAL_VOC/pascal_test2007.json

/content/DLCV_Lab_Dataset/pascal_voc/PASCAL_VOC/pascal_train2007.json

/content/DLCV_Lab_Dataset/pascal_voc/PASCAL_VOC/pascal_train2012.json

/content/DLCV_Lab_Dataset/pascal_voc/PASCAL_VOC/pascal_val2007.json

/content/DLCV_Lab_Dataset/pascal_voc/PASCAL_VOC/pascal_val2012.json
  /content/DLCV_Lab_Dataset/PASCAL_VOC
    /content/DLCV_Lab_Dataset/PASCAL_VOC/PASCAL_VOC

/content/DLCV_Lab_Dataset/PASCAL_VOC/PASCAL_VOC/pascal_test2007.json

/content/DLCV_Lab_Dataset/PASCAL_VOC/PASCAL_VOC/pascal_train2007.json

/content/DLCV_Lab_Dataset/PASCAL_VOC/PASCAL_VOC/pascal_train2012.json

/content/DLCV_Lab_Dataset/PASCAL_VOC/PASCAL_VOC/pascal_val2007.json

/content/DLCV_Lab_Dataset/PASCAL_VOC/PASCAL_VOC/pascal_val2012.json
  /content/DLCV_Lab_Dataset/raw_zips
    /content/DLCV_Lab_Dataset/raw_zips/pascal-voc-2007.zip
  /content/DLCV_Lab_Dataset/voc_test_06-nov-2007
    /content/DLCV_Lab_Dataset/voc_test_06-nov-2007/VOCdevkit

/content/DLCV_Lab_Dataset/voc_test_06-nov-2007/VOCdevkit/VOC2007
  /content/DLCV_Lab_Dataset/VOC_test_06-Nov-2007
    /content/DLCV_Lab_Dataset/VOC_test_06-Nov-2007/VOCdevkit
```

```

/content/DLCV_Lab_Dataset/VOCtest_06-Nov-2007/VOCdevkit/VOC2007
/content/DLCV_Lab_Dataset/voctrainval_06-nov-2007
/content/DLCV_Lab_Dataset/voctrainval_06-nov-2007/VOCdevkit

/content/DLCV_Lab_Dataset/voctrainval_06-nov-2007/VOCdevkit/VOC2007
/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007
/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007/VOCdevkit

/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007/VOCdevkit/VOC2007

```

17 directories, 11 files

```
[30]: !tree -f -L 2 /content/DLCV_Lab_Dataset/VOC2012_train_val
```

```
/content/DLCV_Lab_Dataset/VOC2012_train_val [error opening dir]
```

0 directories, 0 files

```
[31]: import os
from pathlib import Path

# Source Pascal VOC 2007 roots
trainval_root = "/content/DLCV_Lab_Dataset/VOCtrainval_06-Nov-2007/VOCdevkit/
↳VOC2007"
test_root = "/content/DLCV_Lab_Dataset/VOCtest_06-Nov-2007/VOCdevkit/VOC2007"

# Output root for YOLOv11 dataset
yolo_root = "/content/DLCV_Lab_Dataset/VOC2007_yolo"
Path(yolo_root).mkdir(parents=True, exist_ok=True)
print(f" YOLO output directory: {yolo_root}")
```

YOLO output directory: /content/DLCV_Lab_Dataset/VOC2007_yolo

```
[32]: VOC_CLASSES = [
    "aeroplane", "bicycle", "bird", "boat", "bottle",
    "bus", "car", "cat", "chair", "cow", "diningtable",
    "dog", "horse", "motorbike", "person", "pottedplant",
    "sheep", "sofa", "train", "tvmonitor"
]
CLASS_TO_ID = {cls: i for i, cls in enumerate(VOC_CLASSES)}
```

```
[33]: def read_split_list(voc_root, split_name):
    path = os.path.join(voc_root, "ImageSets/Main", f"{split_name}.txt")
    with open(path, "r") as f:
        ids = [x.strip() for x in f.readlines() if x.strip()]
    return ids

train_ids = read_split_list(trainval_root, "train")
```

```
val_ids = read_split_list(trainval_root, "val")

print(f" Found {len(train_ids)} train and {len(val_ids)} val IDs")
```

Found 2501 train and 2510 val IDs

```
[34]: for split in ["train", "val", "test"]:
        for sub in ["images", "labels"]:
            Path(os.path.join(yolo_root, sub, split)).mkdir(parents=True,
↪exist_ok=True)
print(" Created YOLO folder structure.")
```

Created YOLO folder structure.

```
[35]: import xml.etree.ElementTree as ET
import shutil
from tqdm import tqdm

def voc_to_yolo(xml_path, txt_path, class_map):
    tree = ET.parse(xml_path)
    root = tree.getroot()
    w = int(root.find("size/width").text)
    h = int(root.find("size/height").text)
    lines = []
    for obj in root.findall("object"):
        cls_name = obj.find("name").text
        if cls_name not in class_map:
            continue
        cls_id = class_map[cls_name]
        box = obj.find("bndbox")
        xmin = float(box.find("xmin").text)
        ymin = float(box.find("ymin").text)
        xmax = float(box.find("xmax").text)
        ymax = float(box.find("ymax").text)
        x_center = ((xmin + xmax) / 2.0) / w
        y_center = ((ymin + ymax) / 2.0) / h
        width = (xmax - xmin) / w
        height = (ymax - ymin) / h
        lines.append(f"{cls_id} {x_center:.6f} {y_center:.6f} {width:.6f}
↪{height:.6f}")
    if lines:
        with open(txt_path, "w") as f:
            f.write("\n".join(lines))
```

```
[36]: def process_split(ids, split_name, voc_root, yolo_root):
        img_src = os.path.join(voc_root, "JPEGImages")
        xml_src = os.path.join(voc_root, "Annotations")
        img_dst = os.path.join(yolo_root, "images", split_name)
```

```

lbl_dst = os.path.join(yolo_root, "labels", split_name)

for img_id in tqdm(ids, desc=f"Processing {split_name}"):
    src_img = os.path.join(img_src, img_id + ".jpg")
    src_xml = os.path.join(xml_src, img_id + ".xml")
    dst_img = os.path.join(img_dst, img_id + ".jpg")
    dst_lbl = os.path.join(lbl_dst, img_id + ".txt")

    if os.path.exists(src_img):
        shutil.copy(src_img, dst_img)
    if os.path.exists(src_xml):
        voc_to_yolo(src_xml, dst_lbl, CLASS_TO_ID)

```

```

[37]: process_split(train_ids, "train", trainval_root, yolo_root)
      process_split(val_ids, "val", trainval_root, yolo_root)

```

```

Processing train: 100%|          | 2501/2501 [00:01<00:00, 1980.75it/s]
Processing val: 100%|          | 2510/2510 [00:01<00:00, 1296.06it/s]

```

```

[38]: def process_test(voc_root, yolo_root):
    img_src = os.path.join(voc_root, "JPEGImages")
    xml_src = os.path.join(voc_root, "Annotations")
    img_dst = os.path.join(yolo_root, "images", "test")
    lbl_dst = os.path.join(yolo_root, "labels", "test")

    for xml in tqdm(os.listdir(xml_src), desc="Processing test"):
        if not xml.endswith(".xml"):
            continue
        img_id = os.path.splitext(xml)[0]
        src_img = os.path.join(img_src, img_id + ".jpg")
        src_xml = os.path.join(xml_src, xml)
        dst_img = os.path.join(img_dst, img_id + ".jpg")
        dst_lbl = os.path.join(lbl_dst, img_id + ".txt")
        if os.path.exists(src_img):
            shutil.copy(src_img, dst_img)
        voc_to_yolo(src_xml, dst_lbl, CLASS_TO_ID)

    process_test(test_root, yolo_root)
    print(" Test set processed and converted.")

```

```

Processing test: 100%|          | 4952/4952 [00:06<00:00, 755.09it/s]

Test set processed and converted.

```

```

[39]: for split in ["train", "val", "test"]:
    imgs = len(os.listdir(os.path.join(yolo_root, "images", split)))
    lbls = len(os.listdir(os.path.join(yolo_root, "labels", split)))

```

```
print(f"{split.upper():<5}: {imgs:>5} images, {lbls:>5} labels")
```

```
TRAIN: 2501 images, 2501 labels  
VAL : 2510 images, 2510 labels  
TEST : 4952 images, 4952 labels
```

```
[40]: import yaml
```

```
yaml_dict = {  
    "train": os.path.join(yolo_root, "images/train"),  
    "val": os.path.join(yolo_root, "images/val"),  
    "test": os.path.join(yolo_root, "images/test"),  
    "nc": len(VOC_CLASSES),  
    "names": VOC_CLASSES  
}
```

```
yaml_path = os.path.join(yolo_root, "voc2007.yaml")  
with open(yaml_path, "w") as f:  
    yaml.dump(yaml_dict, f)  
print(f" Dataset YAML created at: {yaml_path}")
```

```
Dataset YAML created at: /content/DLCV_Lab_Dataset/VOC2007_yolo/voc2007.yaml
```

```
[41]: for split in ["train", "val", "test"]:  
    imgs = len(os.listdir(os.path.join(yolo_root, "images", split)))  
    lbls = len(os.listdir(os.path.join(yolo_root, "labels", split)))  
    print(f"{split.upper():<5}: {imgs:>5} images, {lbls:>5} labels")
```

```
TRAIN: 2501 images, 2501 labels  
VAL : 2510 images, 2510 labels  
TEST : 4952 images, 4952 labels
```

```
[42]: !pip install ultralytics
```

```
Requirement already satisfied: ultralytics in /usr/local/lib/python3.12/dist-packages (8.3.224)  
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2)  
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)  
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (4.12.0.88)  
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)  
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3)  
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)  
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-
```

packages (from ultralytics) (1.16.3)
 Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.8.0+cu126)
 Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (0.23.0+cu126)
 Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from ultralytics) (5.9.5)
 Requirement already satisfied: polars in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.25.2)
 Requirement already satisfied: ultralytics-thop>=2.0.18 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.18)
 Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.3.3)
 Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (4.60.1)
 Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.4.9)
 Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (25.0)
 Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (3.2.5)
 Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (2.9.0.post0)
 Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.4.4)
 Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.11)
 Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2.5.0)
 Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2025.10.5)
 Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.20.0)
 Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (4.15.0)
 Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-

packages (from torch>=1.8.0->ultralytics) (75.2.0)
 Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (1.13.3)
 Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.5)
 Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.1.6)
 Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (2025.3.0)
 Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.6.77)
 Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.6.77)
 Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.6.80)
 Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (9.10.2.21)
 Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.6.4.1)
 Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (11.3.0.4)
 Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (10.3.7.77)
 Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (11.7.1.2)
 Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.5.4.2)
 Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (0.7.1)
 Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (2.27.3)
 Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.6.77)
 Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.6.85)
 Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in

```

/usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
(1.11.1.6)
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-
packages (from torch>=1.8.0->ultralytics) (3.4.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.7->matplotlib>=3.3.0->ultralytics) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from
sympy>=1.13.3->torch>=1.8.0->ultralytics) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.8.0->ultralytics)
(3.0.3)

```

```
[43]: from ultralytics import YOLO
```

```

model = YOLO("yolo11n.pt") # or yolo11n-seg.pt if segmentation
model.train(
    data=yaml_path,
    epochs=50,
    imgsz=640,
    batch=16,
    device=0 if torch.cuda.is_available() else 'cpu'
)

```

Downloading

```

https://github.com/ultralytics/assets/releases/download/v8.3.0/yolo11n.pt to
'yolo11n.pt': 100%          5.4MB 96.0MB/s 0.1s

```

```

Ultralytics 8.3.224 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4,
15095MiB)

```

```

engine/trainer: agnostic_nms=False, amp=True, augment=False,
auto_augment=randaugument, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None,
classes=None, close_mosaic=10, cls=0.5, compile=False, conf=None,
copy_paste=0.0, copy_paste_mode=flip, cos_lr=False, cutmix=0.0,
data=/content/DLCV_Lab_Dataset/VOC2007_yolo/voc2007.yaml, degrees=0.0,
deterministic=True, device=0, dfl=1.5, dnn=False, dropout=0.0, dynamic=False,
embed=None, epochs=50, erasing=0.4, exist_ok=False, fliplr=0.5, flipud=0.0,
format=torchscript, fraction=1.0, freeze=None, half=False, hsv_h=0.015,
hsv_s=0.7, hsv_v=0.4, imgsz=640, int8=False, iou=0.7, keras=False, kobj=1.0,
line_width=None, lr0=0.01, lrf=0.01, mask_ratio=4, max_det=300, mixup=0.0,
mode=train, model=yolo11n.pt, momentum=0.937, mosaic=1.0, multi_scale=False,
name=train, nbs=64, nms=False, opset=None, optimize=False, optimizer=auto,
overlap_mask=True, patience=100, perspective=0.0, plots=True, pose=12.0,
pretrained=True, profile=False, project=None, rect=False, resume=False,
retina_masks=False, save=True, save_conf=False, save_crop=False,
save_dir=/content/runs/detect/train, save_frames=False, save_json=False,
save_period=-1, save_txt=False, scale=0.5, seed=0, shear=0.0, show=False,
show_boxes=True, show_conf=True, show_labels=True, simplify=True,
single_cls=False, source=None, split=val, stream_buffer=False, task=detect,

```

time=None, tracker=botssort.yaml, translate=0.1, val=True, verbose=True,
vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0,
warmup_momentum=0.8, weight_decay=0.0005, workers=8, workspace=None
Downloading https://ultralytics.com/assets/Arial.ttf to
'/root/.config/Ultralytics/Arial.ttf': 100% 755.1KB 26.8MB/s 0.0s
Overriding model.yaml nc=80 with nc=20

	from	n	params	module
arguments				
0	-1	1	464	ultralytics.nn.modules.conv.Conv
[3, 16, 3, 2]				
1	-1	1	4672	ultralytics.nn.modules.conv.Conv
[16, 32, 3, 2]				
2	-1	1	6640	ultralytics.nn.modules.block.C3k2
[32, 64, 1, False, 0.25]				
3	-1	1	36992	ultralytics.nn.modules.conv.Conv
[64, 64, 3, 2]				
4	-1	1	26080	ultralytics.nn.modules.block.C3k2
[64, 128, 1, False, 0.25]				
5	-1	1	147712	ultralytics.nn.modules.conv.Conv
[128, 128, 3, 2]				
6	-1	1	87040	ultralytics.nn.modules.block.C3k2
[128, 128, 1, True]				
7	-1	1	295424	ultralytics.nn.modules.conv.Conv
[128, 256, 3, 2]				
8	-1	1	346112	ultralytics.nn.modules.block.C3k2
[256, 256, 1, True]				
9	-1	1	164608	ultralytics.nn.modules.block.SPPF
[256, 256, 5]				
10	-1	1	249728	ultralytics.nn.modules.block.C2PSA
[256, 256, 1]				
11	-1	1	0	torch.nn.modules.upsampling.Upsample
[None, 2, 'nearest']				
12	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat
[1]				
13	-1	1	111296	ultralytics.nn.modules.block.C3k2
[384, 128, 1, False]				
14	-1	1	0	torch.nn.modules.upsampling.Upsample
[None, 2, 'nearest']				
15	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat
[1]				
16	-1	1	32096	ultralytics.nn.modules.block.C3k2
[256, 64, 1, False]				
17	-1	1	36992	ultralytics.nn.modules.conv.Conv
[64, 64, 3, 2]				
18	[-1, 13]	1	0	ultralytics.nn.modules.conv.Concat
[1]				
19	-1	1	86720	ultralytics.nn.modules.block.C3k2

```

[192, 128, 1, False]
 20                -1  1    147712  ultralytics.nn.modules.conv.Conv
[128, 128, 3, 2]
 21            [-1, 10]  1          0  ultralytics.nn.modules.conv.Concat
[1]
 22                -1  1    378880  ultralytics.nn.modules.block.C3k2
[384, 256, 1, True]
 23            [16, 19, 22]  1    434572  ultralytics.nn.modules.head.Detect
[20, [64, 128, 256]]
YOLO11n summary: 181 layers, 2,593,740 parameters, 2,593,724 gradients, 6.5
GFLOPs

```

```

Transferred 448/499 items from pretrained weights
Freezing layer 'model.23.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: checks passed
train: Fast image access  (ping: 0.0±0.0 ms, read: 1760.0±606.9
MB/s, size: 100.1 KB)
train: Scanning
/content/DLCV_Lab_Dataset/VOC2007_yolo/labels/train... 2501 images, 0
backgrounds, 0 corrupt: 100%          2501/2501 916.7it/s 2.7s
train: New cache created:
/content/DLCV_Lab_Dataset/VOC2007_yolo/labels/train.cache
augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01,
blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average',
num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8,
8))
val: Fast image access  (ping: 0.0±0.0 ms, read: 838.1±581.8 MB/s,
size: 79.5 KB)
val: Scanning
/content/DLCV_Lab_Dataset/VOC2007_yolo/labels/val... 2510 images, 0 backgrounds,
0 corrupt: 100%          2510/2510 702.8it/s 3.6s
val: New cache created:
/content/DLCV_Lab_Dataset/VOC2007_yolo/labels/val.cache
Plotting labels to /content/runs/detect/train/labels.jpg...
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and
'momentum=0.937' and determining best 'optimizer', 'lr0' and 'momentum'
automatically...
optimizer: AdamW(lr=0.000417, momentum=0.9) with parameter groups
81 weight(decay=0.0), 88 weight(decay=0.0005), 87 bias(decay=0.0)
Image sizes 640 train, 640 val
Using 2 dataloader workers
Logging results to /content/runs/detect/train
Starting training for 50 epochs...

```

	Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
	1/50	2.5G	1.05	3.588	1.245	45	
640: 100%		157/157	2.5it/s	1:04			

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	1.5it/s	51.3s		
	all	2510	7818	0.519	0.314	0.363

0.256

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
2/50	3.16G	1.194	2.424	1.35	29	
640: 100%		157/157	3.5it/s	44.9s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.4it/s	17.9s		
	all	2510	7818	0.596	0.494	0.523

0.339

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
3/50	3.18G	1.184	2.071	1.365	28	
640: 100%		157/157	3.6it/s	44.1s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.6it/s	17.0s		
	all	2510	7818	0.586	0.486	0.508

0.312

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
4/50	3.19G	1.197	1.944	1.364	31	
640: 100%		157/157	3.6it/s	44.2s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.4it/s	18.1s		
	all	2510	7818	0.617	0.494	0.527

0.333

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
5/50	3.21G	1.176	1.878	1.357	32	
640: 100%		157/157	3.6it/s	43.1s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.7it/s	16.8s		
	all	2510	7818	0.592	0.507	0.529

0.331

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
6/50	3.23G	1.166	1.76	1.342	55	
640: 100%		157/157	3.6it/s	43.8s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.6it/s	17.1s		
	all	2510	7818	0.651	0.525	0.581

0.371

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
7/50	3.24G	1.157	1.708	1.338	38	
640: 100%		157/157	3.6it/s	43.9s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.5it/s	17.6s		
	all	2510	7818	0.662	0.536	0.598

0.385

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
8/50	3.25G	1.145	1.65	1.323	42	
640: 100%		157/157	3.6it/s	43.5s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.7it/s	17.0s		
	all	2510	7818	0.604	0.516	0.541

0.343

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
9/50	3.27G	1.109	1.597	1.31	30	
640: 100%		157/157	3.6it/s	43.2s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.7it/s	17.0s		
	all	2510	7818	0.634	0.534	0.589

0.379

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
10/50	3.29G	1.103	1.544	1.298	27	
640: 100%		157/157	3.6it/s	43.2s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.5it/s	17.4s		
	all	2510	7818	0.671	0.568	0.622

0.411

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
11/50	3.3G	1.102	1.489	1.293	38	
640: 100%		157/157	3.7it/s	42.9s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.7it/s	16.9s		
	all	2510	7818	0.691	0.549	0.62

0.403

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
12/50	3.31G	1.09	1.448	1.288	35	
640: 100%		157/157	3.6it/s	43.5s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.8it/s	16.5s		
	all	2510	7818	0.672	0.58	0.633

0.416

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
13/50	3.33G	1.065	1.418	1.28	33	
640: 100%		157/157	3.7it/s	42.6s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.5it/s	17.6s		
	all	2510	7818	0.709	0.563	0.64

0.427

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
14/50	3.35G	1.065	1.374	1.265	36	
640: 100%		157/157	3.7it/s	43.0s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.7it/s	16.7s		
	all	2510	7818	0.665	0.572	0.628

0.417

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
15/50	3.36G	1.038	1.353	1.257	26	
640: 100%		157/157	2.6it/s	1:00		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.5it/s	17.7s		
	all	2510	7818	0.696	0.572	0.642

0.425

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
16/50	3.37G	1.035	1.298	1.249	35	
640: 100%		157/157	3.4it/s	45.6s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.8it/s	16.6s		
	all	2510	7818	0.706	0.604	0.66

0.443

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
17/50	3.39G	1.028	1.271	1.246	20	
640: 100%		157/157	3.5it/s	45.1s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.7it/s	16.9s		
	all	2510	7818	0.668	0.575	0.634

0.411

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
18/50	3.4G	1.029	1.257	1.245	35	
640: 100%		157/157	3.6it/s	43.6s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.6it/s	17.2s		
	all	2510	7818	0.712	0.599	0.667

0.449

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
19/50	3.42G	1.02	1.221	1.236	23	
640: 100%		157/157	3.6it/s	43.5s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.8it/s 16.4s			
	all	2510	7818	0.717	0.581	0.657

0.441

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
20/50	3.43G	1.01	1.205	1.232	29	
640: 100%		157/157	3.6it/s 44.1s			

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.7it/s 16.8s			
	all	2510	7818	0.686	0.613	0.662

0.448

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
21/50	3.45G	1.004	1.175	1.217	42	
640: 100%		157/157	3.7it/s 42.5s			

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.6it/s 17.2s			
	all	2510	7818	0.703	0.612	0.675

0.458

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
22/50	3.46G	0.9694	1.145	1.214	24	
640: 100%		157/157	3.7it/s 42.2s			

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.9it/s 16.2s			
	all	2510	7818	0.732	0.602	0.672

0.453

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
23/50	3.48G	0.9924	1.137	1.209	28	
640: 100%		157/157	3.7it/s 42.7s			

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.6it/s 17.2s			
	all	2510	7818	0.724	0.607	0.675

0.453

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
24/50	3.48G	0.9759	1.105	1.2	22	
640: 100%		157/157	3.7it/s 42.2s			

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.8it/s 16.6s			
	all	2510	7818	0.721	0.613	0.676

0.459

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
25/50	3.51G	0.9694	1.086	1.195	45	
640: 100%		157/157	3.6it/s 43.9s			

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.8it/s	16.5s		
	all	2510	7818	0.713	0.62	0.676

0.459

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
26/50	3.53G	0.9632	1.073	1.192	18	
640: 100%		157/157	3.6it/s	43.6s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.5it/s	17.7s		
	all	2510	7818	0.705	0.627	0.683

0.467

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
27/50	3.54G	0.9593	1.075	1.192	29	
640: 100%		157/157	3.6it/s	43.1s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.8it/s	16.3s		
	all	2510	7818	0.716	0.629	0.689

0.471

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
28/50	3.54G	0.9431	1.046	1.181	30	
640: 100%		157/157	3.6it/s	44.1s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.7it/s	16.7s		
	all	2510	7818	0.706	0.63	0.693

0.472

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
29/50	3.57G	0.9396	1.002	1.172	40	
640: 100%		157/157	3.7it/s	42.3s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.6it/s	17.3s		
	all	2510	7818	0.725	0.647	0.702

0.481

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
30/50	3.58G	0.9316	0.9966	1.165	38	
640: 100%		157/157	3.7it/s	42.9s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.7it/s	16.7s		
	all	2510	7818	0.744	0.627	0.703

0.483

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
31/50	3.6G	0.9261	0.9983	1.165	36	
640: 100%		157/157	3.5it/s	44.9s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.9it/s	16.2s		
	all	2510	7818	0.738	0.635	0.704

0.49

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
32/50	3.61G	0.8905	0.9402	1.138	13	
640: 100%		157/157	3.7it/s	42.9s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.6it/s	17.3s		
	all	2510	7818	0.717	0.645	0.702

0.489

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
33/50	3.63G	0.913	0.9565	1.159	50	
640: 100%		157/157	3.7it/s	42.8s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.9it/s	16.3s		
	all	2510	7818	0.758	0.641	0.709

0.491

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
34/50	3.64G	0.9011	0.94	1.156	32	
640: 100%		157/157	3.6it/s	43.8s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.8it/s	16.4s		
	all	2510	7818	0.749	0.634	0.708

0.494

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
35/50	3.66G	0.8943	0.907	1.141	25	
640: 100%		157/157	3.6it/s	43.5s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.6it/s	17.0s		
	all	2510	7818	0.743	0.641	0.709

0.494

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
36/50	3.66G	0.8742	0.9077	1.143	31	
640: 100%		157/157	3.7it/s	43.0s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.9it/s	16.2s		
	all	2510	7818	0.738	0.648	0.711

0.493

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
37/50	3.69G	0.8864	0.8993	1.138	19	
640: 100%		157/157	3.6it/s	43.0s		

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.7it/s 16.8s			
	all	2510	7818	0.758	0.638	0.718

0.498

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
38/50	3.7G	0.8697	0.8775	1.134	25	
640: 100%		157/157	3.7it/s 42.9s			

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.8it/s 16.4s			
	all	2510	7818	0.742	0.663	0.716

0.499

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
39/50	3.71G	0.8716	0.8846	1.135	26	
640: 100%		157/157	3.6it/s 43.4s			

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.8it/s 16.3s			
	all	2510	7818	0.767	0.638	0.718

0.5

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
40/50	3.72G	0.8528	0.8638	1.124	33	
640: 100%		157/157	3.7it/s 42.9s			

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.6it/s 17.2s			
	all	2510	7818	0.755	0.651	0.722

0.503

Closing dataloader mosaic

alumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8, 8))

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
41/50	3.74G	0.8232	0.778	1.087	27	
640: 100%		157/157	3.6it/s 43.3s			

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.8it/s 16.4s			
	all	2510	7818	0.711	0.645	0.697

0.482

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
42/50	3.76G	0.7992	0.712	1.074	21	
640: 100%		157/157	3.7it/s 42.3s			

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.7it/s 16.8s			
	all	2510	7818	0.755	0.628	0.703

0.49

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
43/50	3.78G	0.7924	0.6991	1.063	7	
640: 100%	157/157	3.8it/s	41.6s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	79/79	4.9it/s	16.2s			
	all	2510	7818	0.748	0.637	0.711

0.497

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
44/50	3.78G	0.778	0.6781	1.058	11	
640: 100%	157/157	3.7it/s	42.1s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	79/79	4.7it/s	16.7s			
	all	2510	7818	0.741	0.645	0.712

0.499

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
45/50	3.8G	0.7833	0.6708	1.057	29	
640: 100%	157/157	3.8it/s	41.5s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	79/79	4.9it/s	16.2s			
	all	2510	7818	0.734	0.653	0.716

0.501

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
46/50	3.82G	0.7558	0.6428	1.036	20	
640: 100%	157/157	3.8it/s	41.9s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	79/79	4.7it/s	16.7s			
	all	2510	7818	0.737	0.66	0.716

0.502

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
47/50	3.84G	0.7462	0.6346	1.032	13	
640: 100%	157/157	3.8it/s	41.6s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	79/79	4.9it/s	16.2s			
	all	2510	7818	0.732	0.661	0.718

0.504

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
48/50	3.84G	0.7479	0.6285	1.034	10	
640: 100%	157/157	3.8it/s	41.2s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	79/79	4.6it/s	17.2s			
	all	2510	7818	0.75	0.651	0.721

0.504

	Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
	49/50	3.86G	0.7374	0.6238	1.026	18	
640: 100%		157/157	3.2it/s	49.1s			
	Class	Images	Instances	Box(P	R	mAP50	
mAP50-95): 100%		79/79	4.7it/s	16.8s			
	all	2510	7818	0.742	0.661	0.722	

0.507

	Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
	50/50	3.88G	0.7369	0.6158	1.029	11	
640: 100%		157/157	3.8it/s	41.4s			
	Class	Images	Instances	Box(P	R	mAP50	
mAP50-95): 100%		79/79	4.8it/s	16.3s			
	all	2510	7818	0.768	0.644	0.724	

0.508

50 epochs completed in 0.863 hours.

Optimizer stripped from /content/runs/detect/train/weights/last.pt, 5.5MB

Optimizer stripped from /content/runs/detect/train/weights/best.pt, 5.5MB

Validating /content/runs/detect/train/weights/best.pt...

Ultralytics 8.3.224 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4, 15095MiB)

YOLO11n summary (fused): 100 layers, 2,586,052 parameters, 0 gradients, 6.3 GFLOPs

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		79/79	4.1it/s	19.1s		
	all	2510	7818	0.747	0.653	0.724
0.508	aeroplane	127	175	0.819	0.806	0.859
0.61	bicycle	133	216	0.754	0.667	0.76
0.549	bird	151	305	0.755	0.524	0.616
0.389	boat	101	190	0.674	0.505	0.593
0.357	bottle	109	296	0.728	0.446	0.564
0.358	bus	97	141	0.796	0.652	0.743
0.618	car	359	818	0.798	0.754	0.818
0.594	cat	178	198	0.787	0.729	0.814
0.646	chair	290	706	0.657	0.487	0.548

0.347	cow	75	171	0.749	0.682	0.762
0.553	diningtable	133	162	0.725	0.519	0.644
0.442	dog	220	267	0.718	0.734	0.79
0.599	horse	150	199	0.804	0.802	0.825
0.62	motorbike	126	197	0.714	0.773	0.798
0.563	person	1025	2742	0.842	0.75	0.83
0.533	pottedplant	120	320	0.661	0.453	0.504
0.25	sheep	48	162	0.627	0.71	0.721
0.461	sofa	184	207	0.679	0.589	0.662
0.482	train	135	170	0.878	0.782	0.858
0.643	tvmonitor	135	176	0.783	0.693	0.769

0.554

Speed: 0.2ms preprocess, 1.8ms inference, 0.0ms loss, 1.5ms postprocess per image

Results saved to /content/runs/detect/train

[43]: ultralytics.utils.metrics.DetMetrics object with attributes:

```

ap_class_index: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13,
 14, 15, 16, 17, 18, 19])
box: ultralytics.utils.metrics.Metric object
confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at
0x7d56765a55b0>
curves: ['Precision-Recall(B)', 'F1-Confidence(B)', 'Precision-Confidence(B)',
'Recall-Confidence(B)']
curves_results: [[array([
0, 0.001001, 0.002002, 0.003003,
0.004004, 0.005005, 0.006006, 0.007007, 0.008008, 0.009009,
0.01001, 0.011011, 0.012012, 0.013013, 0.014014, 0.015015,
0.016016, 0.017017, 0.018018, 0.019019, 0.02002, 0.021021,
0.022022, 0.023023,
0.024024, 0.025025, 0.026026, 0.027027, 0.028028,
0.029029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034,
0.035035, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004,
0.041041, 0.042042, 0.043043, 0.044044, 0.045045, 0.046046,
0.047047,
0.048048, 0.049049, 0.05005, 0.051051, 0.052052,
```

0.053053,	0.054054,	0.055055,	0.056056,	0.057057,	0.058058,
0.059059,	0.06006,	0.061061,	0.062062,	0.063063,	0.064064,
0.065065,	0.066066,	0.067067,	0.068068,	0.069069,	0.07007,
0.071071,					
	0.072072,	0.073073,	0.074074,	0.075075,	0.076076,
0.077077,	0.078078,	0.079079,	0.08008,	0.081081,	0.082082,
0.083083,	0.084084,	0.085085,	0.086086,	0.087087,	0.088088,
0.089089,	0.09009,	0.091091,	0.092092,	0.093093,	0.094094,
0.095095,					
	0.096096,	0.097097,	0.098098,	0.099099,	0.1001,
0.1011,	0.1021,	0.1031,	0.1041,	0.10511,	0.10611,
0.10711,	0.10811,	0.10911,	0.11011,	0.11111,	0.11211,
0.11311,	0.11411,	0.11512,	0.11612,	0.11712,	0.11812,
0.11912,					
	0.12012,	0.12112,	0.12212,	0.12312,	0.12412,
0.12513,	0.12613,	0.12713,	0.12813,	0.12913,	0.13013,
0.13113,	0.13213,	0.13313,	0.13413,	0.13514,	0.13614,
0.13714,	0.13814,	0.13914,	0.14014,	0.14114,	0.14214,
0.14314,					
	0.14414,	0.14515,	0.14615,	0.14715,	0.14815,
0.14915,	0.15015,	0.15115,	0.15215,	0.15315,	0.15415,
0.15516,	0.15616,	0.15716,	0.15816,	0.15916,	0.16016,
0.16116,	0.16216,	0.16316,	0.16416,	0.16517,	0.16617,
0.16717,					
	0.16817,	0.16917,	0.17017,	0.17117,	0.17217,
0.17317,	0.17417,	0.17518,	0.17618,	0.17718,	0.17818,
0.17918,	0.18018,	0.18118,	0.18218,	0.18318,	0.18418,
0.18519,	0.18619,	0.18719,	0.18819,	0.18919,	0.19019,
0.19119,					
	0.19219,	0.19319,	0.19419,	0.1952,	0.1962,
0.1972,	0.1982,	0.1992,	0.2002,	0.2012,	0.2022,
0.2032,	0.2042,	0.20521,	0.20621,	0.20721,	0.20821,
0.20921,	0.21021,	0.21121,	0.21221,	0.21321,	0.21421,
0.21522,					
	0.21622,	0.21722,	0.21822,	0.21922,	0.22022,
0.22122,	0.22222,	0.22322,	0.22422,	0.22523,	0.22623,
0.22723,	0.22823,	0.22923,	0.23023,	0.23123,	0.23223,
0.23323,	0.23423,	0.23524,	0.23624,	0.23724,	0.23824,
0.23924,					
	0.24024,	0.24124,	0.24224,	0.24324,	0.24424,
0.24525,	0.24625,	0.24725,	0.24825,	0.24925,	0.25025,
0.25125,	0.25225,	0.25325,	0.25425,	0.25526,	0.25626,
0.25726,	0.25826,	0.25926,	0.26026,	0.26126,	0.26226,
0.26326,					
	0.26426,	0.26527,	0.26627,	0.26727,	0.26827,
0.26927,	0.27027,	0.27127,	0.27227,	0.27327,	0.27427,
0.27528,	0.27628,	0.27728,	0.27828,	0.27928,	0.28028,

0.28128,	0.28228,	0.28328,	0.28428,	0.28529,	0.28629,
0.28729,					
	0.28829,	0.28929,	0.29029,	0.29129,	0.29229,
0.29329,	0.29429,	0.2953,	0.2963,	0.2973,	0.2983,
0.2993,	0.3003,	0.3013,	0.3023,	0.3033,	0.3043,
0.30531,	0.30631,	0.30731,	0.30831,	0.30931,	0.31031,
0.31131,					
	0.31231,	0.31331,	0.31431,	0.31532,	0.31632,
0.31732,	0.31832,	0.31932,	0.32032,	0.32132,	0.32232,
0.32332,	0.32432,	0.32533,	0.32633,	0.32733,	0.32833,
0.32933,	0.33033,	0.33133,	0.33233,	0.33333,	0.33433,
0.33534,					
	0.33634,	0.33734,	0.33834,	0.33934,	0.34034,
0.34134,	0.34234,	0.34334,	0.34434,	0.34535,	0.34635,
0.34735,	0.34835,	0.34935,	0.35035,	0.35135,	0.35235,
0.35335,	0.35435,	0.35536,	0.35636,	0.35736,	0.35836,
0.35936,					
	0.36036,	0.36136,	0.36236,	0.36336,	0.36436,
0.36537,	0.36637,	0.36737,	0.36837,	0.36937,	0.37037,
0.37137,	0.37237,	0.37337,	0.37437,	0.37538,	0.37638,
0.37738,	0.37838,	0.37938,	0.38038,	0.38138,	0.38238,
0.38338,					
	0.38438,	0.38539,	0.38639,	0.38739,	0.38839,
0.38939,	0.39039,	0.39139,	0.39239,	0.39339,	0.39439,
0.3954,	0.3964,	0.3974,	0.3984,	0.3994,	0.4004,
0.4014,	0.4024,	0.4034,	0.4044,	0.40541,	0.40641,
0.40741,					
	0.40841,	0.40941,	0.41041,	0.41141,	0.41241,
0.41341,	0.41441,	0.41542,	0.41642,	0.41742,	0.41842,
0.41942,	0.42042,	0.42142,	0.42242,	0.42342,	0.42442,
0.42543,	0.42643,	0.42743,	0.42843,	0.42943,	0.43043,
0.43143,					
	0.43243,	0.43343,	0.43443,	0.43544,	0.43644,
0.43744,	0.43844,	0.43944,	0.44044,	0.44144,	0.44244,
0.44344,	0.44444,	0.44545,	0.44645,	0.44745,	0.44845,
0.44945,	0.45045,	0.45145,	0.45245,	0.45345,	0.45445,
0.45546,					
	0.45646,	0.45746,	0.45846,	0.45946,	0.46046,
0.46146,	0.46246,	0.46346,	0.46446,	0.46547,	0.46647,
0.46747,	0.46847,	0.46947,	0.47047,	0.47147,	0.47247,
0.47347,	0.47447,	0.47548,	0.47648,	0.47748,	0.47848,
0.47948,					
	0.48048,	0.48148,	0.48248,	0.48348,	0.48448,
0.48549,	0.48649,	0.48749,	0.48849,	0.48949,	0.49049,
0.49149,	0.49249,	0.49349,	0.49449,	0.4955,	0.4965,
0.4975,	0.4985,	0.4995,	0.5005,	0.5015,	0.5025,
0.5035,					

	0.5045,	0.50551,	0.50651,	0.50751,	0.50851,
0.50951,	0.51051,	0.51151,	0.51251,	0.51351,	0.51451,
0.51552,	0.51652,	0.51752,	0.51852,	0.51952,	0.52052,
0.52152,	0.52252,	0.52352,	0.52452,	0.52553,	0.52653,
0.52753,					
	0.52853,	0.52953,	0.53053,	0.53153,	0.53253,
0.53353,	0.53453,	0.53554,	0.53654,	0.53754,	0.53854,
0.53954,	0.54054,	0.54154,	0.54254,	0.54354,	0.54454,
0.54555,	0.54655,	0.54755,	0.54855,	0.54955,	0.55055,
0.55155,					
	0.55255,	0.55355,	0.55455,	0.55556,	0.55656,
0.55756,	0.55856,	0.55956,	0.56056,	0.56156,	0.56256,
0.56356,	0.56456,	0.56557,	0.56657,	0.56757,	0.56857,
0.56957,	0.57057,	0.57157,	0.57257,	0.57357,	0.57457,
0.57558,					
	0.57658,	0.57758,	0.57858,	0.57958,	0.58058,
0.58158,	0.58258,	0.58358,	0.58458,	0.58559,	0.58659,
0.58759,	0.58859,	0.58959,	0.59059,	0.59159,	0.59259,
0.59359,	0.59459,	0.5956,	0.5966,	0.5976,	0.5986,
0.5996,					
	0.6006,	0.6016,	0.6026,	0.6036,	0.6046,
0.60561,	0.60661,	0.60761,	0.60861,	0.60961,	0.61061,
0.61161,	0.61261,	0.61361,	0.61461,	0.61562,	0.61662,
0.61762,	0.61862,	0.61962,	0.62062,	0.62162,	0.62262,
0.62362,					
	0.62462,	0.62563,	0.62663,	0.62763,	0.62863,
0.62963,	0.63063,	0.63163,	0.63263,	0.63363,	0.63463,
0.63564,	0.63664,	0.63764,	0.63864,	0.63964,	0.64064,
0.64164,	0.64264,	0.64364,	0.64464,	0.64565,	0.64665,
0.64765,					
	0.64865,	0.64965,	0.65065,	0.65165,	0.65265,
0.65365,	0.65465,	0.65566,	0.65666,	0.65766,	0.65866,
0.65966,	0.66066,	0.66166,	0.66266,	0.66366,	0.66466,
0.66567,	0.66667,	0.66767,	0.66867,	0.66967,	0.67067,
0.67167,					
	0.67267,	0.67367,	0.67467,	0.67568,	0.67668,
0.67768,	0.67868,	0.67968,	0.68068,	0.68168,	0.68268,
0.68368,	0.68468,	0.68569,	0.68669,	0.68769,	0.68869,
0.68969,	0.69069,	0.69169,	0.69269,	0.69369,	0.69469,
0.6957,					
	0.6967,	0.6977,	0.6987,	0.6997,	0.7007,
0.7017,	0.7027,	0.7037,	0.7047,	0.70571,	0.70671,
0.70771,	0.70871,	0.70971,	0.71071,	0.71171,	0.71271,
0.71371,	0.71471,	0.71572,	0.71672,	0.71772,	0.71872,
0.71972,					
	0.72072,	0.72172,	0.72272,	0.72372,	0.72472,
0.72573,	0.72673,	0.72773,	0.72873,	0.72973,	0.73073,

0.73173,	0.73273,	0.73373,	0.73473,	0.73574,	0.73674,
0.73774,	0.73874,	0.73974,	0.74074,	0.74174,	0.74274,
0.74374,	0.74474,	0.74575,	0.74675,	0.74775,	0.74875,
0.74975,	0.75075,	0.75175,	0.75275,	0.75375,	0.75475,
0.75576,	0.75676,	0.75776,	0.75876,	0.75976,	0.76076,
0.76176,	0.76276,	0.76376,	0.76476,	0.76577,	0.76677,
0.76777,	0.76877,	0.76977,	0.77077,	0.77177,	0.77277,
0.77377,	0.77477,	0.77578,	0.77678,	0.77778,	0.77878,
0.77978,	0.78078,	0.78178,	0.78278,	0.78378,	0.78478,
0.78579,	0.78679,	0.78779,	0.78879,	0.78979,	0.79079,
0.79179,	0.79279,	0.79379,	0.79479,	0.7958,	0.7968,
0.7978,	0.7988,	0.7998,	0.8008,	0.8018,	0.8028,
0.8038,	0.8048,	0.80581,	0.80681,	0.80781,	0.80881,
0.80981,	0.81081,	0.81181,	0.81281,	0.81381,	0.81481,
0.81582,	0.81682,	0.81782,	0.81882,	0.81982,	0.82082,
0.82182,	0.82282,	0.82382,	0.82482,	0.82583,	0.82683,
0.82783,	0.82883,	0.82983,	0.83083,	0.83183,	0.83283,
0.83383,	0.83483,	0.83584,	0.83684,	0.83784,	0.83884,
0.83984,	0.84084,	0.84184,	0.84284,	0.84384,	0.84484,
0.84585,	0.84685,	0.84785,	0.84885,	0.84985,	0.85085,
0.85185,	0.85285,	0.85385,	0.85485,	0.85586,	0.85686,
0.85786,	0.85886,	0.85986,	0.86086,	0.86186,	0.86286,
0.86386,	0.86486,	0.86587,	0.86687,	0.86787,	0.86887,
0.86987,	0.87087,	0.87187,	0.87287,	0.87387,	0.87487,
0.87588,	0.87688,	0.87788,	0.87888,	0.87988,	0.88088,
0.88188,	0.88288,	0.88388,	0.88488,	0.88589,	0.88689,
0.88789,	0.88889,	0.88989,	0.89089,	0.89189,	0.89289,
0.89389,	0.89489,	0.8959,	0.8969,	0.8979,	0.8989,
0.8999,	0.9009,	0.9019,	0.9029,	0.9039,	0.9049,
0.90591,	0.90691,	0.90791,	0.90891,	0.90991,	0.91091,
0.91191,	0.91291,	0.91391,	0.91491,	0.91592,	0.91692,
0.91792,	0.91892,	0.91992,	0.92092,	0.92192,	0.92292,
0.92392,	0.92492,	0.92593,	0.92693,	0.92793,	0.92893,
0.92993,	0.93093,	0.93193,	0.93293,	0.93393,	0.93493,
0.93594,	0.93694,	0.93794,	0.93894,	0.93994,	0.94094,
0.94194,	0.94294,	0.94394,	0.94494,	0.94595,	0.94695,
0.94795,	0.94895,	0.94995,	0.95095,	0.95195,	0.95295,
0.95395,	0.95495,	0.95596,	0.95696,	0.95796,	0.95896,

```

0.95996,
0.96096, 0.96196, 0.96296, 0.96396, 0.96496,
0.96597, 0.96697, 0.96797, 0.96897, 0.96997, 0.97097,
0.97197, 0.97297, 0.97397, 0.97497, 0.97598, 0.97698,
0.97798, 0.97898, 0.97998, 0.98098, 0.98198, 0.98298,
0.98398,
0.98498, 0.98599, 0.98699, 0.98799, 0.98899,
0.98999, 0.99099, 0.99199, 0.99299, 0.99399, 0.99499,
0.996, 0.997, 0.998, 0.999, 1]), array([[
1, 1, 1, ..., 0.0040017, 0.0020009, 0],
[ 1, 1, 1, ..., 0.0011476, 0.0005738,
0],
[ 0.95652, 0.95652, 0.95652, ..., 0.00071805, 0.00035903,
0],
...,
[ 1, 1, 1, ..., 0.0015651, 0.00078255,
0],
[ 1, 1, 1, ..., 0.0019247, 0.00096234,
0],
[ 1, 1, 1, ..., 0.0016077, 0.00080385,
0]]), 'Recall', 'Precision'], [array([
0, 0.001001, 0.002002,
0.003003, 0.004004, 0.005005, 0.006006, 0.007007, 0.008008,
0.009009, 0.01001, 0.011011, 0.012012, 0.013013, 0.014014,
0.015015, 0.016016, 0.017017, 0.018018, 0.019019, 0.02002,
0.021021, 0.022022, 0.023023,
0.024024, 0.025025, 0.026026, 0.027027, 0.028028,
0.029029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034,
0.035035, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004,
0.041041, 0.042042, 0.043043, 0.044044, 0.045045, 0.046046,
0.047047,
0.048048, 0.049049, 0.05005, 0.051051, 0.052052,
0.053053, 0.054054, 0.055055, 0.056056, 0.057057, 0.058058,
0.059059, 0.06006, 0.061061, 0.062062, 0.063063, 0.064064,
0.065065, 0.066066, 0.067067, 0.068068, 0.069069, 0.07007,
0.071071,
0.072072, 0.073073, 0.074074, 0.075075, 0.076076,
0.077077, 0.078078, 0.079079, 0.08008, 0.081081, 0.082082,
0.083083, 0.084084, 0.085085, 0.086086, 0.087087, 0.088088,
0.089089, 0.09009, 0.091091, 0.092092, 0.093093, 0.094094,
0.095095,
0.096096, 0.097097, 0.098098, 0.099099, 0.1001,
0.1011, 0.1021, 0.1031, 0.1041, 0.10511, 0.10611,
0.10711, 0.10811, 0.10911, 0.11011, 0.11111, 0.11211,
0.11311, 0.11411, 0.11512, 0.11612, 0.11712, 0.11812,
0.11912,
0.12012, 0.12112, 0.12212, 0.12312, 0.12412,
0.12513, 0.12613, 0.12713, 0.12813, 0.12913, 0.13013,

```

0.13113,	0.13213,	0.13313,	0.13413,	0.13514,	0.13614,
0.13714,	0.13814,	0.13914,	0.14014,	0.14114,	0.14214,
0.14314,	0.14414,	0.14515,	0.14615,	0.14715,	0.14815,
0.14915,	0.15015,	0.15115,	0.15215,	0.15315,	0.15415,
0.15516,	0.15616,	0.15716,	0.15816,	0.15916,	0.16016,
0.16116,	0.16216,	0.16316,	0.16416,	0.16517,	0.16617,
0.16717,	0.16817,	0.16917,	0.17017,	0.17117,	0.17217,
0.17317,	0.17417,	0.17518,	0.17618,	0.17718,	0.17818,
0.17918,	0.18018,	0.18118,	0.18218,	0.18318,	0.18418,
0.18519,	0.18619,	0.18719,	0.18819,	0.18919,	0.19019,
0.19119,	0.19219,	0.19319,	0.19419,	0.1952,	0.1962,
0.1972,	0.1982,	0.1992,	0.2002,	0.2012,	0.2022,
0.2032,	0.2042,	0.20521,	0.20621,	0.20721,	0.20821,
0.20921,	0.21021,	0.21121,	0.21221,	0.21321,	0.21421,
0.21522,	0.21622,	0.21722,	0.21822,	0.21922,	0.22022,
0.22122,	0.22222,	0.22322,	0.22422,	0.22523,	0.22623,
0.22723,	0.22823,	0.22923,	0.23023,	0.23123,	0.23223,
0.23323,	0.23423,	0.23524,	0.23624,	0.23724,	0.23824,
0.23924,	0.24024,	0.24124,	0.24224,	0.24324,	0.24424,
0.24525,	0.24625,	0.24725,	0.24825,	0.24925,	0.25025,
0.25125,	0.25225,	0.25325,	0.25425,	0.25526,	0.25626,
0.25726,	0.25826,	0.25926,	0.26026,	0.26126,	0.26226,
0.26326,	0.26426,	0.26527,	0.26627,	0.26727,	0.26827,
0.26927,	0.27027,	0.27127,	0.27227,	0.27327,	0.27427,
0.27528,	0.27628,	0.27728,	0.27828,	0.27928,	0.28028,
0.28128,	0.28228,	0.28328,	0.28428,	0.28529,	0.28629,
0.28729,	0.28829,	0.28929,	0.29029,	0.29129,	0.29229,
0.29329,	0.29429,	0.2953,	0.2963,	0.2973,	0.2983,
0.2993,	0.3003,	0.3013,	0.3023,	0.3033,	0.3043,
0.30531,	0.30631,	0.30731,	0.30831,	0.30931,	0.31031,
0.31131,	0.31231,	0.31331,	0.31431,	0.31532,	0.31632,
0.31732,	0.31832,	0.31932,	0.32032,	0.32132,	0.32232,
0.32332,	0.32432,	0.32533,	0.32633,	0.32733,	0.32833,
0.32933,	0.33033,	0.33133,	0.33233,	0.33333,	0.33433,
0.33534,	0.33634,	0.33734,	0.33834,	0.33934,	0.34034,
0.34134,	0.34234,	0.34334,	0.34434,	0.34535,	0.34635,
0.34735,	0.34835,	0.34935,	0.35035,	0.35135,	0.35235,
0.35335,	0.35435,	0.35536,	0.35636,	0.35736,	0.35836,

0.35936,					
	0.36036,	0.36136,	0.36236,	0.36336,	0.36436,
0.36537,	0.36637,	0.36737,	0.36837,	0.36937,	0.37037,
0.37137,	0.37237,	0.37337,	0.37437,	0.37538,	0.37638,
0.37738,	0.37838,	0.37938,	0.38038,	0.38138,	0.38238,
0.38338,					
	0.38438,	0.38539,	0.38639,	0.38739,	0.38839,
0.38939,	0.39039,	0.39139,	0.39239,	0.39339,	0.39439,
0.3954,	0.3964,	0.3974,	0.3984,	0.3994,	0.4004,
0.4014,	0.4024,	0.4034,	0.4044,	0.40541,	0.40641,
0.40741,					
	0.40841,	0.40941,	0.41041,	0.41141,	0.41241,
0.41341,	0.41441,	0.41542,	0.41642,	0.41742,	0.41842,
0.41942,	0.42042,	0.42142,	0.42242,	0.42342,	0.42442,
0.42543,	0.42643,	0.42743,	0.42843,	0.42943,	0.43043,
0.43143,					
	0.43243,	0.43343,	0.43443,	0.43544,	0.43644,
0.43744,	0.43844,	0.43944,	0.44044,	0.44144,	0.44244,
0.44344,	0.44444,	0.44545,	0.44645,	0.44745,	0.44845,
0.44945,	0.45045,	0.45145,	0.45245,	0.45345,	0.45445,
0.45546,					
	0.45646,	0.45746,	0.45846,	0.45946,	0.46046,
0.46146,	0.46246,	0.46346,	0.46446,	0.46547,	0.46647,
0.46747,	0.46847,	0.46947,	0.47047,	0.47147,	0.47247,
0.47347,	0.47447,	0.47548,	0.47648,	0.47748,	0.47848,
0.47948,					
	0.48048,	0.48148,	0.48248,	0.48348,	0.48448,
0.48549,	0.48649,	0.48749,	0.48849,	0.48949,	0.49049,
0.49149,	0.49249,	0.49349,	0.49449,	0.4955,	0.4965,
0.4975,	0.4985,	0.4995,	0.5005,	0.5015,	0.5025,
0.5035,					
	0.5045,	0.50551,	0.50651,	0.50751,	0.50851,
0.50951,	0.51051,	0.51151,	0.51251,	0.51351,	0.51451,
0.51552,	0.51652,	0.51752,	0.51852,	0.51952,	0.52052,
0.52152,	0.52252,	0.52352,	0.52452,	0.52553,	0.52653,
0.52753,					
	0.52853,	0.52953,	0.53053,	0.53153,	0.53253,
0.53353,	0.53453,	0.53554,	0.53654,	0.53754,	0.53854,
0.53954,	0.54054,	0.54154,	0.54254,	0.54354,	0.54454,
0.54555,	0.54655,	0.54755,	0.54855,	0.54955,	0.55055,
0.55155,					
	0.55255,	0.55355,	0.55455,	0.55556,	0.55656,
0.55756,	0.55856,	0.55956,	0.56056,	0.56156,	0.56256,
0.56356,	0.56456,	0.56557,	0.56657,	0.56757,	0.56857,
0.56957,	0.57057,	0.57157,	0.57257,	0.57357,	0.57457,
0.57558,					
	0.57658,	0.57758,	0.57858,	0.57958,	0.58058,

0.58158,	0.58258,	0.58358,	0.58458,	0.58559,	0.58659,
0.58759,	0.58859,	0.58959,	0.59059,	0.59159,	0.59259,
0.59359,	0.59459,	0.5956,	0.5966,	0.5976,	0.5986,
0.5996,					
	0.6006,	0.6016,	0.6026,	0.6036,	0.6046,
0.60561,	0.60661,	0.60761,	0.60861,	0.60961,	0.61061,
0.61161,	0.61261,	0.61361,	0.61461,	0.61562,	0.61662,
0.61762,	0.61862,	0.61962,	0.62062,	0.62162,	0.62262,
0.62362,					
	0.62462,	0.62563,	0.62663,	0.62763,	0.62863,
0.62963,	0.63063,	0.63163,	0.63263,	0.63363,	0.63463,
0.63564,	0.63664,	0.63764,	0.63864,	0.63964,	0.64064,
0.64164,	0.64264,	0.64364,	0.64464,	0.64565,	0.64665,
0.64765,					
	0.64865,	0.64965,	0.65065,	0.65165,	0.65265,
0.65365,	0.65465,	0.65566,	0.65666,	0.65766,	0.65866,
0.65966,	0.66066,	0.66166,	0.66266,	0.66366,	0.66466,
0.66567,	0.66667,	0.66767,	0.66867,	0.66967,	0.67067,
0.67167,					
	0.67267,	0.67367,	0.67467,	0.67568,	0.67668,
0.67768,	0.67868,	0.67968,	0.68068,	0.68168,	0.68268,
0.68368,	0.68468,	0.68569,	0.68669,	0.68769,	0.68869,
0.68969,	0.69069,	0.69169,	0.69269,	0.69369,	0.69469,
0.6957,					
	0.6967,	0.6977,	0.6987,	0.6997,	0.7007,
0.7017,	0.7027,	0.7037,	0.7047,	0.70571,	0.70671,
0.70771,	0.70871,	0.70971,	0.71071,	0.71171,	0.71271,
0.71371,	0.71471,	0.71572,	0.71672,	0.71772,	0.71872,
0.71972,					
	0.72072,	0.72172,	0.72272,	0.72372,	0.72472,
0.72573,	0.72673,	0.72773,	0.72873,	0.72973,	0.73073,
0.73173,	0.73273,	0.73373,	0.73473,	0.73574,	0.73674,
0.73774,	0.73874,	0.73974,	0.74074,	0.74174,	0.74274,
0.74374,					
	0.74474,	0.74575,	0.74675,	0.74775,	0.74875,
0.74975,	0.75075,	0.75175,	0.75275,	0.75375,	0.75475,
0.75576,	0.75676,	0.75776,	0.75876,	0.75976,	0.76076,
0.76176,	0.76276,	0.76376,	0.76476,	0.76577,	0.76677,
0.76777,					
	0.76877,	0.76977,	0.77077,	0.77177,	0.77277,
0.77377,	0.77477,	0.77578,	0.77678,	0.77778,	0.77878,
0.77978,	0.78078,	0.78178,	0.78278,	0.78378,	0.78478,
0.78579,	0.78679,	0.78779,	0.78879,	0.78979,	0.79079,
0.79179,					
	0.79279,	0.79379,	0.79479,	0.7958,	0.7968,
0.7978,	0.7988,	0.7998,	0.8008,	0.8018,	0.8028,
0.8038,	0.8048,	0.80581,	0.80681,	0.80781,	0.80881,

0.80981,	0.81081,	0.81181,	0.81281,	0.81381,	0.81481,
0.81582,					
	0.81682,	0.81782,	0.81882,	0.81982,	0.82082,
0.82182,	0.82282,	0.82382,	0.82482,	0.82583,	0.82683,
0.82783,	0.82883,	0.82983,	0.83083,	0.83183,	0.83283,
0.83383,	0.83483,	0.83584,	0.83684,	0.83784,	0.83884,
0.83984,					
	0.84084,	0.84184,	0.84284,	0.84384,	0.84484,
0.84585,	0.84685,	0.84785,	0.84885,	0.84985,	0.85085,
0.85185,	0.85285,	0.85385,	0.85485,	0.85586,	0.85686,
0.85786,	0.85886,	0.85986,	0.86086,	0.86186,	0.86286,
0.86386,					
	0.86486,	0.86587,	0.86687,	0.86787,	0.86887,
0.86987,	0.87087,	0.87187,	0.87287,	0.87387,	0.87487,
0.87588,	0.87688,	0.87788,	0.87888,	0.87988,	0.88088,
0.88188,	0.88288,	0.88388,	0.88488,	0.88589,	0.88689,
0.88789,					
	0.88889,	0.88989,	0.89089,	0.89189,	0.89289,
0.89389,	0.89489,	0.8959,	0.8969,	0.8979,	0.8989,
0.8999,	0.9009,	0.9019,	0.9029,	0.9039,	0.9049,
0.90591,	0.90691,	0.90791,	0.90891,	0.90991,	0.91091,
0.91191,					
	0.91291,	0.91391,	0.91491,	0.91592,	0.91692,
0.91792,	0.91892,	0.91992,	0.92092,	0.92192,	0.92292,
0.92392,	0.92492,	0.92593,	0.92693,	0.92793,	0.92893,
0.92993,	0.93093,	0.93193,	0.93293,	0.93393,	0.93493,
0.93594,					
	0.93694,	0.93794,	0.93894,	0.93994,	0.94094,
0.94194,	0.94294,	0.94394,	0.94494,	0.94595,	0.94695,
0.94795,	0.94895,	0.94995,	0.95095,	0.95195,	0.95295,
0.95395,	0.95495,	0.95596,	0.95696,	0.95796,	0.95896,
0.95996,					
	0.96096,	0.96196,	0.96296,	0.96396,	0.96496,
0.96597,	0.96697,	0.96797,	0.96897,	0.96997,	0.97097,
0.97197,	0.97297,	0.97397,	0.97497,	0.97598,	0.97698,
0.97798,	0.97898,	0.97998,	0.98098,	0.98198,	0.98298,
0.98398,					
	0.98498,	0.98599,	0.98699,	0.98799,	0.98899,
0.98999,	0.99099,	0.99199,	0.99299,	0.99399,	0.99499,
0.996,	0.997,	0.998,	0.999,	1]), array([[	
0.12798,	0.12798,	0.22162, ...,	0,	0,	0],
[	0.095561,	0.095561,	0.15101, ...,	0,	0,
0],					
[	0.10376,	0.10376,	0.17315, ...,	0,	0,
0],					
...,					
[	0.079594,	0.079594,	0.13956, ...,	0,	0,

```

0],
    [    0.11665,    0.11665,    0.20495, ...,    0,    0,
0],
    [    0.11149,    0.11149,    0.18921, ...,    0,    0,
0]]), 'Confidence', 'F1'], [array([
0.003003,    0.004004,    0.005005,    0.006006,    0.007007,    0.008008,
0.009009,    0.01001,    0.011011,    0.012012,    0.013013,    0.014014,
0.015015,    0.016016,    0.017017,    0.018018,    0.019019,    0.02002,
0.021021,    0.022022,    0.023023,
0.024024,    0.025025,    0.026026,    0.027027,    0.028028,
0.029029,    0.03003,    0.031031,    0.032032,    0.033033,    0.034034,
0.035035,    0.036036,    0.037037,    0.038038,    0.039039,    0.04004,
0.041041,    0.042042,    0.043043,    0.044044,    0.045045,    0.046046,
0.047047,
0.048048,    0.049049,    0.05005,    0.051051,    0.052052,
0.053053,    0.054054,    0.055055,    0.056056,    0.057057,    0.058058,
0.059059,    0.06006,    0.061061,    0.062062,    0.063063,    0.064064,
0.065065,    0.066066,    0.067067,    0.068068,    0.069069,    0.07007,
0.071071,
0.072072,    0.073073,    0.074074,    0.075075,    0.076076,
0.077077,    0.078078,    0.079079,    0.08008,    0.081081,    0.082082,
0.083083,    0.084084,    0.085085,    0.086086,    0.087087,    0.088088,
0.089089,    0.09009,    0.091091,    0.092092,    0.093093,    0.094094,
0.095095,
0.096096,    0.097097,    0.098098,    0.099099,    0.1001,
0.1011,    0.1021,    0.1031,    0.1041,    0.10511,    0.10611,
0.10711,    0.10811,    0.10911,    0.11011,    0.11111,    0.11211,
0.11311,    0.11411,    0.11512,    0.11612,    0.11712,    0.11812,
0.11912,
0.12012,    0.12112,    0.12212,    0.12312,    0.12412,
0.12513,    0.12613,    0.12713,    0.12813,    0.12913,    0.13013,
0.13113,    0.13213,    0.13313,    0.13413,    0.13514,    0.13614,
0.13714,    0.13814,    0.13914,    0.14014,    0.14114,    0.14214,
0.14314,
0.14414,    0.14515,    0.14615,    0.14715,    0.14815,
0.14915,    0.15015,    0.15115,    0.15215,    0.15315,    0.15415,
0.15516,    0.15616,    0.15716,    0.15816,    0.15916,    0.16016,
0.16116,    0.16216,    0.16316,    0.16416,    0.16517,    0.16617,
0.16717,
0.16817,    0.16917,    0.17017,    0.17117,    0.17217,
0.17317,    0.17417,    0.17518,    0.17618,    0.17718,    0.17818,
0.17918,    0.18018,    0.18118,    0.18218,    0.18318,    0.18418,
0.18519,    0.18619,    0.18719,    0.18819,    0.18919,    0.19019,
0.19119,
0.19219,    0.19319,    0.19419,    0.1952,    0.1962,
0.1972,    0.1982,    0.1992,    0.2002,    0.2012,    0.2022,
0.2032,    0.2042,    0.20521,    0.20621,    0.20721,    0.20821,

```


0.20921, 0.21522,	0.21021,	0.21121,	0.21221,	0.21321,	0.21421,
	0.21622,	0.21722,	0.21822,	0.21922,	0.22022,
0.22122,	0.22222,	0.22322,	0.22422,	0.22523,	0.22623,
0.22723,	0.22823,	0.22923,	0.23023,	0.23123,	0.23223,
0.23323,	0.23423,	0.23524,	0.23624,	0.23724,	0.23824,
0.23924,					
	0.24024,	0.24124,	0.24224,	0.24324,	0.24424,
0.24525,	0.24625,	0.24725,	0.24825,	0.24925,	0.25025,
0.25125,	0.25225,	0.25325,	0.25425,	0.25526,	0.25626,
0.25726,	0.25826,	0.25926,	0.26026,	0.26126,	0.26226,
0.26326,					
	0.26426,	0.26527,	0.26627,	0.26727,	0.26827,
0.26927,	0.27027,	0.27127,	0.27227,	0.27327,	0.27427,
0.27528,	0.27628,	0.27728,	0.27828,	0.27928,	0.28028,
0.28128,	0.28228,	0.28328,	0.28428,	0.28529,	0.28629,
0.28729,					
	0.28829,	0.28929,	0.29029,	0.29129,	0.29229,
0.29329,	0.29429,	0.2953,	0.2963,	0.2973,	0.2983,
0.2993,	0.3003,	0.3013,	0.3023,	0.3033,	0.3043,
0.30531,	0.30631,	0.30731,	0.30831,	0.30931,	0.31031,
0.31131,					
	0.31231,	0.31331,	0.31431,	0.31532,	0.31632,
0.31732,	0.31832,	0.31932,	0.32032,	0.32132,	0.32232,
0.32332,	0.32432,	0.32533,	0.32633,	0.32733,	0.32833,
0.32933,	0.33033,	0.33133,	0.33233,	0.33333,	0.33433,
0.33534,					
	0.33634,	0.33734,	0.33834,	0.33934,	0.34034,
0.34134,	0.34234,	0.34334,	0.34434,	0.34535,	0.34635,
0.34735,	0.34835,	0.34935,	0.35035,	0.35135,	0.35235,
0.35335,	0.35435,	0.35536,	0.35636,	0.35736,	0.35836,
0.35936,					
	0.36036,	0.36136,	0.36236,	0.36336,	0.36436,
0.36537,	0.36637,	0.36737,	0.36837,	0.36937,	0.37037,
0.37137,	0.37237,	0.37337,	0.37437,	0.37538,	0.37638,
0.37738,	0.37838,	0.37938,	0.38038,	0.38138,	0.38238,
0.38338,					
	0.38438,	0.38539,	0.38639,	0.38739,	0.38839,
0.38939,	0.39039,	0.39139,	0.39239,	0.39339,	0.39439,
0.3954,	0.3964,	0.3974,	0.3984,	0.3994,	0.4004,
0.4014,	0.4024,	0.4034,	0.4044,	0.40541,	0.40641,
0.40741,					
	0.40841,	0.40941,	0.41041,	0.41141,	0.41241,
0.41341,	0.41441,	0.41542,	0.41642,	0.41742,	0.41842,
0.41942,	0.42042,	0.42142,	0.42242,	0.42342,	0.42442,
0.42543,	0.42643,	0.42743,	0.42843,	0.42943,	0.43043,
0.43143,					

	0.43243,	0.43343,	0.43443,	0.43544,	0.43644,
0.43744,	0.43844,	0.43944,	0.44044,	0.44144,	0.44244,
0.44344,	0.44444,	0.44545,	0.44645,	0.44745,	0.44845,
0.44945,	0.45045,	0.45145,	0.45245,	0.45345,	0.45445,
0.45546,					
	0.45646,	0.45746,	0.45846,	0.45946,	0.46046,
0.46146,	0.46246,	0.46346,	0.46446,	0.46547,	0.46647,
0.46747,	0.46847,	0.46947,	0.47047,	0.47147,	0.47247,
0.47347,	0.47447,	0.47548,	0.47648,	0.47748,	0.47848,
0.47948,					
	0.48048,	0.48148,	0.48248,	0.48348,	0.48448,
0.48549,	0.48649,	0.48749,	0.48849,	0.48949,	0.49049,
0.49149,	0.49249,	0.49349,	0.49449,	0.4955,	0.4965,
0.4975,	0.4985,	0.4995,	0.5005,	0.5015,	0.5025,
0.5035,					
	0.5045,	0.50551,	0.50651,	0.50751,	0.50851,
0.50951,	0.51051,	0.51151,	0.51251,	0.51351,	0.51451,
0.51552,	0.51652,	0.51752,	0.51852,	0.51952,	0.52052,
0.52152,	0.52252,	0.52352,	0.52452,	0.52553,	0.52653,
0.52753,					
	0.52853,	0.52953,	0.53053,	0.53153,	0.53253,
0.53353,	0.53453,	0.53554,	0.53654,	0.53754,	0.53854,
0.53954,	0.54054,	0.54154,	0.54254,	0.54354,	0.54454,
0.54555,	0.54655,	0.54755,	0.54855,	0.54955,	0.55055,
0.55155,					
	0.55255,	0.55355,	0.55455,	0.55556,	0.55656,
0.55756,	0.55856,	0.55956,	0.56056,	0.56156,	0.56256,
0.56356,	0.56456,	0.56557,	0.56657,	0.56757,	0.56857,
0.56957,	0.57057,	0.57157,	0.57257,	0.57357,	0.57457,
0.57558,					
	0.57658,	0.57758,	0.57858,	0.57958,	0.58058,
0.58158,	0.58258,	0.58358,	0.58458,	0.58559,	0.58659,
0.58759,	0.58859,	0.58959,	0.59059,	0.59159,	0.59259,
0.59359,	0.59459,	0.5956,	0.5966,	0.5976,	0.5986,
0.5996,					
	0.6006,	0.6016,	0.6026,	0.6036,	0.6046,
0.60561,	0.60661,	0.60761,	0.60861,	0.60961,	0.61061,
0.61161,	0.61261,	0.61361,	0.61461,	0.61562,	0.61662,
0.61762,	0.61862,	0.61962,	0.62062,	0.62162,	0.62262,
0.62362,					
	0.62462,	0.62563,	0.62663,	0.62763,	0.62863,
0.62963,	0.63063,	0.63163,	0.63263,	0.63363,	0.63463,
0.63564,	0.63664,	0.63764,	0.63864,	0.63964,	0.64064,
0.64164,	0.64264,	0.64364,	0.64464,	0.64565,	0.64665,
0.64765,					
	0.64865,	0.64965,	0.65065,	0.65165,	0.65265,
0.65365,	0.65465,	0.65566,	0.65666,	0.65766,	0.65866,

0.65966,	0.66066,	0.66166,	0.66266,	0.66366,	0.66466,
0.66567,	0.66667,	0.66767,	0.66867,	0.66967,	0.67067,
0.67167,	0.67267,	0.67367,	0.67467,	0.67568,	0.67668,
0.67768,	0.67868,	0.67968,	0.68068,	0.68168,	0.68268,
0.68368,	0.68468,	0.68569,	0.68669,	0.68769,	0.68869,
0.68969,	0.69069,	0.69169,	0.69269,	0.69369,	0.69469,
0.6957,	0.6967,	0.6977,	0.6987,	0.6997,	0.7007,
0.7017,	0.7027,	0.7037,	0.7047,	0.70571,	0.70671,
0.70771,	0.70871,	0.70971,	0.71071,	0.71171,	0.71271,
0.71371,	0.71471,	0.71572,	0.71672,	0.71772,	0.71872,
0.71972,	0.72072,	0.72172,	0.72272,	0.72372,	0.72472,
0.72573,	0.72673,	0.72773,	0.72873,	0.72973,	0.73073,
0.73173,	0.73273,	0.73373,	0.73473,	0.73574,	0.73674,
0.73774,	0.73874,	0.73974,	0.74074,	0.74174,	0.74274,
0.74374,	0.74474,	0.74575,	0.74675,	0.74775,	0.74875,
0.74975,	0.75075,	0.75175,	0.75275,	0.75375,	0.75475,
0.75576,	0.75676,	0.75776,	0.75876,	0.75976,	0.76076,
0.76176,	0.76276,	0.76376,	0.76476,	0.76577,	0.76677,
0.76777,	0.76877,	0.76977,	0.77077,	0.77177,	0.77277,
0.77377,	0.77477,	0.77578,	0.77678,	0.77778,	0.77878,
0.77978,	0.78078,	0.78178,	0.78278,	0.78378,	0.78478,
0.78579,	0.78679,	0.78779,	0.78879,	0.78979,	0.79079,
0.79179,	0.79279,	0.79379,	0.79479,	0.7958,	0.7968,
0.7978,	0.7988,	0.7998,	0.8008,	0.8018,	0.8028,
0.8038,	0.8048,	0.80581,	0.80681,	0.80781,	0.80881,
0.80981,	0.81081,	0.81181,	0.81281,	0.81381,	0.81481,
0.81582,	0.81682,	0.81782,	0.81882,	0.81982,	0.82082,
0.82182,	0.82282,	0.82382,	0.82482,	0.82583,	0.82683,
0.82783,	0.82883,	0.82983,	0.83083,	0.83183,	0.83283,
0.83383,	0.83483,	0.83584,	0.83684,	0.83784,	0.83884,
0.83984,	0.84084,	0.84184,	0.84284,	0.84384,	0.84484,
0.84585,	0.84685,	0.84785,	0.84885,	0.84985,	0.85085,
0.85185,	0.85285,	0.85385,	0.85485,	0.85586,	0.85686,
0.85786,	0.85886,	0.85986,	0.86086,	0.86186,	0.86286,
0.86386,	0.86486,	0.86587,	0.86687,	0.86787,	0.86887,
0.86987,	0.87087,	0.87187,	0.87287,	0.87387,	0.87487,
0.87588,	0.87688,	0.87788,	0.87888,	0.87988,	0.88088,
0.88188,	0.88288,	0.88388,	0.88488,	0.88589,	0.88689,

```

0.88789,
0.88889, 0.88989, 0.89089, 0.89189, 0.89289,
0.89389, 0.89489, 0.8959, 0.8969, 0.8979, 0.8989,
0.8999, 0.9009, 0.9019, 0.9029, 0.9039, 0.9049,
0.90591, 0.90691, 0.90791, 0.90891, 0.90991, 0.91091,
0.91191,
0.91291, 0.91391, 0.91491, 0.91592, 0.91692,
0.91792, 0.91892, 0.91992, 0.92092, 0.92192, 0.92292,
0.92392, 0.92492, 0.92593, 0.92693, 0.92793, 0.92893,
0.92993, 0.93093, 0.93193, 0.93293, 0.93393, 0.93493,
0.93594,
0.93694, 0.93794, 0.93894, 0.93994, 0.94094,
0.94194, 0.94294, 0.94394, 0.94494, 0.94595, 0.94695,
0.94795, 0.94895, 0.94995, 0.95095, 0.95195, 0.95295,
0.95395, 0.95495, 0.95596, 0.95696, 0.95796, 0.95896,
0.95996,
0.96096, 0.96196, 0.96296, 0.96396, 0.96496,
0.96597, 0.96697, 0.96797, 0.96897, 0.96997, 0.97097,
0.97197, 0.97297, 0.97397, 0.97497, 0.97598, 0.97698,
0.97798, 0.97898, 0.97998, 0.98098, 0.98198, 0.98298,
0.98398,
0.98498, 0.98599, 0.98699, 0.98799, 0.98899,
0.98999, 0.99099, 0.99199, 0.99299, 0.99399, 0.99499,
0.996, 0.997, 0.998, 0.999, 1]), array([[
0.068532, 0.068532, 0.12527, ..., 1, 1, 1],
[ 0.050422, 0.050422, 0.082475, ..., 1, 1,
1],
[ 0.05527, 0.05527, 0.096846, ..., 1, 1,
1],
...,
[ 0.041543, 0.041543, 0.075491, ..., 1, 1,
1],
[ 0.062207, 0.062207, 0.11526, ..., 1, 1,
1],
[ 0.059316, 0.059316, 0.10575, ..., 1, 1,
1]]), 'Confidence', 'Precision'], [array([
0.003003, 0.004004, 0.005005, 0.006006, 0.007007, 0.008008,
0.009009, 0.01001, 0.011011, 0.012012, 0.013013, 0.014014,
0.015015, 0.016016, 0.017017, 0.018018, 0.019019, 0.02002,
0.021021, 0.022022, 0.023023,
0.024024, 0.025025, 0.026026, 0.027027, 0.028028,
0.029029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034,
0.035035, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004,
0.041041, 0.042042, 0.043043, 0.044044, 0.045045, 0.046046,
0.047047,
0.048048, 0.049049, 0.05005, 0.051051, 0.052052,
0.053053, 0.054054, 0.055055, 0.056056, 0.057057, 0.058058,

```

0.059059,	0.06006,	0.061061,	0.062062,	0.063063,	0.064064,
0.065065,	0.066066,	0.067067,	0.068068,	0.069069,	0.07007,
0.071071,	0.072072,	0.073073,	0.074074,	0.075075,	0.076076,
0.077077,	0.078078,	0.079079,	0.08008,	0.081081,	0.082082,
0.083083,	0.084084,	0.085085,	0.086086,	0.087087,	0.088088,
0.089089,	0.09009,	0.091091,	0.092092,	0.093093,	0.094094,
0.095095,	0.096096,	0.097097,	0.098098,	0.099099,	0.1001,
0.1011,	0.1021,	0.1031,	0.1041,	0.10511,	0.10611,
0.10711,	0.10811,	0.10911,	0.11011,	0.11111,	0.11211,
0.11311,	0.11411,	0.11512,	0.11612,	0.11712,	0.11812,
0.11912,	0.12012,	0.12112,	0.12212,	0.12312,	0.12412,
0.12513,	0.12613,	0.12713,	0.12813,	0.12913,	0.13013,
0.13113,	0.13213,	0.13313,	0.13413,	0.13514,	0.13614,
0.13714,	0.13814,	0.13914,	0.14014,	0.14114,	0.14214,
0.14314,	0.14414,	0.14515,	0.14615,	0.14715,	0.14815,
0.14915,	0.15015,	0.15115,	0.15215,	0.15315,	0.15415,
0.15516,	0.15616,	0.15716,	0.15816,	0.15916,	0.16016,
0.16116,	0.16216,	0.16316,	0.16416,	0.16517,	0.16617,
0.16717,	0.16817,	0.16917,	0.17017,	0.17117,	0.17217,
0.17317,	0.17417,	0.17518,	0.17618,	0.17718,	0.17818,
0.17918,	0.18018,	0.18118,	0.18218,	0.18318,	0.18418,
0.18519,	0.18619,	0.18719,	0.18819,	0.18919,	0.19019,
0.19119,	0.19219,	0.19319,	0.19419,	0.1952,	0.1962,
0.1972,	0.1982,	0.1992,	0.2002,	0.2012,	0.2022,
0.2032,	0.2042,	0.20521,	0.20621,	0.20721,	0.20821,
0.20921,	0.21021,	0.21121,	0.21221,	0.21321,	0.21421,
0.21522,	0.21622,	0.21722,	0.21822,	0.21922,	0.22022,
0.22122,	0.22222,	0.22322,	0.22422,	0.22523,	0.22623,
0.22723,	0.22823,	0.22923,	0.23023,	0.23123,	0.23223,
0.23323,	0.23423,	0.23524,	0.23624,	0.23724,	0.23824,
0.23924,	0.24024,	0.24124,	0.24224,	0.24324,	0.24424,
0.24525,	0.24625,	0.24725,	0.24825,	0.24925,	0.25025,
0.25125,	0.25225,	0.25325,	0.25425,	0.25526,	0.25626,
0.25726,	0.25826,	0.25926,	0.26026,	0.26126,	0.26226,
0.26326,	0.26426,	0.26527,	0.26627,	0.26727,	0.26827,
0.26927,	0.27027,	0.27127,	0.27227,	0.27327,	0.27427,
0.27528,	0.27628,	0.27728,	0.27828,	0.27928,	0.28028,
0.28128,	0.28228,	0.28328,	0.28428,	0.28529,	0.28629,

0.28729,					
	0.28829,	0.28929,	0.29029,	0.29129,	0.29229,
0.29329,	0.29429,	0.2953,	0.2963,	0.2973,	0.2983,
0.2993,	0.3003,	0.3013,	0.3023,	0.3033,	0.3043,
0.30531,	0.30631,	0.30731,	0.30831,	0.30931,	0.31031,
0.31131,					
	0.31231,	0.31331,	0.31431,	0.31532,	0.31632,
0.31732,	0.31832,	0.31932,	0.32032,	0.32132,	0.32232,
0.32332,	0.32432,	0.32533,	0.32633,	0.32733,	0.32833,
0.32933,	0.33033,	0.33133,	0.33233,	0.33333,	0.33433,
0.33534,					
	0.33634,	0.33734,	0.33834,	0.33934,	0.34034,
0.34134,	0.34234,	0.34334,	0.34434,	0.34535,	0.34635,
0.34735,	0.34835,	0.34935,	0.35035,	0.35135,	0.35235,
0.35335,	0.35435,	0.35536,	0.35636,	0.35736,	0.35836,
0.35936,					
	0.36036,	0.36136,	0.36236,	0.36336,	0.36436,
0.36537,	0.36637,	0.36737,	0.36837,	0.36937,	0.37037,
0.37137,	0.37237,	0.37337,	0.37437,	0.37538,	0.37638,
0.37738,	0.37838,	0.37938,	0.38038,	0.38138,	0.38238,
0.38338,					
	0.38438,	0.38539,	0.38639,	0.38739,	0.38839,
0.38939,	0.39039,	0.39139,	0.39239,	0.39339,	0.39439,
0.3954,	0.3964,	0.3974,	0.3984,	0.3994,	0.4004,
0.4014,	0.4024,	0.4034,	0.4044,	0.40541,	0.40641,
0.40741,					
	0.40841,	0.40941,	0.41041,	0.41141,	0.41241,
0.41341,	0.41441,	0.41542,	0.41642,	0.41742,	0.41842,
0.41942,	0.42042,	0.42142,	0.42242,	0.42342,	0.42442,
0.42543,	0.42643,	0.42743,	0.42843,	0.42943,	0.43043,
0.43143,					
	0.43243,	0.43343,	0.43443,	0.43544,	0.43644,
0.43744,	0.43844,	0.43944,	0.44044,	0.44144,	0.44244,
0.44344,	0.44444,	0.44545,	0.44645,	0.44745,	0.44845,
0.44945,	0.45045,	0.45145,	0.45245,	0.45345,	0.45445,
0.45546,					
	0.45646,	0.45746,	0.45846,	0.45946,	0.46046,
0.46146,	0.46246,	0.46346,	0.46446,	0.46547,	0.46647,
0.46747,	0.46847,	0.46947,	0.47047,	0.47147,	0.47247,
0.47347,	0.47447,	0.47548,	0.47648,	0.47748,	0.47848,
0.47948,					
	0.48048,	0.48148,	0.48248,	0.48348,	0.48448,
0.48549,	0.48649,	0.48749,	0.48849,	0.48949,	0.49049,
0.49149,	0.49249,	0.49349,	0.49449,	0.4955,	0.4965,
0.4975,	0.4985,	0.4995,	0.5005,	0.5015,	0.5025,
0.5035,					
	0.5045,	0.50551,	0.50651,	0.50751,	0.50851,

0.50951,	0.51051,	0.51151,	0.51251,	0.51351,	0.51451,
0.51552,	0.51652,	0.51752,	0.51852,	0.51952,	0.52052,
0.52152,	0.52252,	0.52352,	0.52452,	0.52553,	0.52653,
0.52753,					
	0.52853,	0.52953,	0.53053,	0.53153,	0.53253,
0.53353,	0.53453,	0.53554,	0.53654,	0.53754,	0.53854,
0.53954,	0.54054,	0.54154,	0.54254,	0.54354,	0.54454,
0.54555,	0.54655,	0.54755,	0.54855,	0.54955,	0.55055,
0.55155,					
	0.55255,	0.55355,	0.55455,	0.55556,	0.55656,
0.55756,	0.55856,	0.55956,	0.56056,	0.56156,	0.56256,
0.56356,	0.56456,	0.56557,	0.56657,	0.56757,	0.56857,
0.56957,	0.57057,	0.57157,	0.57257,	0.57357,	0.57457,
0.57558,					
	0.57658,	0.57758,	0.57858,	0.57958,	0.58058,
0.58158,	0.58258,	0.58358,	0.58458,	0.58559,	0.58659,
0.58759,	0.58859,	0.58959,	0.59059,	0.59159,	0.59259,
0.59359,	0.59459,	0.5956,	0.5966,	0.5976,	0.5986,
0.5996,					
	0.6006,	0.6016,	0.6026,	0.6036,	0.6046,
0.60561,	0.60661,	0.60761,	0.60861,	0.60961,	0.61061,
0.61161,	0.61261,	0.61361,	0.61461,	0.61562,	0.61662,
0.61762,	0.61862,	0.61962,	0.62062,	0.62162,	0.62262,
0.62362,					
	0.62462,	0.62563,	0.62663,	0.62763,	0.62863,
0.62963,	0.63063,	0.63163,	0.63263,	0.63363,	0.63463,
0.63564,	0.63664,	0.63764,	0.63864,	0.63964,	0.64064,
0.64164,	0.64264,	0.64364,	0.64464,	0.64565,	0.64665,
0.64765,					
	0.64865,	0.64965,	0.65065,	0.65165,	0.65265,
0.65365,	0.65465,	0.65566,	0.65666,	0.65766,	0.65866,
0.65966,	0.66066,	0.66166,	0.66266,	0.66366,	0.66466,
0.66567,	0.66667,	0.66767,	0.66867,	0.66967,	0.67067,
0.67167,					
	0.67267,	0.67367,	0.67467,	0.67568,	0.67668,
0.67768,	0.67868,	0.67968,	0.68068,	0.68168,	0.68268,
0.68368,	0.68468,	0.68569,	0.68669,	0.68769,	0.68869,
0.68969,	0.69069,	0.69169,	0.69269,	0.69369,	0.69469,
0.6957,					
	0.6967,	0.6977,	0.6987,	0.6997,	0.7007,
0.7017,	0.7027,	0.7037,	0.7047,	0.70571,	0.70671,
0.70771,	0.70871,	0.70971,	0.71071,	0.71171,	0.71271,
0.71371,	0.71471,	0.71572,	0.71672,	0.71772,	0.71872,
0.71972,					
	0.72072,	0.72172,	0.72272,	0.72372,	0.72472,
0.72573,	0.72673,	0.72773,	0.72873,	0.72973,	0.73073,
0.73173,	0.73273,	0.73373,	0.73473,	0.73574,	0.73674,

0.73774, 0.74374,	0.73874,	0.73974,	0.74074,	0.74174,	0.74274,
	0.74474,	0.74575,	0.74675,	0.74775,	0.74875,
0.74975,	0.75075,	0.75175,	0.75275,	0.75375,	0.75475,
0.75576,	0.75676,	0.75776,	0.75876,	0.75976,	0.76076,
0.76176,	0.76276,	0.76376,	0.76476,	0.76577,	0.76677,
0.76777,					
	0.76877,	0.76977,	0.77077,	0.77177,	0.77277,
0.77377,	0.77477,	0.77578,	0.77678,	0.77778,	0.77878,
0.77978,	0.78078,	0.78178,	0.78278,	0.78378,	0.78478,
0.78579,	0.78679,	0.78779,	0.78879,	0.78979,	0.79079,
0.79179,					
	0.79279,	0.79379,	0.79479,	0.7958,	0.7968,
0.7978,	0.7988,	0.7998,	0.8008,	0.8018,	0.8028,
0.8038,	0.8048,	0.80581,	0.80681,	0.80781,	0.80881,
0.80981,	0.81081,	0.81181,	0.81281,	0.81381,	0.81481,
0.81582,					
	0.81682,	0.81782,	0.81882,	0.81982,	0.82082,
0.82182,	0.82282,	0.82382,	0.82482,	0.82583,	0.82683,
0.82783,	0.82883,	0.82983,	0.83083,	0.83183,	0.83283,
0.83383,	0.83483,	0.83584,	0.83684,	0.83784,	0.83884,
0.83984,					
	0.84084,	0.84184,	0.84284,	0.84384,	0.84484,
0.84585,	0.84685,	0.84785,	0.84885,	0.84985,	0.85085,
0.85185,	0.85285,	0.85385,	0.85485,	0.85586,	0.85686,
0.85786,	0.85886,	0.85986,	0.86086,	0.86186,	0.86286,
0.86386,					
	0.86486,	0.86587,	0.86687,	0.86787,	0.86887,
0.86987,	0.87087,	0.87187,	0.87287,	0.87387,	0.87487,
0.87588,	0.87688,	0.87788,	0.87888,	0.87988,	0.88088,
0.88188,	0.88288,	0.88388,	0.88488,	0.88589,	0.88689,
0.88789,					
	0.88889,	0.88989,	0.89089,	0.89189,	0.89289,
0.89389,	0.89489,	0.8959,	0.8969,	0.8979,	0.8989,
0.8999,	0.9009,	0.9019,	0.9029,	0.9039,	0.9049,
0.90591,	0.90691,	0.90791,	0.90891,	0.90991,	0.91091,
0.91191,					
	0.91291,	0.91391,	0.91491,	0.91592,	0.91692,
0.91792,	0.91892,	0.91992,	0.92092,	0.92192,	0.92292,
0.92392,	0.92492,	0.92593,	0.92693,	0.92793,	0.92893,
0.92993,	0.93093,	0.93193,	0.93293,	0.93393,	0.93493,
0.93594,					
	0.93694,	0.93794,	0.93894,	0.93994,	0.94094,
0.94194,	0.94294,	0.94394,	0.94494,	0.94595,	0.94695,
0.94795,	0.94895,	0.94995,	0.95095,	0.95195,	0.95295,
0.95395,	0.95495,	0.95596,	0.95696,	0.95796,	0.95896,
0.95996,					


```

0.96597, 0.96096, 0.96196, 0.96296, 0.96396, 0.96496,
0.97197, 0.96697, 0.96797, 0.96897, 0.96997, 0.97097,
0.97798, 0.97297, 0.97397, 0.97497, 0.97598, 0.97698,
0.98398, 0.97898, 0.97998, 0.98098, 0.98198, 0.98298,
0.98498, 0.98599, 0.98699, 0.98799, 0.98899,
0.98999, 0.99099, 0.99199, 0.99299, 0.99399, 0.99499,
0.996, 0.997, 0.998, 0.999, 1]), array([[
0.96571, 0.96571, 0.96, ..., 0, 0, 0],
[ 0.91204, 0.91204, 0.89352, ..., 0, 0,
0],
[ 0.8459, 0.8459, 0.81639, ..., 0, 0,
0],
...,
[ 0.94686, 0.94686, 0.92271, ..., 0, 0,
0],
[ 0.93529, 0.93529, 0.92353, ..., 0, 0,
0],
[ 0.92614, 0.92614, 0.89773, ..., 0, 0,
0]]), 'Confidence', 'Recall'])
fitness: np.float64(0.508253711054525)
keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)',
'metrics/mAP50-95(B)']
maps: array([ 0.60975, 0.54936, 0.38948, 0.35653, 0.35758,
0.61783, 0.5935, 0.64571, 0.34734, 0.55255, 0.44202,
0.59862, 0.6197, 0.56289, 0.53296, 0.25026, 0.46121,
0.48152, 0.64269, 0.55358])
names: {0: 'aeroplane', 1: 'bicycle', 2: 'bird', 3: 'boat', 4: 'bottle', 5:
'bus', 6: 'car', 7: 'cat', 8: 'chair', 9: 'cow', 10: 'diningtable', 11: 'dog',
12: 'horse', 13: 'motorbike', 14: 'person', 15: 'pottedplant', 16: 'sheep', 17:
'sofa', 18: 'train', 19: 'tvmonitor'}
nt_per_class: array([ 175, 216, 305, 190, 296, 141, 818, 198, 706, 171,
162, 267, 199, 197, 2742, 320, 162, 207, 170, 176])
nt_per_image: array([ 127, 133, 151, 101, 109, 97, 359, 178, 290, 75,
133, 220, 150, 126, 1025, 120, 48, 184, 135, 135])
results_dict: {'metrics/precision(B)': 0.7474706952050233, 'metrics/recall(B)':
0.6529819707212455, 'metrics/mAP50(B)': 0.7239952377512701,
'metrics/mAP50-95(B)': 0.508253711054525, 'fitness': 0.508253711054525}
save_dir: PosixPath('/content/runs/detect/train')
speed: {'preprocess': 0.17357870956112745, 'inference': 1.8305190370505358,
'loss': 0.00037374063875872653, 'postprocess': 1.4765085980077102}
stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [], 'target_img':
[]}
task: 'detect'

```

```

[44]: import matplotlib.pyplot as plt
from pathlib import Path

```

```

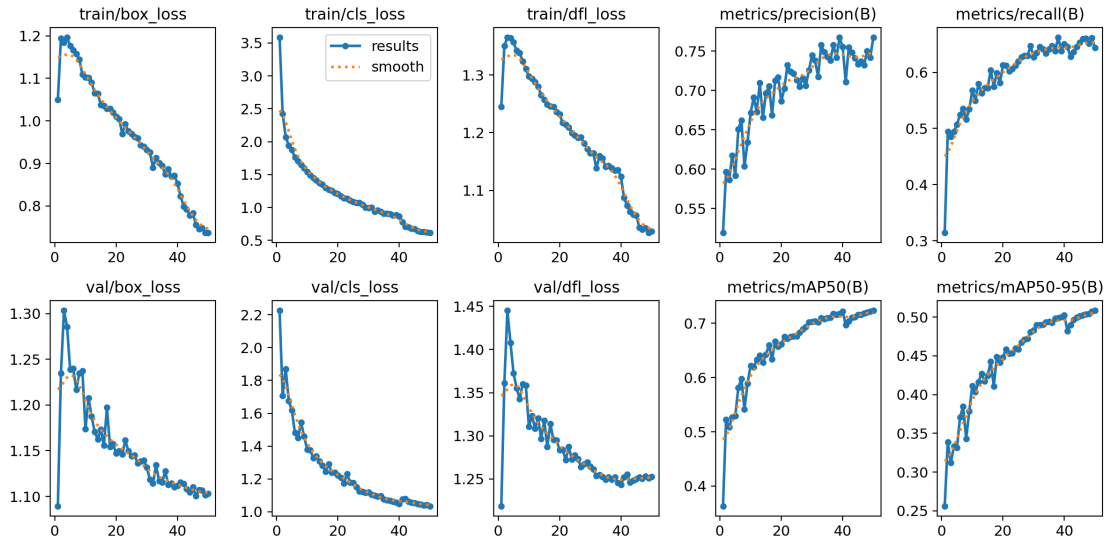
def plot_yolo_training_curves(run_dir):
    """
    Plot YOLO training losses and AP curves from results.csv or results.png.
    """
    run_dir = Path(run_dir)
    results_csv = run_dir / "results.csv"
    results_png = run_dir / "results.png"

    # YOLO already generates results.png automatically
    if results_png.exists():
        from IPython.display import Image
        display(Image(filename=results_png, width=800))
        print(" Displayed Ultralytics built-in training summary.")
    elif results_csv.exists():
        import pandas as pd
        df = pd.read_csv(results_csv)
        # Plot Loss curves
        plt.figure(figsize=(8,5))
        plt.plot(df["train/box_loss"], label="Box Loss")
        plt.plot(df["train/cls_loss"], label="Cls Loss")
        plt.plot(df["val/box_loss"], label="Val Box Loss")
        plt.xlabel("Epoch")
        plt.ylabel("Loss")
        plt.title("Training and Validation Loss")
        plt.legend()
        plt.show()

        # Plot mAP
        plt.figure(figsize=(8,5))
        plt.plot(df["metrics/mAP50"], label="mAP@0.5")
        plt.plot(df["metrics/mAP50-95"], label="mAP@0.5:0.95")
        plt.xlabel("Epoch")
        plt.ylabel("mAP")
        plt.title("Mean Average Precision (Validation)")
        plt.legend()
        plt.show()
    else:
        print(" No results.csv or results.png found in", run_dir)

```

```
[45]: plot_yolo_training_curves("/content/runs/detect/train")
```



Displayed Ultralytics built-in training summary.

```
[46]: save_model(model, name="voc2007_yolov11n", folder="saved_models")
```

Ultralytics YOLO model saved to saved_models/voc2007_yolov11n.pt

```
[46]: 'saved_models/voc2007_yolov11n.pt'
```

```
[47]: import matplotlib.pyplot as plt
import cv2
from pathlib import Path
from ultralytics import YOLO

def plot_yolo_test_predictions(model_path, test_dir, conf=0.25, num_images=5,
    ↪save_dir="/content/yolo_test_results"):
    """
    Run inference on a few test images and display annotated predictions inline.

    Args:
        model_path (str): Path to trained YOLO model (e.g., best.pt)
        test_dir (str): Path to test image directory
        conf (float): Confidence threshold for predictions
        num_images (int): Number of random test images to visualize
        save_dir (str): Directory where annotated images will be saved
    """
    import random
    model = YOLO(model_path)
    test_dir = Path(test_dir)
```

```

# Pick random sample images
imgs = sorted(test_dir.glob("*.jpg"))
if len(imgs) == 0:
    raise FileNotFoundError(f"No test images found in {test_dir}")
sample_imgs = random.sample(imgs, min(num_images, len(imgs)))

# Run predictions
results = model.predict(source=sample_imgs, conf=conf, save=True,
project=save_dir)
print(f" Predictions saved at: {results[0].save_dir}")

# Display inline
fig, axes = plt.subplots(1, len(sample_imgs), figsize=(20, 6))
if len(sample_imgs) == 1:
    axes = [axes]
for ax, img_path in zip(axes, sample_imgs):
    out_path = Path(results[0].save_dir) / img_path.name
    img = cv2.cvtColor(cv2.imread(str(out_path)), cv2.COLOR_BGR2RGB)
    ax.imshow(img)
    ax.set_title(img_path.name)
    ax.axis("off")
plt.show()

```

```

[48]: plot_yolo_test_predictions(
    # model_path="/content/yolo_results/voc2012_dynamic/weights/best.pt",
    model_path="/content/saved_models/voc2007_yolov11n.pt",
    test_dir="/content/DLCV_Lab_Dataset/VOC2007_yolo/images/test",
    conf=0.25,
    num_images=4
)

```

```

0: 640x640 1 bicycle, 1 motorbike, 1 person, 12.6ms
1: 640x640 2 trains, 12.6ms
2: 640x640 1 chair, 1 sofa, 12.6ms
3: 640x640 2 bicycles, 2 persons, 12.6ms
Speed: 2.8ms preprocess, 12.6ms inference, 0.8ms postprocess per image at shape
(1, 3, 640, 640)
Results saved to /content/yolo_test_results/predict
Predictions saved at: /content/yolo_test_results/predict

```



```
[49]: # evaluate_ultra_yolo(model, yaml_path)
metrics = model.val()
print(metrics)
```

Ultralytics 8.3.224 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4, 15095MiB)
YOLO11n summary (fused): 100 layers, 2,586,052 parameters, 0 gradients, 6.3 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 1836.3±238.9 MB/s, size: 106.1 KB)
val: Scanning
/content/DLCV_Lab_Dataset/VOC2007_yolo/labels/val.cache... 2510 images, 0 backgrounds, 0 corrupt: 100% 2510/2510 4.0Mit/s 0.0s

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	all	157/157	4.9it/s 31.9s			
0.508		2510	7818	0.768	0.644	0.724
0.609	aeroplane	127	175	0.821	0.789	0.86
0.548	bicycle	133	216	0.769	0.664	0.758
0.387	bird	151	305	0.77	0.518	0.614
0.355	boat	101	190	0.729	0.489	0.593
0.355	bottle	109	296	0.763	0.436	0.567
0.612	bus	97	141	0.797	0.645	0.734
0.594	car	359	818	0.82	0.744	0.82
0.649	cat	178	198	0.81	0.732	0.818
0.347	chair	290	706	0.683	0.472	0.547

0.552	cow	75	171	0.771	0.668	0.759
0.441	diningtable	133	162	0.752	0.519	0.642
0.596	dog	220	267	0.729	0.727	0.789
0.62	horse	150	199	0.82	0.799	0.826
0.566	motorbike	126	197	0.733	0.761	0.8
0.532	person	1025	2742	0.859	0.739	0.83
0.257	pottedplant	120	320	0.673	0.425	0.512
0.459	sheep	48	162	0.672	0.709	0.721
0.484	sofa	184	207	0.687	0.584	0.665
0.643	train	135	170	0.891	0.772	0.858
0.551	tvmonitor	135	176	0.812	0.688	0.769

Speed: 1.2ms preprocess, 3.9ms inference, 0.0ms loss, 1.8ms postprocess per image

Results saved to /content/runs/detect/val

ultralytics.utils.metrics.DetMetrics object with attributes:

ap_class_index: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19])

box: ultralytics.utils.metrics.Metric object

confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at 0x7d566bc4ac30>

curves: ['Precision-Recall(B)', 'F1-Confidence(B)', 'Precision-Confidence(B)', 'Recall-Confidence(B)']

curves_results: [[array([0, 0.001001, 0.002002, 0.003003, 0.004004, 0.005005, 0.006006, 0.007007, 0.008008, 0.009009, 0.01001, 0.011011, 0.012012, 0.013013, 0.014014, 0.015015, 0.016016, 0.017017, 0.018018, 0.019019, 0.02002, 0.021021, 0.022022, 0.023023, 0.024024, 0.025025, 0.026026, 0.027027, 0.028028, 0.029029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034, 0.035035, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004, 0.041041, 0.042042, 0.043043, 0.044044, 0.045045, 0.046046, 0.047047, 0.048048, 0.049049, 0.05005, 0.051051, 0.052052, 0.053053, 0.054054, 0.055055, 0.056056, 0.057057, 0.058058, 0.059059, 0.06006, 0.061061, 0.062062, 0.063063, 0.064064, 0.065065, 0.066066, 0.067067, 0.068068, 0.069069, 0.07007,

0.071071,					
	0.072072,	0.073073,	0.074074,	0.075075,	0.076076,
0.077077,	0.078078,	0.079079,	0.08008,	0.081081,	0.082082,
0.083083,	0.084084,	0.085085,	0.086086,	0.087087,	0.088088,
0.089089,	0.09009,	0.091091,	0.092092,	0.093093,	0.094094,
0.095095,					
	0.096096,	0.097097,	0.098098,	0.099099,	0.1001,
0.1011,	0.1021,	0.1031,	0.1041,	0.10511,	0.10611,
0.10711,	0.10811,	0.10911,	0.11011,	0.11111,	0.11211,
0.11311,	0.11411,	0.11512,	0.11612,	0.11712,	0.11812,
0.11912,					
	0.12012,	0.12112,	0.12212,	0.12312,	0.12412,
0.12513,	0.12613,	0.12713,	0.12813,	0.12913,	0.13013,
0.13113,	0.13213,	0.13313,	0.13413,	0.13514,	0.13614,
0.13714,	0.13814,	0.13914,	0.14014,	0.14114,	0.14214,
0.14314,					
	0.14414,	0.14515,	0.14615,	0.14715,	0.14815,
0.14915,	0.15015,	0.15115,	0.15215,	0.15315,	0.15415,
0.15516,	0.15616,	0.15716,	0.15816,	0.15916,	0.16016,
0.16116,	0.16216,	0.16316,	0.16416,	0.16517,	0.16617,
0.16717,					
	0.16817,	0.16917,	0.17017,	0.17117,	0.17217,
0.17317,	0.17417,	0.17518,	0.17618,	0.17718,	0.17818,
0.17918,	0.18018,	0.18118,	0.18218,	0.18318,	0.18418,
0.18519,	0.18619,	0.18719,	0.18819,	0.18919,	0.19019,
0.19119,					
	0.19219,	0.19319,	0.19419,	0.1952,	0.1962,
0.1972,	0.1982,	0.1992,	0.2002,	0.2012,	0.2022,
0.2032,	0.2042,	0.20521,	0.20621,	0.20721,	0.20821,
0.20921,	0.21021,	0.21121,	0.21221,	0.21321,	0.21421,
0.21522,					
	0.21622,	0.21722,	0.21822,	0.21922,	0.22022,
0.22122,	0.22222,	0.22322,	0.22422,	0.22523,	0.22623,
0.22723,	0.22823,	0.22923,	0.23023,	0.23123,	0.23223,
0.23323,	0.23423,	0.23524,	0.23624,	0.23724,	0.23824,
0.23924,					
	0.24024,	0.24124,	0.24224,	0.24324,	0.24424,
0.24525,	0.24625,	0.24725,	0.24825,	0.24925,	0.25025,
0.25125,	0.25225,	0.25325,	0.25425,	0.25526,	0.25626,
0.25726,	0.25826,	0.25926,	0.26026,	0.26126,	0.26226,
0.26326,					
	0.26426,	0.26527,	0.26627,	0.26727,	0.26827,
0.26927,	0.27027,	0.27127,	0.27227,	0.27327,	0.27427,
0.27528,	0.27628,	0.27728,	0.27828,	0.27928,	0.28028,
0.28128,	0.28228,	0.28328,	0.28428,	0.28529,	0.28629,
0.28729,					
	0.28829,	0.28929,	0.29029,	0.29129,	0.29229,
0.29329,	0.29429,	0.2953,	0.2963,	0.2973,	0.2983,

0.2993,	0.3003,	0.3013,	0.3023,	0.3033,	0.3043,
0.30531,	0.30631,	0.30731,	0.30831,	0.30931,	0.31031,
0.31131,					
	0.31231,	0.31331,	0.31431,	0.31532,	0.31632,
0.31732,	0.31832,	0.31932,	0.32032,	0.32132,	0.32232,
0.32332,	0.32432,	0.32533,	0.32633,	0.32733,	0.32833,
0.32933,	0.33033,	0.33133,	0.33233,	0.33333,	0.33433,
0.33534,					
	0.33634,	0.33734,	0.33834,	0.33934,	0.34034,
0.34134,	0.34234,	0.34334,	0.34434,	0.34535,	0.34635,
0.34735,	0.34835,	0.34935,	0.35035,	0.35135,	0.35235,
0.35335,	0.35435,	0.35536,	0.35636,	0.35736,	0.35836,
0.35936,					
	0.36036,	0.36136,	0.36236,	0.36336,	0.36436,
0.36537,	0.36637,	0.36737,	0.36837,	0.36937,	0.37037,
0.37137,	0.37237,	0.37337,	0.37437,	0.37538,	0.37638,
0.37738,	0.37838,	0.37938,	0.38038,	0.38138,	0.38238,
0.38338,					
	0.38438,	0.38539,	0.38639,	0.38739,	0.38839,
0.38939,	0.39039,	0.39139,	0.39239,	0.39339,	0.39439,
0.3954,	0.3964,	0.3974,	0.3984,	0.3994,	0.4004,
0.4014,	0.4024,	0.4034,	0.4044,	0.40541,	0.40641,
0.40741,					
	0.40841,	0.40941,	0.41041,	0.41141,	0.41241,
0.41341,	0.41441,	0.41542,	0.41642,	0.41742,	0.41842,
0.41942,	0.42042,	0.42142,	0.42242,	0.42342,	0.42442,
0.42543,	0.42643,	0.42743,	0.42843,	0.42943,	0.43043,
0.43143,					
	0.43243,	0.43343,	0.43443,	0.43544,	0.43644,
0.43744,	0.43844,	0.43944,	0.44044,	0.44144,	0.44244,
0.44344,	0.44444,	0.44545,	0.44645,	0.44745,	0.44845,
0.44945,	0.45045,	0.45145,	0.45245,	0.45345,	0.45445,
0.45546,					
	0.45646,	0.45746,	0.45846,	0.45946,	0.46046,
0.46146,	0.46246,	0.46346,	0.46446,	0.46547,	0.46647,
0.46747,	0.46847,	0.46947,	0.47047,	0.47147,	0.47247,
0.47347,	0.47447,	0.47548,	0.47648,	0.47748,	0.47848,
0.47948,					
	0.48048,	0.48148,	0.48248,	0.48348,	0.48448,
0.48549,	0.48649,	0.48749,	0.48849,	0.48949,	0.49049,
0.49149,	0.49249,	0.49349,	0.49449,	0.4955,	0.4965,
0.4975,	0.4985,	0.4995,	0.5005,	0.5015,	0.5025,
0.5035,					
	0.5045,	0.50551,	0.50651,	0.50751,	0.50851,
0.50951,	0.51051,	0.51151,	0.51251,	0.51351,	0.51451,
0.51552,	0.51652,	0.51752,	0.51852,	0.51952,	0.52052,
0.52152,	0.52252,	0.52352,	0.52452,	0.52553,	0.52653,
0.52753,					

	0.52853,	0.52953,	0.53053,	0.53153,	0.53253,
0.53353,	0.53453,	0.53554,	0.53654,	0.53754,	0.53854,
0.53954,	0.54054,	0.54154,	0.54254,	0.54354,	0.54454,
0.54555,	0.54655,	0.54755,	0.54855,	0.54955,	0.55055,
0.55155,					
	0.55255,	0.55355,	0.55455,	0.55556,	0.55656,
0.55756,	0.55856,	0.55956,	0.56056,	0.56156,	0.56256,
0.56356,	0.56456,	0.56557,	0.56657,	0.56757,	0.56857,
0.56957,	0.57057,	0.57157,	0.57257,	0.57357,	0.57457,
0.57558,					
	0.57658,	0.57758,	0.57858,	0.57958,	0.58058,
0.58158,	0.58258,	0.58358,	0.58458,	0.58559,	0.58659,
0.58759,	0.58859,	0.58959,	0.59059,	0.59159,	0.59259,
0.59359,	0.59459,	0.5956,	0.5966,	0.5976,	0.5986,
0.5996,					
	0.6006,	0.6016,	0.6026,	0.6036,	0.6046,
0.60561,	0.60661,	0.60761,	0.60861,	0.60961,	0.61061,
0.61161,	0.61261,	0.61361,	0.61461,	0.61562,	0.61662,
0.61762,	0.61862,	0.61962,	0.62062,	0.62162,	0.62262,
0.62362,					
	0.62462,	0.62563,	0.62663,	0.62763,	0.62863,
0.62963,	0.63063,	0.63163,	0.63263,	0.63363,	0.63463,
0.63564,	0.63664,	0.63764,	0.63864,	0.63964,	0.64064,
0.64164,	0.64264,	0.64364,	0.64464,	0.64565,	0.64665,
0.64765,					
	0.64865,	0.64965,	0.65065,	0.65165,	0.65265,
0.65365,	0.65465,	0.65566,	0.65666,	0.65766,	0.65866,
0.65966,	0.66066,	0.66166,	0.66266,	0.66366,	0.66466,
0.66567,	0.66667,	0.66767,	0.66867,	0.66967,	0.67067,
0.67167,					
	0.67267,	0.67367,	0.67467,	0.67568,	0.67668,
0.67768,	0.67868,	0.67968,	0.68068,	0.68168,	0.68268,
0.68368,	0.68468,	0.68569,	0.68669,	0.68769,	0.68869,
0.68969,	0.69069,	0.69169,	0.69269,	0.69369,	0.69469,
0.6957,					
	0.6967,	0.6977,	0.6987,	0.6997,	0.7007,
0.7017,	0.7027,	0.7037,	0.7047,	0.70571,	0.70671,
0.70771,	0.70871,	0.70971,	0.71071,	0.71171,	0.71271,
0.71371,	0.71471,	0.71572,	0.71672,	0.71772,	0.71872,
0.71972,					
	0.72072,	0.72172,	0.72272,	0.72372,	0.72472,
0.72573,	0.72673,	0.72773,	0.72873,	0.72973,	0.73073,
0.73173,	0.73273,	0.73373,	0.73473,	0.73574,	0.73674,
0.73774,	0.73874,	0.73974,	0.74074,	0.74174,	0.74274,
0.74374,					
	0.74474,	0.74575,	0.74675,	0.74775,	0.74875,
0.74975,	0.75075,	0.75175,	0.75275,	0.75375,	0.75475,
0.75576,	0.75676,	0.75776,	0.75876,	0.75976,	0.76076,

0.76176, 0.76777,	0.76276,	0.76376,	0.76476,	0.76577,	0.76677,
	0.76877,	0.76977,	0.77077,	0.77177,	0.77277,
0.77377,	0.77477,	0.77578,	0.77678,	0.77778,	0.77878,
0.77978,	0.78078,	0.78178,	0.78278,	0.78378,	0.78478,
0.78579,	0.78679,	0.78779,	0.78879,	0.78979,	0.79079,
0.79179,					
	0.79279,	0.79379,	0.79479,	0.7958,	0.7968,
0.7978,	0.7988,	0.7998,	0.8008,	0.8018,	0.8028,
0.8038,	0.8048,	0.80581,	0.80681,	0.80781,	0.80881,
0.80981,	0.81081,	0.81181,	0.81281,	0.81381,	0.81481,
0.81582,					
	0.81682,	0.81782,	0.81882,	0.81982,	0.82082,
0.82182,	0.82282,	0.82382,	0.82482,	0.82583,	0.82683,
0.82783,	0.82883,	0.82983,	0.83083,	0.83183,	0.83283,
0.83383,	0.83483,	0.83584,	0.83684,	0.83784,	0.83884,
0.83984,					
	0.84084,	0.84184,	0.84284,	0.84384,	0.84484,
0.84585,	0.84685,	0.84785,	0.84885,	0.84985,	0.85085,
0.85185,	0.85285,	0.85385,	0.85485,	0.85586,	0.85686,
0.85786,	0.85886,	0.85986,	0.86086,	0.86186,	0.86286,
0.86386,					
	0.86486,	0.86587,	0.86687,	0.86787,	0.86887,
0.86987,	0.87087,	0.87187,	0.87287,	0.87387,	0.87487,
0.87588,	0.87688,	0.87788,	0.87888,	0.87988,	0.88088,
0.88188,	0.88288,	0.88388,	0.88488,	0.88589,	0.88689,
0.88789,					
	0.88889,	0.88989,	0.89089,	0.89189,	0.89289,
0.89389,	0.89489,	0.8959,	0.8969,	0.8979,	0.8989,
0.8999,	0.9009,	0.9019,	0.9029,	0.9039,	0.9049,
0.90591,	0.90691,	0.90791,	0.90891,	0.90991,	0.91091,
0.91191,					
	0.91291,	0.91391,	0.91491,	0.91592,	0.91692,
0.91792,	0.91892,	0.91992,	0.92092,	0.92192,	0.92292,
0.92392,	0.92492,	0.92593,	0.92693,	0.92793,	0.92893,
0.92993,	0.93093,	0.93193,	0.93293,	0.93393,	0.93493,
0.93594,					
	0.93694,	0.93794,	0.93894,	0.93994,	0.94094,
0.94194,	0.94294,	0.94394,	0.94494,	0.94595,	0.94695,
0.94795,	0.94895,	0.94995,	0.95095,	0.95195,	0.95295,
0.95395,	0.95495,	0.95596,	0.95696,	0.95796,	0.95896,
0.95996,					
	0.96096,	0.96196,	0.96296,	0.96396,	0.96496,
0.96597,	0.96697,	0.96797,	0.96897,	0.96997,	0.97097,
0.97197,	0.97297,	0.97397,	0.97497,	0.97598,	0.97698,
0.97798,	0.97898,	0.97998,	0.98098,	0.98198,	0.98298,
0.98398,					
	0.98498,	0.98599,	0.98699,	0.98799,	0.98899,

```

0.98999,    0.99099,    0.99199,    0.99299,    0.99399,    0.99499,
0.996,      0.997,      0.998,      0.999,      1]], array([[
1,          1,          1, ..., 0.0039823, 0.0019912, 0],
0],
[          1,          1,          1, ..., 0.0011406, 0.00057029,
0],
...,
[          1,          1,          1, ..., 0.0014177, 0.00070887,
0],
[          1,          1,          1, ..., 0.0019179, 0.00095896,
0],
[          1,          1,          1, ..., 0.0016077, 0.00080385,
0]]), 'Recall', 'Precision'], [array([
0,          0.001001, 0.002002,
0.003003, 0.004004, 0.005005, 0.006006, 0.007007, 0.008008,
0.009009, 0.01001, 0.011011, 0.012012, 0.013013, 0.014014,
0.015015, 0.016016, 0.017017, 0.018018, 0.019019, 0.02002,
0.021021, 0.022022, 0.023023,
0.024024, 0.025025, 0.026026, 0.027027, 0.028028,
0.029029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034,
0.035035, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004,
0.041041, 0.042042, 0.043043, 0.044044, 0.045045, 0.046046,
0.047047,
0.048048, 0.049049, 0.05005, 0.051051, 0.052052,
0.053053, 0.054054, 0.055055, 0.056056, 0.057057, 0.058058,
0.059059, 0.06006, 0.061061, 0.062062, 0.063063, 0.064064,
0.065065, 0.066066, 0.067067, 0.068068, 0.069069, 0.07007,
0.071071,
0.072072, 0.073073, 0.074074, 0.075075, 0.076076,
0.077077, 0.078078, 0.079079, 0.08008, 0.081081, 0.082082,
0.083083, 0.084084, 0.085085, 0.086086, 0.087087, 0.088088,
0.089089, 0.09009, 0.091091, 0.092092, 0.093093, 0.094094,
0.095095,
0.096096, 0.097097, 0.098098, 0.099099, 0.1001,
0.1011, 0.1021, 0.1031, 0.1041, 0.10511, 0.10611,
0.10711, 0.10811, 0.10911, 0.11011, 0.11111, 0.11211,
0.11311, 0.11411, 0.11512, 0.11612, 0.11712, 0.11812,
0.11912,
0.12012, 0.12112, 0.12212, 0.12312, 0.12412,
0.12513, 0.12613, 0.12713, 0.12813, 0.12913, 0.13013,
0.13113, 0.13213, 0.13313, 0.13413, 0.13514, 0.13614,
0.13714, 0.13814, 0.13914, 0.14014, 0.14114, 0.14214,
0.14314,
0.14414, 0.14515, 0.14615, 0.14715, 0.14815,
0.14915, 0.15015, 0.15115, 0.15215, 0.15315, 0.15415,
0.15516, 0.15616, 0.15716, 0.15816, 0.15916, 0.16016,
0.16116, 0.16216, 0.16316, 0.16416, 0.16517, 0.16617,
0.16717,

```

	0.16817,	0.16917,	0.17017,	0.17117,	0.17217,
0.17317,	0.17417,	0.17518,	0.17618,	0.17718,	0.17818,
0.17918,	0.18018,	0.18118,	0.18218,	0.18318,	0.18418,
0.18519,	0.18619,	0.18719,	0.18819,	0.18919,	0.19019,
0.19119,					
	0.19219,	0.19319,	0.19419,	0.1952,	0.1962,
0.1972,	0.1982,	0.1992,	0.2002,	0.2012,	0.2022,
0.2032,	0.2042,	0.20521,	0.20621,	0.20721,	0.20821,
0.20921,	0.21021,	0.21121,	0.21221,	0.21321,	0.21421,
0.21522,					
	0.21622,	0.21722,	0.21822,	0.21922,	0.22022,
0.22122,	0.22222,	0.22322,	0.22422,	0.22523,	0.22623,
0.22723,	0.22823,	0.22923,	0.23023,	0.23123,	0.23223,
0.23323,	0.23423,	0.23524,	0.23624,	0.23724,	0.23824,
0.23924,					
	0.24024,	0.24124,	0.24224,	0.24324,	0.24424,
0.24525,	0.24625,	0.24725,	0.24825,	0.24925,	0.25025,
0.25125,	0.25225,	0.25325,	0.25425,	0.25526,	0.25626,
0.25726,	0.25826,	0.25926,	0.26026,	0.26126,	0.26226,
0.26326,					
	0.26426,	0.26527,	0.26627,	0.26727,	0.26827,
0.26927,	0.27027,	0.27127,	0.27227,	0.27327,	0.27427,
0.27528,	0.27628,	0.27728,	0.27828,	0.27928,	0.28028,
0.28128,	0.28228,	0.28328,	0.28428,	0.28529,	0.28629,
0.28729,					
	0.28829,	0.28929,	0.29029,	0.29129,	0.29229,
0.29329,	0.29429,	0.2953,	0.2963,	0.2973,	0.2983,
0.2993,	0.3003,	0.3013,	0.3023,	0.3033,	0.3043,
0.30531,	0.30631,	0.30731,	0.30831,	0.30931,	0.31031,
0.31131,					
	0.31231,	0.31331,	0.31431,	0.31532,	0.31632,
0.31732,	0.31832,	0.31932,	0.32032,	0.32132,	0.32232,
0.32332,	0.32432,	0.32533,	0.32633,	0.32733,	0.32833,
0.32933,	0.33033,	0.33133,	0.33233,	0.33333,	0.33433,
0.33534,					
	0.33634,	0.33734,	0.33834,	0.33934,	0.34034,
0.34134,	0.34234,	0.34334,	0.34434,	0.34535,	0.34635,
0.34735,	0.34835,	0.34935,	0.35035,	0.35135,	0.35235,
0.35335,	0.35435,	0.35536,	0.35636,	0.35736,	0.35836,
0.35936,					
	0.36036,	0.36136,	0.36236,	0.36336,	0.36436,
0.36537,	0.36637,	0.36737,	0.36837,	0.36937,	0.37037,
0.37137,	0.37237,	0.37337,	0.37437,	0.37538,	0.37638,
0.37738,	0.37838,	0.37938,	0.38038,	0.38138,	0.38238,
0.38338,					
	0.38438,	0.38539,	0.38639,	0.38739,	0.38839,
0.38939,	0.39039,	0.39139,	0.39239,	0.39339,	0.39439,
0.3954,	0.3964,	0.3974,	0.3984,	0.3994,	0.4004,

0.4014,	0.4024,	0.4034,	0.4044,	0.40541,	0.40641,
0.40741,					
	0.40841,	0.40941,	0.41041,	0.41141,	0.41241,
0.41341,	0.41441,	0.41542,	0.41642,	0.41742,	0.41842,
0.41942,	0.42042,	0.42142,	0.42242,	0.42342,	0.42442,
0.42543,	0.42643,	0.42743,	0.42843,	0.42943,	0.43043,
0.43143,					
	0.43243,	0.43343,	0.43443,	0.43544,	0.43644,
0.43744,	0.43844,	0.43944,	0.44044,	0.44144,	0.44244,
0.44344,	0.44444,	0.44545,	0.44645,	0.44745,	0.44845,
0.44945,	0.45045,	0.45145,	0.45245,	0.45345,	0.45445,
0.45546,					
	0.45646,	0.45746,	0.45846,	0.45946,	0.46046,
0.46146,	0.46246,	0.46346,	0.46446,	0.46547,	0.46647,
0.46747,	0.46847,	0.46947,	0.47047,	0.47147,	0.47247,
0.47347,	0.47447,	0.47548,	0.47648,	0.47748,	0.47848,
0.47948,					
	0.48048,	0.48148,	0.48248,	0.48348,	0.48448,
0.48549,	0.48649,	0.48749,	0.48849,	0.48949,	0.49049,
0.49149,	0.49249,	0.49349,	0.49449,	0.4955,	0.4965,
0.4975,	0.4985,	0.4995,	0.5005,	0.5015,	0.5025,
0.5035,					
	0.5045,	0.50551,	0.50651,	0.50751,	0.50851,
0.50951,	0.51051,	0.51151,	0.51251,	0.51351,	0.51451,
0.51552,	0.51652,	0.51752,	0.51852,	0.51952,	0.52052,
0.52152,	0.52252,	0.52352,	0.52452,	0.52553,	0.52653,
0.52753,					
	0.52853,	0.52953,	0.53053,	0.53153,	0.53253,
0.53353,	0.53453,	0.53554,	0.53654,	0.53754,	0.53854,
0.53954,	0.54054,	0.54154,	0.54254,	0.54354,	0.54454,
0.54555,	0.54655,	0.54755,	0.54855,	0.54955,	0.55055,
0.55155,					
	0.55255,	0.55355,	0.55455,	0.55556,	0.55656,
0.55756,	0.55856,	0.55956,	0.56056,	0.56156,	0.56256,
0.56356,	0.56456,	0.56557,	0.56657,	0.56757,	0.56857,
0.56957,	0.57057,	0.57157,	0.57257,	0.57357,	0.57457,
0.57558,					
	0.57658,	0.57758,	0.57858,	0.57958,	0.58058,
0.58158,	0.58258,	0.58358,	0.58458,	0.58559,	0.58659,
0.58759,	0.58859,	0.58959,	0.59059,	0.59159,	0.59259,
0.59359,	0.59459,	0.5956,	0.5966,	0.5976,	0.5986,
0.5996,					
	0.6006,	0.6016,	0.6026,	0.6036,	0.6046,
0.60561,	0.60661,	0.60761,	0.60861,	0.60961,	0.61061,
0.61161,	0.61261,	0.61361,	0.61461,	0.61562,	0.61662,
0.61762,	0.61862,	0.61962,	0.62062,	0.62162,	0.62262,
0.62362,					
	0.62462,	0.62563,	0.62663,	0.62763,	0.62863,

0.62963,	0.63063,	0.63163,	0.63263,	0.63363,	0.63463,
0.63564,	0.63664,	0.63764,	0.63864,	0.63964,	0.64064,
0.64164,	0.64264,	0.64364,	0.64464,	0.64565,	0.64665,
0.64765,	0.64865,	0.64965,	0.65065,	0.65165,	0.65265,
0.65365,	0.65465,	0.65566,	0.65666,	0.65766,	0.65866,
0.65966,	0.66066,	0.66166,	0.66266,	0.66366,	0.66466,
0.66567,	0.66667,	0.66767,	0.66867,	0.66967,	0.67067,
0.67167,	0.67267,	0.67367,	0.67467,	0.67568,	0.67668,
0.67768,	0.67868,	0.67968,	0.68068,	0.68168,	0.68268,
0.68368,	0.68468,	0.68569,	0.68669,	0.68769,	0.68869,
0.68969,	0.69069,	0.69169,	0.69269,	0.69369,	0.69469,
0.6957,	0.6967,	0.6977,	0.6987,	0.6997,	0.7007,
0.7017,	0.7027,	0.7037,	0.7047,	0.70571,	0.70671,
0.70771,	0.70871,	0.70971,	0.71071,	0.71171,	0.71271,
0.71371,	0.71471,	0.71572,	0.71672,	0.71772,	0.71872,
0.71972,	0.72072,	0.72172,	0.72272,	0.72372,	0.72472,
0.72573,	0.72673,	0.72773,	0.72873,	0.72973,	0.73073,
0.73173,	0.73273,	0.73373,	0.73473,	0.73574,	0.73674,
0.73774,	0.73874,	0.73974,	0.74074,	0.74174,	0.74274,
0.74374,	0.74474,	0.74575,	0.74675,	0.74775,	0.74875,
0.74975,	0.75075,	0.75175,	0.75275,	0.75375,	0.75475,
0.75576,	0.75676,	0.75776,	0.75876,	0.75976,	0.76076,
0.76176,	0.76276,	0.76376,	0.76476,	0.76577,	0.76677,
0.76777,	0.76877,	0.76977,	0.77077,	0.77177,	0.77277,
0.77377,	0.77477,	0.77578,	0.77678,	0.77778,	0.77878,
0.77978,	0.78078,	0.78178,	0.78278,	0.78378,	0.78478,
0.78579,	0.78679,	0.78779,	0.78879,	0.78979,	0.79079,
0.79179,	0.79279,	0.79379,	0.79479,	0.7958,	0.7968,
0.7978,	0.7988,	0.7998,	0.8008,	0.8018,	0.8028,
0.8038,	0.8048,	0.80581,	0.80681,	0.80781,	0.80881,
0.80981,	0.81081,	0.81181,	0.81281,	0.81381,	0.81481,
0.81582,	0.81682,	0.81782,	0.81882,	0.81982,	0.82082,
0.82182,	0.82282,	0.82382,	0.82482,	0.82583,	0.82683,
0.82783,	0.82883,	0.82983,	0.83083,	0.83183,	0.83283,
0.83383,	0.83483,	0.83584,	0.83684,	0.83784,	0.83884,
0.83984,	0.84084,	0.84184,	0.84284,	0.84384,	0.84484,
0.84585,	0.84685,	0.84785,	0.84885,	0.84985,	0.85085,
0.85185,	0.85285,	0.85385,	0.85485,	0.85586,	0.85686,
0.85786,	0.85886,	0.85986,	0.86086,	0.86186,	0.86286,

0.86386,					
	0.86486,	0.86587,	0.86687,	0.86787,	0.86887,
0.86987,	0.87087,	0.87187,	0.87287,	0.87387,	0.87487,
0.87588,	0.87688,	0.87788,	0.87888,	0.87988,	0.88088,
0.88188,	0.88288,	0.88388,	0.88488,	0.88589,	0.88689,
0.88789,					
	0.88889,	0.88989,	0.89089,	0.89189,	0.89289,
0.89389,	0.89489,	0.8959,	0.8969,	0.8979,	0.8989,
0.8999,	0.9009,	0.9019,	0.9029,	0.9039,	0.9049,
0.90591,	0.90691,	0.90791,	0.90891,	0.90991,	0.91091,
0.91191,					
	0.91291,	0.91391,	0.91491,	0.91592,	0.91692,
0.91792,	0.91892,	0.91992,	0.92092,	0.92192,	0.92292,
0.92392,	0.92492,	0.92593,	0.92693,	0.92793,	0.92893,
0.92993,	0.93093,	0.93193,	0.93293,	0.93393,	0.93493,
0.93594,					
	0.93694,	0.93794,	0.93894,	0.93994,	0.94094,
0.94194,	0.94294,	0.94394,	0.94494,	0.94595,	0.94695,
0.94795,	0.94895,	0.94995,	0.95095,	0.95195,	0.95295,
0.95395,	0.95495,	0.95596,	0.95696,	0.95796,	0.95896,
0.95996,					
	0.96096,	0.96196,	0.96296,	0.96396,	0.96496,
0.96597,	0.96697,	0.96797,	0.96897,	0.96997,	0.97097,
0.97197,	0.97297,	0.97397,	0.97497,	0.97598,	0.97698,
0.97798,	0.97898,	0.97998,	0.98098,	0.98198,	0.98298,
0.98398,					
	0.98498,	0.98599,	0.98699,	0.98799,	0.98899,
0.98999,	0.99099,	0.99199,	0.99299,	0.99399,	0.99499,
0.996,	0.997,	0.998,	0.999,	1]], array([[	
0.1274,	0.12743,	0.2193, ...,	0,	0,	0],
[	0.095008,	0.095056,	0.15103, ...,	0,	0,
0],					
[	0.10314,	0.10317,	0.17177, ...,	0,	0,
0],					
...,					
[	0.078677,	0.078788,	0.13995, ...,	0,	0,
0],					
[	0.11627,	0.11629,	0.20408, ...,	0,	0,
0],					
[	0.11149,	0.11156,	0.18863, ...,	0,	0,
0]]), 'Confidence', 'F1'], [array([					
0.003003,	0.004004,	0.005005,	0.006006,	0.007007,	0.008008,
0.009009,	0.01001,	0.011011,	0.012012,	0.013013,	0.014014,
0.015015,	0.016016,	0.017017,	0.018018,	0.019019,	0.02002,
0.021021,	0.022022,	0.023023,			
	0.024024,	0.025025,	0.026026,	0.027027,	0.028028,
0.029029,	0.03003,	0.031031,	0.032032,	0.033033,	0.034034,
0.035035,	0.036036,	0.037037,	0.038038,	0.039039,	0.04004,

0.041041,	0.042042,	0.043043,	0.044044,	0.045045,	0.046046,
0.047047,	0.048048,	0.049049,	0.05005,	0.051051,	0.052052,
0.053053,	0.054054,	0.055055,	0.056056,	0.057057,	0.058058,
0.059059,	0.06006,	0.061061,	0.062062,	0.063063,	0.064064,
0.065065,	0.066066,	0.067067,	0.068068,	0.069069,	0.07007,
0.071071,	0.072072,	0.073073,	0.074074,	0.075075,	0.076076,
0.077077,	0.078078,	0.079079,	0.08008,	0.081081,	0.082082,
0.083083,	0.084084,	0.085085,	0.086086,	0.087087,	0.088088,
0.089089,	0.09009,	0.091091,	0.092092,	0.093093,	0.094094,
0.095095,	0.096096,	0.097097,	0.098098,	0.099099,	0.1001,
0.1011,	0.1021,	0.1031,	0.1041,	0.10511,	0.10611,
0.10711,	0.10811,	0.10911,	0.11011,	0.11111,	0.11211,
0.11311,	0.11411,	0.11512,	0.11612,	0.11712,	0.11812,
0.11912,	0.12012,	0.12112,	0.12212,	0.12312,	0.12412,
0.12513,	0.12613,	0.12713,	0.12813,	0.12913,	0.13013,
0.13113,	0.13213,	0.13313,	0.13413,	0.13514,	0.13614,
0.13714,	0.13814,	0.13914,	0.14014,	0.14114,	0.14214,
0.14314,	0.14414,	0.14515,	0.14615,	0.14715,	0.14815,
0.14915,	0.15015,	0.15115,	0.15215,	0.15315,	0.15415,
0.15516,	0.15616,	0.15716,	0.15816,	0.15916,	0.16016,
0.16116,	0.16216,	0.16316,	0.16416,	0.16517,	0.16617,
0.16717,	0.16817,	0.16917,	0.17017,	0.17117,	0.17217,
0.17317,	0.17417,	0.17518,	0.17618,	0.17718,	0.17818,
0.17918,	0.18018,	0.18118,	0.18218,	0.18318,	0.18418,
0.18519,	0.18619,	0.18719,	0.18819,	0.18919,	0.19019,
0.19119,	0.19219,	0.19319,	0.19419,	0.1952,	0.1962,
0.1972,	0.1982,	0.1992,	0.2002,	0.2012,	0.2022,
0.2032,	0.2042,	0.20521,	0.20621,	0.20721,	0.20821,
0.20921,	0.21021,	0.21121,	0.21221,	0.21321,	0.21421,
0.21522,	0.21622,	0.21722,	0.21822,	0.21922,	0.22022,
0.22122,	0.22222,	0.22322,	0.22422,	0.22523,	0.22623,
0.22723,	0.22823,	0.22923,	0.23023,	0.23123,	0.23223,
0.23323,	0.23423,	0.23524,	0.23624,	0.23724,	0.23824,
0.23924,	0.24024,	0.24124,	0.24224,	0.24324,	0.24424,
0.24525,	0.24625,	0.24725,	0.24825,	0.24925,	0.25025,
0.25125,	0.25225,	0.25325,	0.25425,	0.25526,	0.25626,
0.25726,	0.25826,	0.25926,	0.26026,	0.26126,	0.26226,
0.26326,	0.26426,	0.26527,	0.26627,	0.26727,	0.26827,

0.26927,	0.27027,	0.27127,	0.27227,	0.27327,	0.27427,
0.27528,	0.27628,	0.27728,	0.27828,	0.27928,	0.28028,
0.28128,	0.28228,	0.28328,	0.28428,	0.28529,	0.28629,
0.28729,	0.28829,	0.28929,	0.29029,	0.29129,	0.29229,
0.29329,	0.29429,	0.2953,	0.2963,	0.2973,	0.2983,
0.2993,	0.3003,	0.3013,	0.3023,	0.3033,	0.3043,
0.30531,	0.30631,	0.30731,	0.30831,	0.30931,	0.31031,
0.31131,	0.31231,	0.31331,	0.31431,	0.31532,	0.31632,
0.31732,	0.31832,	0.31932,	0.32032,	0.32132,	0.32232,
0.32332,	0.32432,	0.32533,	0.32633,	0.32733,	0.32833,
0.32933,	0.33033,	0.33133,	0.33233,	0.33333,	0.33433,
0.33534,	0.33634,	0.33734,	0.33834,	0.33934,	0.34034,
0.34134,	0.34234,	0.34334,	0.34434,	0.34535,	0.34635,
0.34735,	0.34835,	0.34935,	0.35035,	0.35135,	0.35235,
0.35335,	0.35435,	0.35536,	0.35636,	0.35736,	0.35836,
0.35936,	0.36036,	0.36136,	0.36236,	0.36336,	0.36436,
0.36537,	0.36637,	0.36737,	0.36837,	0.36937,	0.37037,
0.37137,	0.37237,	0.37337,	0.37437,	0.37538,	0.37638,
0.37738,	0.37838,	0.37938,	0.38038,	0.38138,	0.38238,
0.38338,	0.38438,	0.38539,	0.38639,	0.38739,	0.38839,
0.38939,	0.39039,	0.39139,	0.39239,	0.39339,	0.39439,
0.3954,	0.3964,	0.3974,	0.3984,	0.3994,	0.4004,
0.4014,	0.4024,	0.4034,	0.4044,	0.40541,	0.40641,
0.40741,	0.40841,	0.40941,	0.41041,	0.41141,	0.41241,
0.41341,	0.41441,	0.41542,	0.41642,	0.41742,	0.41842,
0.41942,	0.42042,	0.42142,	0.42242,	0.42342,	0.42442,
0.42543,	0.42643,	0.42743,	0.42843,	0.42943,	0.43043,
0.43143,	0.43243,	0.43343,	0.43443,	0.43544,	0.43644,
0.43744,	0.43844,	0.43944,	0.44044,	0.44144,	0.44244,
0.44344,	0.44444,	0.44545,	0.44645,	0.44745,	0.44845,
0.44945,	0.45045,	0.45145,	0.45245,	0.45345,	0.45445,
0.45546,	0.45646,	0.45746,	0.45846,	0.45946,	0.46046,
0.46146,	0.46246,	0.46346,	0.46446,	0.46547,	0.46647,
0.46747,	0.46847,	0.46947,	0.47047,	0.47147,	0.47247,
0.47347,	0.47447,	0.47548,	0.47648,	0.47748,	0.47848,
0.47948,	0.48048,	0.48148,	0.48248,	0.48348,	0.48448,
0.48549,	0.48649,	0.48749,	0.48849,	0.48949,	0.49049,
0.49149,	0.49249,	0.49349,	0.49449,	0.4955,	0.4965,
0.4975,	0.4985,	0.4995,	0.5005,	0.5015,	0.5025,

0.5035,					
	0.5045,	0.50551,	0.50651,	0.50751,	0.50851,
0.50951,	0.51051,	0.51151,	0.51251,	0.51351,	0.51451,
0.51552,	0.51652,	0.51752,	0.51852,	0.51952,	0.52052,
0.52152,	0.52252,	0.52352,	0.52452,	0.52553,	0.52653,
0.52753,					
	0.52853,	0.52953,	0.53053,	0.53153,	0.53253,
0.53353,	0.53453,	0.53554,	0.53654,	0.53754,	0.53854,
0.53954,	0.54054,	0.54154,	0.54254,	0.54354,	0.54454,
0.54555,	0.54655,	0.54755,	0.54855,	0.54955,	0.55055,
0.55155,					
	0.55255,	0.55355,	0.55455,	0.55556,	0.55656,
0.55756,	0.55856,	0.55956,	0.56056,	0.56156,	0.56256,
0.56356,	0.56456,	0.56557,	0.56657,	0.56757,	0.56857,
0.56957,	0.57057,	0.57157,	0.57257,	0.57357,	0.57457,
0.57558,					
	0.57658,	0.57758,	0.57858,	0.57958,	0.58058,
0.58158,	0.58258,	0.58358,	0.58458,	0.58559,	0.58659,
0.58759,	0.58859,	0.58959,	0.59059,	0.59159,	0.59259,
0.59359,	0.59459,	0.5956,	0.5966,	0.5976,	0.5986,
0.5996,					
	0.6006,	0.6016,	0.6026,	0.6036,	0.6046,
0.60561,	0.60661,	0.60761,	0.60861,	0.60961,	0.61061,
0.61161,	0.61261,	0.61361,	0.61461,	0.61562,	0.61662,
0.61762,	0.61862,	0.61962,	0.62062,	0.62162,	0.62262,
0.62362,					
	0.62462,	0.62563,	0.62663,	0.62763,	0.62863,
0.62963,	0.63063,	0.63163,	0.63263,	0.63363,	0.63463,
0.63564,	0.63664,	0.63764,	0.63864,	0.63964,	0.64064,
0.64164,	0.64264,	0.64364,	0.64464,	0.64565,	0.64665,
0.64765,					
	0.64865,	0.64965,	0.65065,	0.65165,	0.65265,
0.65365,	0.65465,	0.65566,	0.65666,	0.65766,	0.65866,
0.65966,	0.66066,	0.66166,	0.66266,	0.66366,	0.66466,
0.66567,	0.66667,	0.66767,	0.66867,	0.66967,	0.67067,
0.67167,					
	0.67267,	0.67367,	0.67467,	0.67568,	0.67668,
0.67768,	0.67868,	0.67968,	0.68068,	0.68168,	0.68268,
0.68368,	0.68468,	0.68569,	0.68669,	0.68769,	0.68869,
0.68969,	0.69069,	0.69169,	0.69269,	0.69369,	0.69469,
0.6957,					
	0.6967,	0.6977,	0.6987,	0.6997,	0.7007,
0.7017,	0.7027,	0.7037,	0.7047,	0.70571,	0.70671,
0.70771,	0.70871,	0.70971,	0.71071,	0.71171,	0.71271,
0.71371,	0.71471,	0.71572,	0.71672,	0.71772,	0.71872,
0.71972,					
	0.72072,	0.72172,	0.72272,	0.72372,	0.72472,
0.72573,	0.72673,	0.72773,	0.72873,	0.72973,	0.73073,

0.73173,	0.73273,	0.73373,	0.73473,	0.73574,	0.73674,
0.73774,	0.73874,	0.73974,	0.74074,	0.74174,	0.74274,
0.74374,					
	0.74474,	0.74575,	0.74675,	0.74775,	0.74875,
0.74975,	0.75075,	0.75175,	0.75275,	0.75375,	0.75475,
0.75576,	0.75676,	0.75776,	0.75876,	0.75976,	0.76076,
0.76176,	0.76276,	0.76376,	0.76476,	0.76577,	0.76677,
0.76777,					
	0.76877,	0.76977,	0.77077,	0.77177,	0.77277,
0.77377,	0.77477,	0.77578,	0.77678,	0.77778,	0.77878,
0.77978,	0.78078,	0.78178,	0.78278,	0.78378,	0.78478,
0.78579,	0.78679,	0.78779,	0.78879,	0.78979,	0.79079,
0.79179,					
	0.79279,	0.79379,	0.79479,	0.7958,	0.7968,
0.7978,	0.7988,	0.7998,	0.8008,	0.8018,	0.8028,
0.8038,	0.8048,	0.80581,	0.80681,	0.80781,	0.80881,
0.80981,	0.81081,	0.81181,	0.81281,	0.81381,	0.81481,
0.81582,					
	0.81682,	0.81782,	0.81882,	0.81982,	0.82082,
0.82182,	0.82282,	0.82382,	0.82482,	0.82583,	0.82683,
0.82783,	0.82883,	0.82983,	0.83083,	0.83183,	0.83283,
0.83383,	0.83483,	0.83584,	0.83684,	0.83784,	0.83884,
0.83984,					
	0.84084,	0.84184,	0.84284,	0.84384,	0.84484,
0.84585,	0.84685,	0.84785,	0.84885,	0.84985,	0.85085,
0.85185,	0.85285,	0.85385,	0.85485,	0.85586,	0.85686,
0.85786,	0.85886,	0.85986,	0.86086,	0.86186,	0.86286,
0.86386,					
	0.86486,	0.86587,	0.86687,	0.86787,	0.86887,
0.86987,	0.87087,	0.87187,	0.87287,	0.87387,	0.87487,
0.87588,	0.87688,	0.87788,	0.87888,	0.87988,	0.88088,
0.88188,	0.88288,	0.88388,	0.88488,	0.88589,	0.88689,
0.88789,					
	0.88889,	0.88989,	0.89089,	0.89189,	0.89289,
0.89389,	0.89489,	0.8959,	0.8969,	0.8979,	0.8989,
0.8999,	0.9009,	0.9019,	0.9029,	0.9039,	0.9049,
0.90591,	0.90691,	0.90791,	0.90891,	0.90991,	0.91091,
0.91191,					
	0.91291,	0.91391,	0.91491,	0.91592,	0.91692,
0.91792,	0.91892,	0.91992,	0.92092,	0.92192,	0.92292,
0.92392,	0.92492,	0.92593,	0.92693,	0.92793,	0.92893,
0.92993,	0.93093,	0.93193,	0.93293,	0.93393,	0.93493,
0.93594,					
	0.93694,	0.93794,	0.93894,	0.93994,	0.94094,
0.94194,	0.94294,	0.94394,	0.94494,	0.94595,	0.94695,
0.94795,	0.94895,	0.94995,	0.95095,	0.95195,	0.95295,
0.95395,	0.95495,	0.95596,	0.95696,	0.95796,	0.95896,
0.95996,					

	0.96096,	0.96196,	0.96296,	0.96396,	0.96496,	
0.96597,	0.96697,	0.96797,	0.96897,	0.96997,	0.97097,	
0.97197,	0.97297,	0.97397,	0.97497,	0.97598,	0.97698,	
0.97798,	0.97898,	0.97998,	0.98098,	0.98198,	0.98298,	
0.98398,						
	0.98498,	0.98599,	0.98699,	0.98799,	0.98899,	
0.98999,	0.99099,	0.99199,	0.99299,	0.99399,	0.99499,	
0.996,	0.997,	0.998,	0.999,	1]], array([[		
0.0682,	0.068217,	0.12379, ...,	1,	1,	1],	
[	0.050114,	0.050141,	0.082448, ...,	1,	1,	
1],						
[	0.054917,	0.054932,	0.09598, ...,	1,	1,	
1],						
...,						
[	0.041053,	0.041113,	0.075717, ...,	1,	1,	
1],						
[	0.061988,	0.062002,	0.11472, ...,	1,	1,	
1],						
[	0.059316,	0.059353,	0.10539, ...,	1,	1,	
1]]), 'Confidence', 'Precision'], [array([				0,	0.001001,	0.002002,
0.003003,	0.004004,	0.005005,	0.006006,	0.007007,	0.008008,	
0.009009,	0.01001,	0.011011,	0.012012,	0.013013,	0.014014,	
0.015015,	0.016016,	0.017017,	0.018018,	0.019019,	0.02002,	
0.021021,	0.022022,	0.023023,				
	0.024024,	0.025025,	0.026026,	0.027027,	0.028028,	
0.029029,	0.03003,	0.031031,	0.032032,	0.033033,	0.034034,	
0.035035,	0.036036,	0.037037,	0.038038,	0.039039,	0.04004,	
0.041041,	0.042042,	0.043043,	0.044044,	0.045045,	0.046046,	
0.047047,						
	0.048048,	0.049049,	0.05005,	0.051051,	0.052052,	
0.053053,	0.054054,	0.055055,	0.056056,	0.057057,	0.058058,	
0.059059,	0.06006,	0.061061,	0.062062,	0.063063,	0.064064,	
0.065065,	0.066066,	0.067067,	0.068068,	0.069069,	0.07007,	
0.071071,						
	0.072072,	0.073073,	0.074074,	0.075075,	0.076076,	
0.077077,	0.078078,	0.079079,	0.08008,	0.081081,	0.082082,	
0.083083,	0.084084,	0.085085,	0.086086,	0.087087,	0.088088,	
0.089089,	0.09009,	0.091091,	0.092092,	0.093093,	0.094094,	
0.095095,						
	0.096096,	0.097097,	0.098098,	0.099099,	0.1001,	
0.1011,	0.1021,	0.1031,	0.1041,	0.10511,	0.10611,	
0.10711,	0.10811,	0.10911,	0.11011,	0.11111,	0.11211,	
0.11311,	0.11411,	0.11512,	0.11612,	0.11712,	0.11812,	
0.11912,						
	0.12012,	0.12112,	0.12212,	0.12312,	0.12412,	
0.12513,	0.12613,	0.12713,	0.12813,	0.12913,	0.13013,	
0.13113,	0.13213,	0.13313,	0.13413,	0.13514,	0.13614,	
0.13714,	0.13814,	0.13914,	0.14014,	0.14114,	0.14214,	

0.14314,					
	0.14414,	0.14515,	0.14615,	0.14715,	0.14815,
0.14915,	0.15015,	0.15115,	0.15215,	0.15315,	0.15415,
0.15516,	0.15616,	0.15716,	0.15816,	0.15916,	0.16016,
0.16116,	0.16216,	0.16316,	0.16416,	0.16517,	0.16617,
0.16717,					
	0.16817,	0.16917,	0.17017,	0.17117,	0.17217,
0.17317,	0.17417,	0.17518,	0.17618,	0.17718,	0.17818,
0.17918,	0.18018,	0.18118,	0.18218,	0.18318,	0.18418,
0.18519,	0.18619,	0.18719,	0.18819,	0.18919,	0.19019,
0.19119,					
	0.19219,	0.19319,	0.19419,	0.1952,	0.1962,
0.1972,	0.1982,	0.1992,	0.2002,	0.2012,	0.2022,
0.2032,	0.2042,	0.20521,	0.20621,	0.20721,	0.20821,
0.20921,	0.21021,	0.21121,	0.21221,	0.21321,	0.21421,
0.21522,					
	0.21622,	0.21722,	0.21822,	0.21922,	0.22022,
0.22122,	0.22222,	0.22322,	0.22422,	0.22523,	0.22623,
0.22723,	0.22823,	0.22923,	0.23023,	0.23123,	0.23223,
0.23323,	0.23423,	0.23524,	0.23624,	0.23724,	0.23824,
0.23924,					
	0.24024,	0.24124,	0.24224,	0.24324,	0.24424,
0.24525,	0.24625,	0.24725,	0.24825,	0.24925,	0.25025,
0.25125,	0.25225,	0.25325,	0.25425,	0.25526,	0.25626,
0.25726,	0.25826,	0.25926,	0.26026,	0.26126,	0.26226,
0.26326,					
	0.26426,	0.26527,	0.26627,	0.26727,	0.26827,
0.26927,	0.27027,	0.27127,	0.27227,	0.27327,	0.27427,
0.27528,	0.27628,	0.27728,	0.27828,	0.27928,	0.28028,
0.28128,	0.28228,	0.28328,	0.28428,	0.28529,	0.28629,
0.28729,					
	0.28829,	0.28929,	0.29029,	0.29129,	0.29229,
0.29329,	0.29429,	0.2953,	0.2963,	0.2973,	0.2983,
0.2993,	0.3003,	0.3013,	0.3023,	0.3033,	0.3043,
0.30531,	0.30631,	0.30731,	0.30831,	0.30931,	0.31031,
0.31131,					
	0.31231,	0.31331,	0.31431,	0.31532,	0.31632,
0.31732,	0.31832,	0.31932,	0.32032,	0.32132,	0.32232,
0.32332,	0.32432,	0.32533,	0.32633,	0.32733,	0.32833,
0.32933,	0.33033,	0.33133,	0.33233,	0.33333,	0.33433,
0.33534,					
	0.33634,	0.33734,	0.33834,	0.33934,	0.34034,
0.34134,	0.34234,	0.34334,	0.34434,	0.34535,	0.34635,
0.34735,	0.34835,	0.34935,	0.35035,	0.35135,	0.35235,
0.35335,	0.35435,	0.35536,	0.35636,	0.35736,	0.35836,
0.35936,					
	0.36036,	0.36136,	0.36236,	0.36336,	0.36436,
0.36537,	0.36637,	0.36737,	0.36837,	0.36937,	0.37037,

0.37137,	0.37237,	0.37337,	0.37437,	0.37538,	0.37638,
0.37738,	0.37838,	0.37938,	0.38038,	0.38138,	0.38238,
0.38338,					
	0.38438,	0.38539,	0.38639,	0.38739,	0.38839,
0.38939,	0.39039,	0.39139,	0.39239,	0.39339,	0.39439,
0.3954,	0.3964,	0.3974,	0.3984,	0.3994,	0.4004,
0.4014,	0.4024,	0.4034,	0.4044,	0.40541,	0.40641,
0.40741,					
	0.40841,	0.40941,	0.41041,	0.41141,	0.41241,
0.41341,	0.41441,	0.41542,	0.41642,	0.41742,	0.41842,
0.41942,	0.42042,	0.42142,	0.42242,	0.42342,	0.42442,
0.42543,	0.42643,	0.42743,	0.42843,	0.42943,	0.43043,
0.43143,					
	0.43243,	0.43343,	0.43443,	0.43544,	0.43644,
0.43744,	0.43844,	0.43944,	0.44044,	0.44144,	0.44244,
0.44344,	0.44444,	0.44545,	0.44645,	0.44745,	0.44845,
0.44945,	0.45045,	0.45145,	0.45245,	0.45345,	0.45445,
0.45546,					
	0.45646,	0.45746,	0.45846,	0.45946,	0.46046,
0.46146,	0.46246,	0.46346,	0.46446,	0.46547,	0.46647,
0.46747,	0.46847,	0.46947,	0.47047,	0.47147,	0.47247,
0.47347,	0.47447,	0.47548,	0.47648,	0.47748,	0.47848,
0.47948,					
	0.48048,	0.48148,	0.48248,	0.48348,	0.48448,
0.48549,	0.48649,	0.48749,	0.48849,	0.48949,	0.49049,
0.49149,	0.49249,	0.49349,	0.49449,	0.4955,	0.4965,
0.4975,	0.4985,	0.4995,	0.5005,	0.5015,	0.5025,
0.5035,					
	0.5045,	0.50551,	0.50651,	0.50751,	0.50851,
0.50951,	0.51051,	0.51151,	0.51251,	0.51351,	0.51451,
0.51552,	0.51652,	0.51752,	0.51852,	0.51952,	0.52052,
0.52152,	0.52252,	0.52352,	0.52452,	0.52553,	0.52653,
0.52753,					
	0.52853,	0.52953,	0.53053,	0.53153,	0.53253,
0.53353,	0.53453,	0.53554,	0.53654,	0.53754,	0.53854,
0.53954,	0.54054,	0.54154,	0.54254,	0.54354,	0.54454,
0.54555,	0.54655,	0.54755,	0.54855,	0.54955,	0.55055,
0.55155,					
	0.55255,	0.55355,	0.55455,	0.55556,	0.55656,
0.55756,	0.55856,	0.55956,	0.56056,	0.56156,	0.56256,
0.56356,	0.56456,	0.56557,	0.56657,	0.56757,	0.56857,
0.56957,	0.57057,	0.57157,	0.57257,	0.57357,	0.57457,
0.57558,					
	0.57658,	0.57758,	0.57858,	0.57958,	0.58058,
0.58158,	0.58258,	0.58358,	0.58458,	0.58559,	0.58659,
0.58759,	0.58859,	0.58959,	0.59059,	0.59159,	0.59259,
0.59359,	0.59459,	0.5956,	0.5966,	0.5976,	0.5986,
0.5996,					

	0.6006,	0.6016,	0.6026,	0.6036,	0.6046,
0.60561,	0.60661,	0.60761,	0.60861,	0.60961,	0.61061,
0.61161,	0.61261,	0.61361,	0.61461,	0.61562,	0.61662,
0.61762,	0.61862,	0.61962,	0.62062,	0.62162,	0.62262,
0.62362,					
	0.62462,	0.62563,	0.62663,	0.62763,	0.62863,
0.62963,	0.63063,	0.63163,	0.63263,	0.63363,	0.63463,
0.63564,	0.63664,	0.63764,	0.63864,	0.63964,	0.64064,
0.64164,	0.64264,	0.64364,	0.64464,	0.64565,	0.64665,
0.64765,					
	0.64865,	0.64965,	0.65065,	0.65165,	0.65265,
0.65365,	0.65465,	0.65566,	0.65666,	0.65766,	0.65866,
0.65966,	0.66066,	0.66166,	0.66266,	0.66366,	0.66466,
0.66567,	0.66667,	0.66767,	0.66867,	0.66967,	0.67067,
0.67167,					
	0.67267,	0.67367,	0.67467,	0.67568,	0.67668,
0.67768,	0.67868,	0.67968,	0.68068,	0.68168,	0.68268,
0.68368,	0.68468,	0.68569,	0.68669,	0.68769,	0.68869,
0.68969,	0.69069,	0.69169,	0.69269,	0.69369,	0.69469,
0.6957,					
	0.6967,	0.6977,	0.6987,	0.6997,	0.7007,
0.7017,	0.7027,	0.7037,	0.7047,	0.70571,	0.70671,
0.70771,	0.70871,	0.70971,	0.71071,	0.71171,	0.71271,
0.71371,	0.71471,	0.71572,	0.71672,	0.71772,	0.71872,
0.71972,					
	0.72072,	0.72172,	0.72272,	0.72372,	0.72472,
0.72573,	0.72673,	0.72773,	0.72873,	0.72973,	0.73073,
0.73173,	0.73273,	0.73373,	0.73473,	0.73574,	0.73674,
0.73774,	0.73874,	0.73974,	0.74074,	0.74174,	0.74274,
0.74374,					
	0.74474,	0.74575,	0.74675,	0.74775,	0.74875,
0.74975,	0.75075,	0.75175,	0.75275,	0.75375,	0.75475,
0.75576,	0.75676,	0.75776,	0.75876,	0.75976,	0.76076,
0.76176,	0.76276,	0.76376,	0.76476,	0.76577,	0.76677,
0.76777,					
	0.76877,	0.76977,	0.77077,	0.77177,	0.77277,
0.77377,	0.77477,	0.77578,	0.77678,	0.77778,	0.77878,
0.77978,	0.78078,	0.78178,	0.78278,	0.78378,	0.78478,
0.78579,	0.78679,	0.78779,	0.78879,	0.78979,	0.79079,
0.79179,					
	0.79279,	0.79379,	0.79479,	0.7958,	0.7968,
0.7978,	0.7988,	0.7998,	0.8008,	0.8018,	0.8028,
0.8038,	0.8048,	0.80581,	0.80681,	0.80781,	0.80881,
0.80981,	0.81081,	0.81181,	0.81281,	0.81381,	0.81481,
0.81582,					
	0.81682,	0.81782,	0.81882,	0.81982,	0.82082,
0.82182,	0.82282,	0.82382,	0.82482,	0.82583,	0.82683,
0.82783,	0.82883,	0.82983,	0.83083,	0.83183,	0.83283,

```

0.83383,    0.83483,    0.83584,    0.83684,    0.83784,    0.83884,
0.83984,
    0.84084,    0.84184,    0.84284,    0.84384,    0.84484,
0.84585,    0.84685,    0.84785,    0.84885,    0.84985,    0.85085,
0.85185,    0.85285,    0.85385,    0.85485,    0.85586,    0.85686,
0.85786,    0.85886,    0.85986,    0.86086,    0.86186,    0.86286,
0.86386,
    0.86486,    0.86587,    0.86687,    0.86787,    0.86887,
0.86987,    0.87087,    0.87187,    0.87287,    0.87387,    0.87487,
0.87588,    0.87688,    0.87788,    0.87888,    0.87988,    0.88088,
0.88188,    0.88288,    0.88388,    0.88488,    0.88589,    0.88689,
0.88789,
    0.88889,    0.88989,    0.89089,    0.89189,    0.89289,
0.89389,    0.89489,    0.8959,    0.8969,    0.8979,    0.8989,
0.8999,    0.9009,    0.9019,    0.9029,    0.9039,    0.9049,
0.90591,    0.90691,    0.90791,    0.90891,    0.90991,    0.91091,
0.91191,
    0.91291,    0.91391,    0.91491,    0.91592,    0.91692,
0.91792,    0.91892,    0.91992,    0.92092,    0.92192,    0.92292,
0.92392,    0.92492,    0.92593,    0.92693,    0.92793,    0.92893,
0.92993,    0.93093,    0.93193,    0.93293,    0.93393,    0.93493,
0.93594,
    0.93694,    0.93794,    0.93894,    0.93994,    0.94094,
0.94194,    0.94294,    0.94394,    0.94494,    0.94595,    0.94695,
0.94795,    0.94895,    0.94995,    0.95095,    0.95195,    0.95295,
0.95395,    0.95495,    0.95596,    0.95696,    0.95796,    0.95896,
0.95996,
    0.96096,    0.96196,    0.96296,    0.96396,    0.96496,
0.96597,    0.96697,    0.96797,    0.96897,    0.96997,    0.97097,
0.97197,    0.97297,    0.97397,    0.97497,    0.97598,    0.97698,
0.97798,    0.97898,    0.97998,    0.98098,    0.98198,    0.98298,
0.98398,
    0.98498,    0.98599,    0.98699,    0.98799,    0.98899,
0.98999,    0.99099,    0.99199,    0.99299,    0.99399,    0.99499,
0.996,    0.997,    0.998,    0.999,    1]), array([[
0.96571,    0.96571,    0.96, ...,    0,    0,    0],
[    0.91204,    0.91204,    0.89815, ...,    0,    0,
0],
[    0.8459,    0.8459,    0.81639, ...,    0,    0,
0],
...,
[    0.94203,    0.94203,    0.92271, ...,    0,    0,
0],
[    0.93529,    0.93529,    0.92353, ...,    0,    0,
0],
[    0.92614,    0.92614,    0.89773, ...,    0,    0,
0]]), 'Confidence', 'Recall'])
fitness: np.float64(0.5078632857938359)

```



```

keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)',
'metrics/mAP50-95(B)']
maps: array([[ 0.60927,  0.54796,  0.38744,  0.3553,  0.35498,
 0.61231,  0.59352,  0.64899,  0.3469,  0.5525,  0.44091,
 0.59596,  0.6196,  0.56555,  0.53174,  0.25698,  0.45903,
 0.48439,  0.64278,  0.55118])
names: {0: 'aeroplane', 1: 'bicycle', 2: 'bird', 3: 'boat', 4: 'bottle', 5:
'bus', 6: 'car', 7: 'cat', 8: 'chair', 9: 'cow', 10: 'diningtable', 11: 'dog',
12: 'horse', 13: 'motorbike', 14: 'person', 15: 'pottedplant', 16: 'sheep', 17:
'sofa', 18: 'train', 19: 'tvmonitor'}
nt_per_class: array([ 175,  216,  305,  190,  296,  141,  818,  198,  706,  171,
 162,  267,  199,  197, 2742,  320,  162,  207,  170,  176])
nt_per_image: array([ 127,  133,  151,  101,  109,  97,  359,  178,  290,  75,
 133,  220,  150,  126, 1025,  120,  48,  184,  135,  135])
results_dict: {'metrics/precision(B)': 0.7681034814185664, 'metrics/recall(B)':
0.6439342975760063, 'metrics/mAP50(B)': 0.7241941548679938,
'metrics/mAP50-95(B)': 0.5078632857938359, 'fitness': 0.5078632857938359}
save_dir: PosixPath('/content/runs/detect/val')
speed: {'preprocess': 1.199725855779713, 'inference': 3.8720909920301207,
'loss': 0.0007625824724687208, 'postprocess': 1.8309408856597533}
stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [], 'target_img':
[]}]
task: 'detect'

```

```

[50]: import torch
import matplotlib.pyplot as plt
from ultralytics import YOLO
import cv2
import numpy as np
import os

def test_ultra_yolo(model, img_paths, device="cuda", conf=0.25, num_images=6,
    cols=3):
    """
    Test YOLO (Ultralytics) model on multiple images and plot predictions.

    Args:
        model: YOLO model (from ultralytics import YOLO)
        img_paths: list of image paths or a directory
        device: 'cuda', 'cpu', or int (e.g. 0)
        conf: confidence threshold (float)
        num_images: number of images to display
        cols: number of columns in the subplot grid
    """
    # Resolve device automatically
    if isinstance(device, str):
        if device == "cuda" and not torch.cuda.is_available():

```

```

        print(" CUDA not available, switching to CPU.")
        device = "cpu"

    # handle single path / folder / list
    if isinstance(img_paths, str) and os.path.isdir(img_paths):
        all_imgs = [os.path.join(img_paths, f) for f in os.listdir(img_paths)
                     if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    elif isinstance(img_paths, (list, tuple)):
        all_imgs = img_paths
    else:
        all_imgs = [img_paths]

    imgs_to_show = all_imgs[:num_images]

    # run inference
    results = model.predict(source=imgs_to_show, conf=conf, device=device,
↪ verbose=False)

    # plotting grid
    rows = int(np.ceil(len(imgs_to_show) / cols))
    plt.figure(figsize=(5*cols, 5*rows))

    for i, r in enumerate(results):
        annotated = r.plot() # YOLO built-in visualization (with labels)
        annotated = cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)

        ax = plt.subplot(rows, cols, i + 1)
        ax.imshow(annotated)
        ax.axis("off")

        # summary text
        if len(r.bboxes) > 0:
            labels_conf = [
                f"{r.names[int(c)]} ({float(s):.2f})"
                for c, s in zip(r.bboxes.cls, r.bboxes.conf)
            ]
            summary = "\n".join(labels_conf[:5])
            ax.set_title(summary, fontsize=9, color="green")
        else:
            ax.set_title("No detection", color="gray", fontsize=9)

    plt.tight_layout()
    plt.show()

    return results

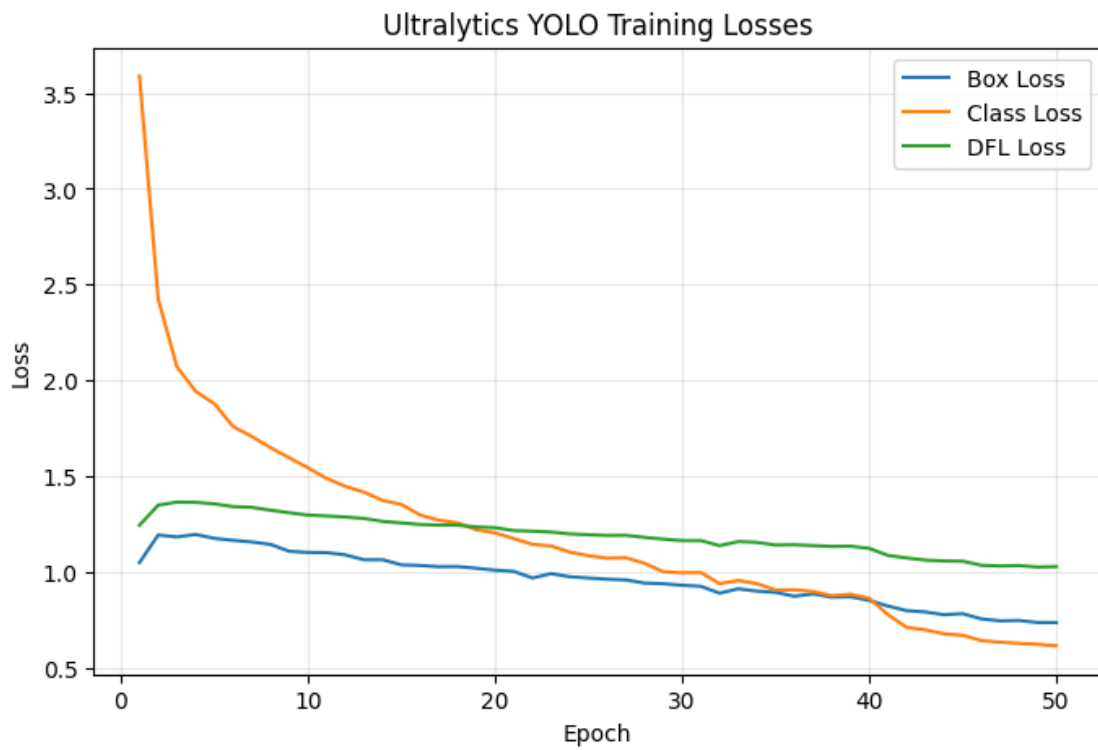
```

```
[51]: import pandas as pd

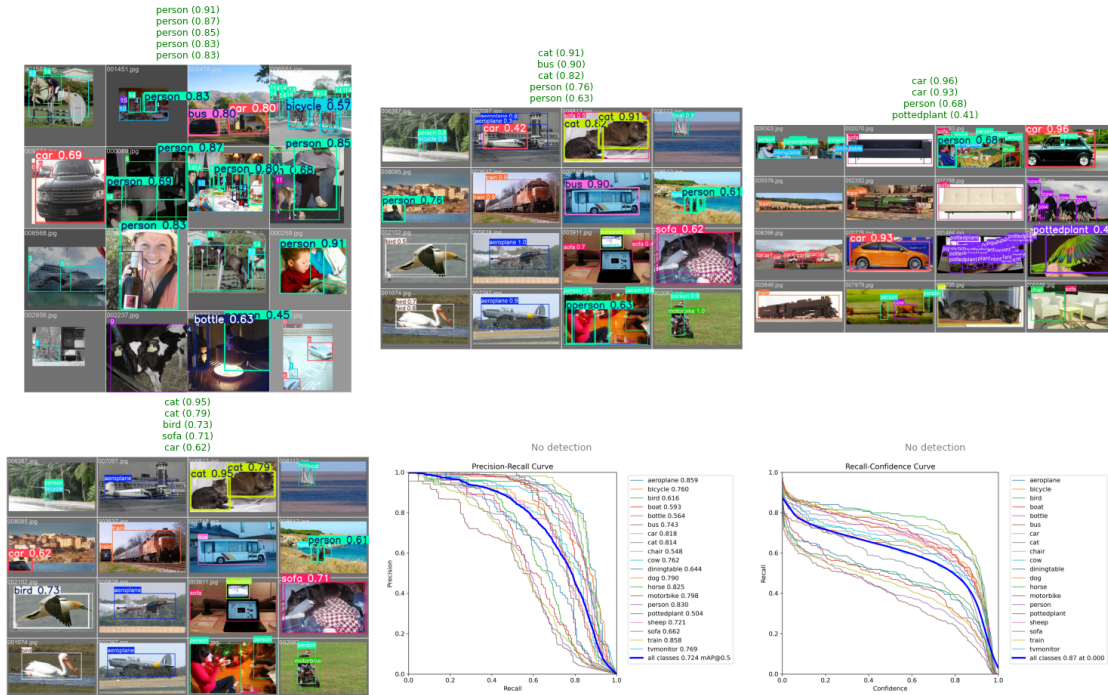
def plot_ultra_yolo_loss(results_dir="runs/detect/train_run"):
    """
    Plot Ultralytics YOLO training loss curves from results.csv
    """
    csv_path = os.path.join(results_dir, "results.csv")
    if not os.path.exists(csv_path):
        print(" results.csv not found - check your YOLO training run folder.")
        return

    df = pd.read_csv(csv_path)
    plt.figure(figsize=(8,5))
    plt.plot(df["epoch"], df["train/box_loss"], label="Box Loss")
    plt.plot(df["epoch"], df["train/cls_loss"], label="Class Loss")
    plt.plot(df["epoch"], df["train/dfl_loss"], label="DFL Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title("Ultralytics YOLO Training Losses")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()
```

```
[52]: plot_ultra_yolo_loss("/content/runs/detect/train")
```



```
[53]: results = test_ultra_yolo(model, "/content/runs/detect/train", device="cuda",
    ↪conf=0.4, num_images=6)
```



```
[54]: # results = test_ultra_yolo(model, "data/test.jpg", device="cuda", conf=0.4)
```

0.2 Exercise 3 Ultratytic with Body part dataset:

```
[55]: raw_zip_dir = "DLCV_Lab_Dataset/raw_zips/body-parts-dataset.zip"
source_url = "https://www.kaggle.com/api/v1/datasets/download/linkanjarad/
    ↪body-parts-dataset"
download_big_file(raw_zip_dir, source_url)
```

body-parts-dataset.zip: 100% | 6.56M/6.56M [00:00<00:00, 39.0MB/s]

```
[56]: zip_path = "DLCV_Lab_Dataset/raw_zips/body-parts-dataset.zip"
extract_path = "DLCV_Lab_Dataset"
unzip_folder(zip_path, extract_path, sub_dir="body-parts-dataset")
```

Extracting body-parts-dataset.zip → DLCV_Lab_Dataset

Extracting: 100% | 2092/2092 [00:00<00:00, 9262.46file/s]

```
[57]: !rm -rf /content/data/body_parts_split
```

```
[58]: import os, random, shutil

def prepare_classification_split(src_dir, dest_dir="/content/data/
↳body_parts_split", val_ratio=0.2, test_ratio=0.1):
    """
    Split a classification dataset into train, val, and test sets.
    Each class folder is split proportionally.

    Args:
        src_dir (str): Source directory containing class folders (e.g., belly/,
↳ears/, etc.)
        dest_dir (str): Destination root for split dataset
        val_ratio (float): Fraction of images to use for validation
        test_ratio (float): Fraction of images to use for testing
    Returns:
        (train_dir, val_dir, test_dir)
    """
    assert val_ratio + test_ratio < 1.0, "val_ratio + test_ratio must be < 1"

    classes = [c for c in os.listdir(src_dir) if os.path.isdir(os.path.
↳join(src_dir, c))]
    os.makedirs(dest_dir, exist_ok=True)

    train_dir = os.path.join(dest_dir, "train")
    val_dir = os.path.join(dest_dir, "val")
    test_dir = os.path.join(dest_dir, "test")

    # Create directories
    for d in [train_dir, val_dir, test_dir]:
        for c in classes:
            os.makedirs(os.path.join(d, c), exist_ok=True)

    # Split and copy files
    for c in classes:
        class_path = os.path.join(src_dir, c)
        imgs = [f for f in os.listdir(class_path) if f.lower().endswith(("
↳.jpg", ".jpeg", ".png"))]
        random.shuffle(imgs)

        n_total = len(imgs)
        n_val = int(n_total * val_ratio)
        n_test = int(n_total * test_ratio)
        n_train = n_total - n_val - n_test

        val_imgs = imgs[:n_val]
        test_imgs = imgs[n_val:n_val+n_test]
        train_imgs = imgs[n_val+n_test:]
```

```

        for img_list, out_dir in zip([train_imgs, val_imgs, test_imgs],
        ↪ [train_dir, val_dir, test_dir]):
            for img in img_list:
                src_path = os.path.join(class_path, img)
                dst_path = os.path.join(out_dir, c, img)
                shutil.copy(src_path, dst_path)

            print(f" {c}: {n_train} train, {n_val} val, {n_test} test")

    print(f"\n Split complete!")
    print(f"Train: {train_dir}\nVal: {val_dir}\nTest: {test_dir}")
    return train_dir, val_dir, test_dir

```

```

[59]: src_dir = "/content/DLCV_Lab_Dataset/Body Parts Dataset"
      train_dir, val_dir, test_dir = prepare_classification_split(src_dir)

```

```

Shoulders: 147 train, 42 val, 21 test
Elbow: 145 train, 41 val, 20 test
Hand: 153 train, 43 val, 21 test
Ear: 145 train, 41 val, 20 test
Eye: 151 train, 43 val, 21 test
Knee: 145 train, 41 val, 20 test
Foot: 147 train, 42 val, 21 test
Neck: 145 train, 41 val, 20 test
Belly: 147 train, 41 val, 20 test
Nose: 147 train, 41 val, 20 test

```

```

Split complete!
Train: /content/data/body_parts_split/train
Val:   /content/data/body_parts_split/val
Test:  /content/data/body_parts_split/test

```

```

[60]: print(train_dir)
      print(val_dir)

```

```

/content/data/body_parts_split/train
/content/data/body_parts_split/val

```

0.3 For Ultralytic Yolo classification model

```

[61]: from ultralytics import YOLO

      model = YOLO("yolov8n-cls.pt") # you can also try yolov8m-cls.pt or ↪
      ↪ yolov9e-cls.pt

      root_dir = "/content/data/body_parts_split"
      model.train(

```

```

data=root_dir,      # folder with subfolders as class names
epochs=30,
imgsz=224,          # image size
batch=16,
lr0=0.001,
project="YOLO_BodyParts",
name="cls_train_run",
device=0 if torch.cuda.is_available() else "cpu"
)

```

Downloading

<https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n-cls.pt> to
'yolov8n-cls.pt': 100% 5.3MB 102.7MB/s 0.1s

Ultralytics 8.3.224 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4, 15095MiB)

```

engine/trainer: agnostic_nms=False, amp=True, augment=False,
auto_augment=randaugment, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None,
classes=None, close_mosaic=10, cls=0.5, compile=False, conf=None,
copy_paste=0.0, copy_paste_mode=flip, cos_lr=False, cutmix=0.0,
data=/content/data/body_parts_split, degrees=0.0, deterministic=True, device=0,
dfl=1.5, dnn=False, dropout=0.0, dynamic=False, embed=None, epochs=30,
erasing=0.4, exist_ok=False, fliplr=0.5, flipud=0.0, format=torchscript,
fraction=1.0, freeze=None, half=False, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4,
imgsz=224, int8=False, iou=0.7, keras=False, kobj=1.0, line_width=None,
lr0=0.001, lrf=0.01, mask_ratio=4, max_det=300, mixup=0.0, mode=train,
model=yolov8n-cls.pt, momentum=0.937, mosaic=1.0, multi_scale=False,
name=cls_train_run, nbs=64, nms=False, opset=None, optimize=False,
optimizer=auto, overlap_mask=True, patience=100, perspective=0.0, plots=True,
pose=12.0, pretrained=True, profile=False, project=YOLO_BodyParts, rect=False,
resume=False, retina_masks=False, save=True, save_conf=False, save_crop=False,
save_dir=/content/YOLO_BodyParts/cls_train_run, save_frames=False,
save_json=False, save_period=-1, save_txt=False, scale=0.5, seed=0, shear=0.0,
show=False, show_boxes=True, show_conf=True, show_labels=True, simplify=True,
single_cls=False, source=None, split=val, stream_buffer=False, task=classify,
time=None, tracker=botsort.yaml, translate=0.1, val=True, verbose=True,
vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0,
warmup_momentum=0.8, weight_decay=0.0005, workers=8, workspace=None

```

train: /content/data/body_parts_split/train... found 1472 images in 10 classes

val: /content/data/body_parts_split/val... found 416 images in 10 classes

test: /content/data/body_parts_split/test... found 204 images in 10 classes

Overriding model.yaml nc=1000 with nc=10

	from	n	params	module
arguments				
0	-1	1	464	ultralytics.nn.modules.conv.Conv

```

[3, 16, 3, 2]
  1          -1  1      4672  ultralytics.nn.modules.conv.Conv
[16, 32, 3, 2]
  2          -1  1      7360  ultralytics.nn.modules.block.C2f
[32, 32, 1, True]
  3          -1  1     18560  ultralytics.nn.modules.conv.Conv
[32, 64, 3, 2]
  4          -1  2     49664  ultralytics.nn.modules.block.C2f
[64, 64, 2, True]
  5          -1  1     73984  ultralytics.nn.modules.conv.Conv
[64, 128, 3, 2]
  6          -1  2    197632  ultralytics.nn.modules.block.C2f
[128, 128, 2, True]
  7          -1  1    295424  ultralytics.nn.modules.conv.Conv
[128, 256, 3, 2]
  8          -1  1    460288  ultralytics.nn.modules.block.C2f
[256, 256, 1, True]
  9          -1  1    343050  ultralytics.nn.modules.head.Classify
[256, 10]
YOLOv8n-cls summary: 56 layers, 1,451,098 parameters, 1,451,098 gradients, 3.4
GFLOPs
Transferred 156/158 items from pretrained weights
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: checks passed
train: Fast image access  (ping: 0.0±0.0 ms, read: 169.3±84.1
MB/s, size: 3.0 KB)
train: Scanning /content/data/body_parts_split/train... 1472
images, 0 corrupt: 100%          1472/1472 5.6Kit/s 0.3s
train: New cache created:
/content/data/body_parts_split/train.cache
val: Fast image access  (ping: 0.0±0.1 ms, read: 89.1±45.8 MB/s,
size: 3.0 KB)
val: Scanning /content/data/body_parts_split/val... 416 images,
0 corrupt: 100%          416/416 4.1Kit/s 0.1s
val: New cache created: /content/data/body_parts_split/val.cache
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.001' and
'momentum=0.937' and determining best 'optimizer', 'lr0' and 'momentum'
automatically...
optimizer: AdamW(lr=0.000714, momentum=0.9) with parameter groups
26 weight(decay=0.0), 27 weight(decay=0.0005), 27 bias(decay=0.0)
Image sizes 224 train, 224 val
Using 2 dataloader workers
Logging results to /content/YOLO_BodyParts/cls_train_run
Starting training for 30 epochs...

```

Epoch	GPU_mem	loss	Instances	Size
1/30	1.76G	2.182	16	224: 100%

92/92 9.7it/s 9.5s

		classes	top1_acc	top5_acc: 100%	13/13
19.6it/s	0.7s				
		all	0.416	0.827	
Epoch	GPU_mem	loss	Instances	Size	
2/30	1.76G	1.633	16	224: 100%	
92/92	12.4it/s	7.4s			
		classes	top1_acc	top5_acc: 100%	13/13
11.3it/s	1.1s				
		all	0.582	0.942	
Epoch	GPU_mem	loss	Instances	Size	
3/30	1.76G	1.245	16	224: 100%	
92/92	13.2it/s	7.0s			
		classes	top1_acc	top5_acc: 100%	13/13
17.7it/s	0.7s				
		all	0.642	0.942	
Epoch	GPU_mem	loss	Instances	Size	
4/30	1.76G	1.025	16	224: 100%	
92/92	9.8it/s	9.4s			
		classes	top1_acc	top5_acc: 100%	13/13
16.1it/s	0.8s				
		all	0.661	0.952	
Epoch	GPU_mem	loss	Instances	Size	
5/30	1.76G	0.9045	16	224: 100%	
92/92	11.8it/s	7.8s			
		classes	top1_acc	top5_acc: 100%	13/13
12.4it/s	1.0s				
		all	0.712	0.954	
Epoch	GPU_mem	loss	Instances	Size	
6/30	1.76G	0.845	16	224: 100%	
92/92	13.6it/s	6.8s			
		classes	top1_acc	top5_acc: 100%	13/13
21.0it/s	0.6s				
		all	0.719	0.945	
Epoch	GPU_mem	loss	Instances	Size	
7/30	1.76G	0.7255	16	224: 100%	
92/92	11.5it/s	8.0s			
		classes	top1_acc	top5_acc: 100%	13/13
17.3it/s	0.8s				
		all	0.743	0.964	
Epoch	GPU_mem	loss	Instances	Size	
8/30	1.76G	0.6373	16	224: 100%	

92/92	12.7it/s	7.3s			
		classes	top1_acc	top5_acc: 100%	13/13
10.3it/s	1.3s				
		all	0.697	0.959	
	Epoch	GPU_mem	loss	Instances	Size
	9/30	1.76G	0.5736	16	224: 100%
92/92	13.2it/s	7.0s			
		classes	top1_acc	top5_acc: 100%	13/13
20.2it/s	0.6s				
		all	0.733	0.962	
	Epoch	GPU_mem	loss	Instances	Size
	10/30	1.76G	0.5304	16	224: 100%
92/92	11.1it/s	8.3s			
		classes	top1_acc	top5_acc: 100%	13/13
17.9it/s	0.7s				
		all	0.767	0.962	
	Epoch	GPU_mem	loss	Instances	Size
	11/30	1.76G	0.4902	16	224: 100%
92/92	12.1it/s	7.6s			
		classes	top1_acc	top5_acc: 100%	13/13
12.3it/s	1.1s				
		all	0.736	0.954	
	Epoch	GPU_mem	loss	Instances	Size
	12/30	1.76G	0.4506	16	224: 100%
92/92	13.5it/s	6.8s			
		classes	top1_acc	top5_acc: 100%	13/13
16.4it/s	0.8s				
		all	0.733	0.964	
	Epoch	GPU_mem	loss	Instances	Size
	13/30	1.76G	0.446	16	224: 100%
92/92	11.6it/s	7.9s			
		classes	top1_acc	top5_acc: 100%	13/13
20.1it/s	0.6s				
		all	0.733	0.959	
	Epoch	GPU_mem	loss	Instances	Size
	14/30	1.76G	0.4387	16	224: 100%
92/92	13.1it/s	7.0s			
		classes	top1_acc	top5_acc: 100%	13/13
10.4it/s	1.2s				
		all	0.757	0.966	
	Epoch	GPU_mem	loss	Instances	Size

15/30	1.76G	0.3921	16	224: 100%
92/92 13.1it/s	7.0s			
	classes	top1_acc	top5_acc: 100%	13/13
17.7it/s	0.7s			
	all	0.752	0.962	

Epoch	GPU_mem	loss	Instances	Size
16/30	1.76G	0.3457	16	224: 100%
92/92 11.6it/s	7.9s			
	classes	top1_acc	top5_acc: 100%	13/13
15.8it/s	0.8s			
	all	0.755	0.969	

Epoch	GPU_mem	loss	Instances	Size
17/30	1.76G	0.3432	16	224: 100%
92/92 13.7it/s	6.7s			
	classes	top1_acc	top5_acc: 100%	13/13
11.1it/s	1.2s			
	all	0.745	0.962	

Epoch	GPU_mem	loss	Instances	Size
18/30	1.76G	0.3476	16	224: 100%
92/92 12.6it/s	7.3s			
	classes	top1_acc	top5_acc: 100%	13/13
18.2it/s	0.7s			
	all	0.74	0.959	

Epoch	GPU_mem	loss	Instances	Size
19/30	1.76G	0.2961	16	224: 100%
92/92 11.4it/s	8.0s			
	classes	top1_acc	top5_acc: 100%	13/13
19.8it/s	0.7s			
	all	0.75	0.959	

Epoch	GPU_mem	loss	Instances	Size
20/30	1.76G	0.2906	16	224: 100%
92/92 13.9it/s	6.6s			
	classes	top1_acc	top5_acc: 100%	13/13
13.3it/s	1.0s			
	all	0.74	0.959	

Epoch	GPU_mem	loss	Instances	Size
21/30	1.76G	0.2869	16	224: 100%
92/92 12.0it/s	7.7s			
	classes	top1_acc	top5_acc: 100%	13/13
19.6it/s	0.7s			
	all	0.764	0.957	

Epoch	GPU_mem	loss	Instances	Size
22/30	1.76G	0.2477	16	224: 100%
92/92 10.6it/s	8.7s			
	classes	top1_acc	top5_acc: 100%	13/13
18.6it/s	0.7s			
	all	0.76	0.954	

Epoch	GPU_mem	loss	Instances	Size
23/30	1.76G	0.2537	16	224: 100%
92/92 12.9it/s	7.1s			
	classes	top1_acc	top5_acc: 100%	13/13
12.4it/s	1.0s			
	all	0.762	0.962	

Epoch	GPU_mem	loss	Instances	Size
24/30	1.76G	0.2484	16	224: 100%
92/92 13.2it/s	7.0s			
	classes	top1_acc	top5_acc: 100%	13/13
16.6it/s	0.8s			
	all	0.764	0.959	

Epoch	GPU_mem	loss	Instances	Size
25/30	1.76G	0.2418	16	224: 100%
92/92 11.6it/s	7.9s			
	classes	top1_acc	top5_acc: 100%	13/13
17.4it/s	0.7s			
	all	0.755	0.957	

Epoch	GPU_mem	loss	Instances	Size
26/30	1.76G	0.2295	16	224: 100%
92/92 13.3it/s	6.9s			
	classes	top1_acc	top5_acc: 100%	13/13
10.6it/s	1.2s			
	all	0.767	0.962	

Epoch	GPU_mem	loss	Instances	Size
27/30	1.76G	0.2299	16	224: 100%
92/92 13.6it/s	6.8s			
	classes	top1_acc	top5_acc: 100%	13/13
15.9it/s	0.8s			
	all	0.76	0.954	

Epoch	GPU_mem	loss	Instances	Size
28/30	1.76G	0.2392	16	224: 100%
92/92 11.5it/s	8.0s			
	classes	top1_acc	top5_acc: 100%	13/13
16.8it/s	0.8s			
	all	0.74	0.954	

Epoch	GPU_mem	loss	Instances	Size
29/30	1.76G	0.2299	16	224: 100%
92/92	13.7it/s 6.7s			
	classes	top1_acc	top5_acc: 100%	13/13
10.0it/s 1.3s				
	all	0.74	0.954	

Epoch	GPU_mem	loss	Instances	Size
30/30	1.76G	0.2173	16	224: 100%
92/92	12.7it/s 7.3s			
	classes	top1_acc	top5_acc: 100%	13/13
20.5it/s 0.6s				
	all	0.75	0.954	

30 epochs completed in 0.073 hours.

Optimizer stripped from /content/YOLO_BodyParts/cls_train_run/weights/last.pt, 3.0MB

Optimizer stripped from /content/YOLO_BodyParts/cls_train_run/weights/best.pt, 3.0MB

Validating /content/YOLO_BodyParts/cls_train_run/weights/best.pt...

Ultralytics 8.3.224 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4, 15095MiB)

YOLOv8n-cls summary (fused): 30 layers, 1,447,690 parameters, 0 gradients, 3.3 GFLOPs

train: /content/data/body_parts_split/train... found 1472 images in 10 classes

val: /content/data/body_parts_split/val... found 416 images in 10 classes

test: /content/data/body_parts_split/test... found 204 images in 10 classes

	classes	top1_acc	top5_acc: 100%	13/13
14.8it/s 0.9s				
	all	0.767	0.959	

Speed: 0.3ms preprocess, 0.8ms inference, 0.0ms loss, 0.0ms postprocess per image

Results saved to /content/YOLO_BodyParts/cls_train_run

[61]: ultralytics.utils.metrics.ClassifyMetrics object with attributes:

```

confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at
0x7d566b720fe0>
curves: []
curves_results: []
fitness: 0.8629807829856873
keys: ['metrics/accuracy_top1', 'metrics/accuracy_top5']

```

```

results_dict: {'metrics/accuracy_top1': 0.7668269276618958,
'metrics/accuracy_top5': 0.9591346383094788, 'fitness': 0.8629807829856873}
save_dir: PosixPath('/content/YOLO_BodyParts/cls_train_run')
speed: {'preprocess': 0.2922188125073542, 'inference': 0.8299442475976531,
'loss': 0.00032169230315468245, 'postprocess': 0.0006169110607655826}
task: 'classify'
top1: 0.7668269276618958
top5: 0.9591346383094788

```

```

[62]: metrics = model.val()
      print(metrics)

```

```

Ultralytics 8.3.224 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4,
15095MiB)
YOLOv8n-cls summary (fused): 30 layers, 1,447,690 parameters, 0 gradients, 3.3
GFLOPs
train: /content/data/body_parts_split/train... found 1472 images in
10 classes
val: /content/data/body_parts_split/val... found 416 images in 10
classes
test: /content/data/body_parts_split/test... found 204 images in 10
classes
val: Fast image access (ping: 0.0±0.0 ms, read: 112.6±43.0 MB/s,
size: 3.0 KB)
val: Scanning /content/data/body_parts_split/val... 416 images,
0 corrupt: 100% 416/416 759.3Kit/s 0.0s
          classes top1_acc top5_acc: 100% 26/26
18.8it/s 1.4s
          all 0.767 0.962
Speed: 0.3ms preprocess, 1.5ms inference, 0.0ms loss, 0.0ms postprocess per
image
Results saved to /content/runs/classify/val
ultralytics.utils.metrics.ClassifyMetrics object with attributes:

confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at
0x7d56d0bb2240>
curves: []
curves_results: []
fitness: 0.8641826808452606
keys: ['metrics/accuracy_top1', 'metrics/accuracy_top5']
results_dict: {'metrics/accuracy_top1': 0.7668269276618958,
'metrics/accuracy_top5': 0.9615384340286255, 'fitness': 0.8641826808452606}
save_dir: PosixPath('/content/runs/classify/val')
speed: {'preprocess': 0.3024327283622614, 'inference': 1.474886437506484,
'loss': 0.0007168894237216032, 'postprocess': 0.0011647067332467907}
task: 'classify'
top1: 0.7668269276618958
top5: 0.9615384340286255

```

```
[63]: results = model.predict(source="/content/DLCV_Lab_Dataset/Body Parts Dataset/  
↳ Ear/Ear_1.jpg", show=True)
```

WARNING Environment does not support cv2.imshow() or PIL Image.show()

image 1/1 /content/DLCV_Lab_Dataset/Body Parts Dataset/Ear/Ear_1.jpg: 224x224
Ear 1.00, Neck 0.00, Nose 0.00, Belly 0.00, Eye 0.00, 2.3ms
Speed: 2.7ms preprocess, 2.3ms inference, 0.1ms postprocess per image at shape
(1, 3, 224, 224)

```
[64]: import pandas as pd  
import matplotlib.pyplot as plt  
import os  
  
def plot_ultra_yolo_cls_loss(results_dir="runs/classify/cls_train_run"):  
    csv_path = os.path.join(results_dir, "results.csv")  
    if not os.path.exists(csv_path):  
        print(" results.csv not found - check your run folder.")  
        return  
  
    df = pd.read_csv(csv_path)  
    plt.figure(figsize=(8,5))  
    plt.plot(df["epoch"], df["train/loss"], label="Train Loss", marker='o')  
    plt.plot(df["epoch"], df["val/loss"], label="Val Loss", marker='s')  
    plt.xlabel("Epoch")  
    plt.ylabel("Loss")  
    plt.title("YOLO Classification Loss")  
    plt.grid(True, linestyle='--', alpha=0.5)  
    plt.legend()  
    plt.show()  
  
plot_ultra_yolo_cls_loss("runs/classify/cls_train_run")
```

results.csv not found - check your run folder.

```
[65]: import os  
import matplotlib.pyplot as plt  
import cv2  
from ultralytics import YOLO  
  
def test_yolo_classification_model(model, test_dir, num_images=9, conf=0.25,   
↳ device="cuda"):  
    """  
    Evaluate and visualize YOLO classification model on a test set.  
  
    Args:  
        model: Trained YOLO classification model (YOLO object)
```

```

    test_dir (str): Path to test dataset folder
    num_images (int): Number of sample images to visualize
    conf (float): Confidence threshold for predictions
    device (str): 'cuda' or 'cpu'
"""
# Run validation/testing
print(f" Running test on: {test_dir}")
metrics = model.val(data=test_dir, device=device, split="test", conf=conf)
print(" Evaluation complete.")
print(metrics.results_dict)

# Collect test image paths
img_paths = []
for root, _, files in os.walk(test_dir):
    for f in files:
        if f.lower().endswith((".jpg", ".jpeg", ".png")):
            img_paths.append(os.path.join(root, f))
img_paths = sorted(img_paths)

if len(img_paths) == 0:
    print(" No images found in test directory.")
    return

# Run predictions
results = model.predict(source=img_paths[:num_images], conf=conf,
↪device=device, verbose=False)

# Plot results
cols = 3
rows = int((num_images + cols - 1) / cols)
plt.figure(figsize=(5 * cols, 5 * rows))

for i, (path, res) in enumerate(zip(img_paths[:num_images], results)):
    img = cv2.imread(path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    pred_id = int(res.probs.top1)
    pred_label = res.names[pred_id]
    conf_score = float(res.probs.top1conf)

    ax = plt.subplot(rows, cols, i + 1)
    ax.imshow(img)
    ax.axis("off")
    ax.set_title(f"#{i+1}\n{pred_label} ({conf_score:.2f})", color="green")

plt.tight_layout()
plt.show()

```



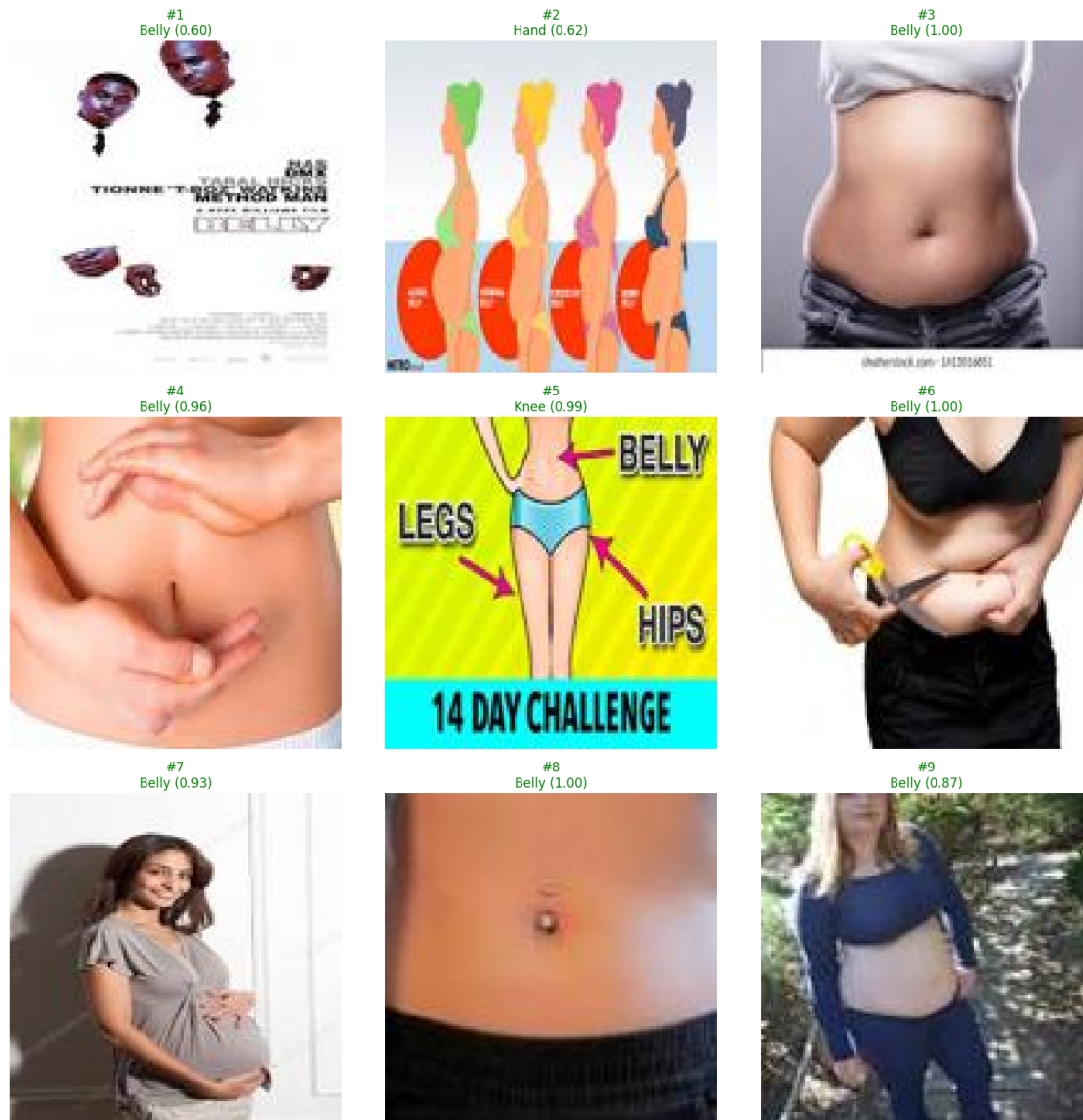
```
[66]: from ultralytics import YOLO
```

```
# Load your trained YOLOv8 classification model
# model = YOLO("runs/classify/train/weights/best.pt")

# Path to test set (created by prepare_classification_split)
test_dir = "/content/data/body_parts_split/test"

# Evaluate and show predictions
test_yolo_classification_model(model, test_dir, num_images=9)
```

```
Running test on: /content/data/body_parts_split/test
Ultralytics 8.3.224 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4,
15095MiB)
WARNING Dataset 'split=train' not found at
/content/data/body_parts_split/test/train
Found 204 images in subdirectories. Attempting to split...
Splitting /content/data/body_parts_split/test (10 classes, 204 images) into 80%
train, 20% val...
Split complete in /content/data/body_parts_split/test_split
WARNING Dataset 'split=test' not found, using 'split=val' instead.
train: /content/data/body_parts_split/test_split/train... found 160
images in 10 classes
val: /content/data/body_parts_split/test_split/val... found 44
images in 10 classes
test: /content/data/body_parts_split/test_split/val... found 44
images in 10 classes
test: Fast image access (ping: 0.0±0.1 ms, read: 92.8±32.7 MB/s,
size: 3.0 KB)
test: Scanning /content/data/body_parts_split/test_split/val...
44 images, 0 corrupt: 100% 44/44 4.9Kit/s 0.0s
test: New cache created:
/content/data/body_parts_split/test_split/val.cache
classes top1_acc top5_acc: 100% 3/3 6.0it/s
0.5s
all 0.795 0.909
Speed: 1.8ms preprocess, 6.3ms inference, 0.0ms loss, 0.0ms postprocess per
image
Results saved to /content/runs/classify/val2
Evaluation complete.
{'metrics/accuracy_top1': 0.7954545617103577, 'metrics/accuracy_top5':
0.9090909361839294, 'fitness': 0.8522727489471436}
```



```
[67]: save_model(name="ultra_yolo_cls", model=model)
```

Ultralytics YOLO model saved to saved_models/ultra_yolo_cls.pt

```
[67]: 'saved_models/ultra_yolo_cls.pt'
```

0.4 For Ultralytic Yolo detection model

```
[68]: from ultralytics import YOLO
```

```
# Load detection model (not classification)
model = YOLO("yolov8n.pt") # or yolov8s.pt for higher accuracy
```

Downloading

<https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n.pt> to
'yolov8n.pt': 100% 6.2MB 122.8MB/s 0.1s

```
[69]: # !rm -rf /content/data/body_parts_split
```

```
[70]: import cv2, numpy as np
from tqdm import tqdm

def generate_pseudo_yolo_labels(root_dir, out_root="body_parts_yolo"):
    """
    Generate YOLO-style pseudo bounding boxes from a folder-based_
    ↪classification dataset.
    One bounding box per image (based on largest contour).
    """
    for split in ["train", "val", "test"]:
        img_split = os.path.join(root_dir, split)
        out_img_dir = os.path.join(out_root, "images", split)
        out_lbl_dir = os.path.join(out_root, "labels", split)
        os.makedirs(out_img_dir, exist_ok=True)
        os.makedirs(out_lbl_dir, exist_ok=True)

        class_names = sorted(os.listdir(img_split))
        class_to_id = {c: i for i, c in enumerate(class_names)}

        for cls in class_names:
            class_dir = os.path.join(img_split, cls)
            imgs = [f for f in os.listdir(class_dir) if f.lower().endswith(("."
            ↪jpg", ".jpeg", ".png"))]

            for f in tqdm(imgs, desc=f"{split}/{cls}"):
                img_path = os.path.join(class_dir, f)
                img = cv2.imread(img_path)
                if img is None:
                    continue

                H, W = img.shape[:2]
                gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
                blur = cv2.GaussianBlur(gray, (5, 5), 0)
                _, thresh = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.
                ↪THRESH_OTSU)

                contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.
                ↪CHAIN_APPROX_SIMPLE)
                if not contours:
                    x, y, w, h = 0, 0, W, H
                else:
```

```

        x, y, w, h = cv2.boundingRect(max(contours, key=cv2.
↪contourArea))

        x_center, y_center = (x + w/2)/W, (y + h/2)/H
        w_norm, h_norm = w/W, h/H

        label_path = os.path.join(out_lbl_dir, f.rsplit('.', 1)[0] + ".
↪txt")

        cls_id = class_to_id[cls]
        with open(label_path, "w") as lf:
            lf.write(f"{cls_id} {x_center:.6f} {y_center:.6f} {w_norm:.
↪6f} {h_norm:.6f}\n")

        cv2.imwrite(os.path.join(out_img_dir, f), img)

    print(f" YOLO pseudo-labels generated at: {out_root}")
    return out_root

```

```
[71]: pseudo_root = generate_pseudo_yolo_labels("/content/data/body_parts_split")
```

```

train/Belly: 100%|      | 147/147 [00:00<00:00, 1548.51it/s]
train/Ear: 100%|      | 145/145 [00:00<00:00, 1938.45it/s]
train/Elbow: 100%|     | 145/145 [00:00<00:00, 1931.81it/s]
train/Eye: 100%|      | 151/151 [00:00<00:00, 1723.60it/s]
train/Foot: 100%|     | 147/147 [00:00<00:00, 1700.44it/s]
train/Hand: 100%|     | 153/153 [00:00<00:00, 1969.19it/s]
train/Knee: 100%|     | 145/145 [00:00<00:00, 1754.87it/s]
train/Neck: 100%|     | 145/145 [00:00<00:00, 1891.08it/s]
train/Nose: 100%|     | 147/147 [00:00<00:00, 1921.57it/s]
train/Shoulders: 100%| | 147/147 [00:00<00:00, 1912.15it/s]
val/Belly: 100%|      | 41/41 [00:00<00:00, 1810.06it/s]
val/Ear: 100%|       | 41/41 [00:00<00:00, 1607.99it/s]
val/Elbow: 100%|     | 41/41 [00:00<00:00, 1863.55it/s]
val/Eye: 100%|      | 43/43 [00:00<00:00, 1799.45it/s]
val/Foot: 100%|     | 42/42 [00:00<00:00, 1912.86it/s]
val/Hand: 100%|     | 43/43 [00:00<00:00, 1998.42it/s]
val/Knee: 100%|     | 41/41 [00:00<00:00, 1833.39it/s]
val/Neck: 100%|     | 41/41 [00:00<00:00, 1873.64it/s]
val/Nose: 100%|     | 41/41 [00:00<00:00, 1857.29it/s]
val/Shoulders: 100%| | 42/42 [00:00<00:00, 1820.41it/s]
test/Belly: 100%|     | 20/20 [00:00<00:00, 1537.42it/s]
test/Ear: 100%|      | 20/20 [00:00<00:00, 1749.49it/s]
test/Elbow: 100%|    | 20/20 [00:00<00:00, 1808.98it/s]
test/Eye: 100%|     | 21/21 [00:00<00:00, 1784.81it/s]
test/Foot: 100%|     | 21/21 [00:00<00:00, 1723.08it/s]
test/Hand: 100%|     | 21/21 [00:00<00:00, 1810.86it/s]
test/Knee: 100%|     | 20/20 [00:00<00:00, 1609.26it/s]

```

```
test/Neck: 100%|      | 20/20 [00:00<00:00, 1715.46it/s]
test/Nose: 100%|      | 20/20 [00:00<00:00, 1720.18it/s]
test/Shoulders: 100%|    | 21/21 [00:00<00:00, 1730.60it/s]
```

YOLO pseudo-labels generated at: body_parts_yolo

```
[72]: import os, yaml

def create_yolo_yaml(out_root, class_names):
    out_root = os.path.abspath(out_root) # make it absolute
    yaml_dict = {
        "train": os.path.join(out_root, "images/train"),
        "val": os.path.join(out_root, "images/val"),
        "test": os.path.join(out_root, "images/test"),
        "nc": len(class_names),
        "names": class_names
    }
    yaml_path = os.path.join(out_root, "body_parts.yaml")
    with open(yaml_path, "w") as f:
        yaml.dump(yaml_dict, f)
    print(f" YAML created at: {yaml_path}")
    return yaml_path
```

```
[73]: class_names = sorted(os.listdir("/content/data/body_parts_split/train"))
yaml_path = create_yolo_yaml("/content/body_parts_yolo", class_names)
```

YAML created at: /content/body_parts_yolo/body_parts.yaml

```
[74]: !cat /content/body_parts_yolo/body_parts.yaml
```

```
names:
- Belly
- Ear
- Elbow
- Eye
- Foot
- Hand
- Knee
- Neck
- Nose
- Shoulders
nc: 10
test: /content/body_parts_yolo/images/test
train: /content/body_parts_yolo/images/train
val: /content/body_parts_yolo/images/val
```

```
[75]: from ultralytics import YOLO
```

```
model = YOLO("yolov8n.pt")
results = model.train(data=yaml_path, epochs=20, imgsz=416, batch=8)
```

Ultralytics 8.3.224 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4, 15095MiB)

```
engine/trainer: agnostic_nms=False, amp=True, augment=False,
auto_augment=randaugment, batch=8, bgr=0.0, box=7.5, cache=False, cfg=None,
classes=None, close_mosaic=10, cls=0.5, compile=False, conf=None,
copy_paste=0.0, copy_paste_mode=flip, cos_lr=False, cutmix=0.0,
data=/content/body_parts_yolo/body_parts.yaml, degrees=0.0, deterministic=True,
device=0, dfl=1.5, dnn=False, dropout=0.0, dynamic=False, embed=None, epochs=30,
erasing=0.4, exist_ok=False, fliplr=0.5, flipud=0.0, format=torchscript,
fraction=1.0, freeze=None, half=False, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4,
imgsz=416, int8=False, iou=0.7, keras=False, kobj=1.0, line_width=None,
lr0=0.01, lrf=0.01, mask_ratio=4, max_det=300, mixup=0.0, mode=train,
model=yolov8n.pt, momentum=0.937, mosaic=1.0, multi_scale=False, name=train2,
nbs=64, nms=False, opset=None, optimize=False, optimizer=auto,
overlap_mask=True, patience=100, perspective=0.0, plots=True, pose=12.0,
pretrained=True, profile=False, project=None, rect=False, resume=False,
retina_masks=False, save=True, save_conf=False, save_crop=False,
save_dir=/content/runs/detect/train2, save_frames=False, save_json=False,
save_period=-1, save_txt=False, scale=0.5, seed=0, shear=0.0, show=False,
show_boxes=True, show_conf=True, show_labels=True, simplify=True,
single_cls=False, source=None, split=val, stream_buffer=False, task=detect,
time=None, tracker=botsort.yaml, translate=0.1, val=True, verbose=True,
vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0,
warmup_momentum=0.8, weight_decay=0.0005, workers=8, workspace=None
Overriding model.yaml nc=80 with nc=10
```

	from	n	params	module
arguments				
0	-1	1	464	ultralytics.nn.modules.conv.Conv
[3, 16, 3, 2]				
1	-1	1	4672	ultralytics.nn.modules.conv.Conv
[16, 32, 3, 2]				
2	-1	1	7360	ultralytics.nn.modules.block.C2f
[32, 32, 1, True]				
3	-1	1	18560	ultralytics.nn.modules.conv.Conv
[32, 64, 3, 2]				
4	-1	2	49664	ultralytics.nn.modules.block.C2f
[64, 64, 2, True]				
5	-1	1	73984	ultralytics.nn.modules.conv.Conv
[64, 128, 3, 2]				
6	-1	2	197632	ultralytics.nn.modules.block.C2f
[128, 128, 2, True]				
7	-1	1	295424	ultralytics.nn.modules.conv.Conv

```

[128, 256, 3, 2]
  8          -1  1    460288  ultralytics.nn.modules.block.C2f
[256, 256, 1, True]
  9          -1  1    164608  ultralytics.nn.modules.block.SPPF
[256, 256, 5]
 10          -1  1         0  torch.nn.modules.upsampling.Upsample
[None, 2, 'nearest']
 11         [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat
[1]
 12          -1  1    148224  ultralytics.nn.modules.block.C2f
[384, 128, 1]
 13          -1  1         0  torch.nn.modules.upsampling.Upsample
[None, 2, 'nearest']
 14         [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat
[1]
 15          -1  1    37248  ultralytics.nn.modules.block.C2f
[192, 64, 1]
 16          -1  1    36992  ultralytics.nn.modules.conv.Conv
[64, 64, 3, 2]
 17         [-1, 12]  1         0  ultralytics.nn.modules.conv.Concat
[1]
 18          -1  1    123648  ultralytics.nn.modules.block.C2f
[192, 128, 1]
 19          -1  1    147712  ultralytics.nn.modules.conv.Conv
[128, 128, 3, 2]
 20         [-1, 9]  1         0  ultralytics.nn.modules.conv.Concat
[1]
 21          -1  1    493056  ultralytics.nn.modules.block.C2f
[384, 256, 1]
 22        [15, 18, 21]  1    753262  ultralytics.nn.modules.head.Detect
[10, [64, 128, 256]]
Model summary: 129 layers, 3,012,798 parameters, 3,012,782 gradients, 8.2 GFLOPs

```

Transferred 319/355 items from pretrained weights

Freezing layer 'model.22.dfl.conv.weight'

AMP: running Automatic Mixed Precision (AMP) checks...

AMP: checks passed

train: Fast image access (ping: 0.0±0.0 ms, read: 367.8±163.2 MB/s, size: 6.5 KB)

train: Scanning /content/body_parts_yolo/labels/train... 1472

images, 0 backgrounds, 0 corrupt: 100% 1472/1472 2.5Kit/s 0.6s

train: New cache created:

/content/body_parts_yolo/labels/train.cache

albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8, 8))

val: Fast image access (ping: 0.0±0.0 ms, read: 119.1±72.0 MB/s,

```

size: 4.9 KB)
val: Scanning /content/body_parts_yolo/labels/val... 416 images,
0 backgrounds, 0 corrupt: 100%          416/416 1.2Kit/s 0.3s
val: New cache created: /content/body_parts_yolo/labels/val.cache
Plotting labels to /content/runs/detect/train2/labels.jpg...
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and
'momentum=0.937' and determining best 'optimizer', 'lr0' and 'momentum'
automatically...
optimizer: AdamW(lr=0.000714, momentum=0.9) with parameter groups
57 weight(decay=0.0), 64 weight(decay=0.0005), 63 bias(decay=0.0)
Image sizes 416 train, 416 val
Using 2 dataloader workers
Logging results to /content/runs/detect/train2
Starting training for 30 epochs...

```

	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	1/30	0.768G	0.6505	3.317	1.249	24	
416: 100%		184/184	8.5it/s	21.6s			
		Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%			26/26	11.9it/s	2.2s		
		all	416	416	0.107	0.206	0.136
0.089							

	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	2/30	0.805G	0.6103	2.789	1.217	23	
416: 100%		184/184	9.4it/s	19.6s			
		Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%			26/26	12.2it/s	2.1s		
		all	416	416	0.135	0.487	0.165
0.1							

	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	3/30	0.805G	0.5652	2.49	1.182	16	
416: 100%		184/184	10.1it/s	18.3s			
		Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%			26/26	8.8it/s	3.0s		
		all	416	416	0.14	0.481	0.222
0.171							

	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	4/30	0.805G	0.5826	2.342	1.181	18	
416: 100%		184/184	10.2it/s	18.1s			
		Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%			26/26	12.4it/s	2.1s		
		all	416	416	0.147	0.529	0.218
0.171							

	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
--	-------	---------	----------	----------	----------	-----------	------

5/30	0.805G	0.5418	2.235	1.159	22	
416: 100%	184/184	9.5it/s	19.4s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.4it/s	2.1s			
	all	416	416	0.201	0.524	0.297
0.242						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
6/30	0.805G	0.5159	2.164	1.148	26	
416: 100%	184/184	10.0it/s	18.4s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	9.3it/s	2.8s			
	all	416	416	0.218	0.552	0.292
0.214						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
7/30	0.811G	0.4893	2.101	1.127	24	
416: 100%	184/184	6.8it/s	27.1s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.6it/s	2.1s			
	all	416	416	0.213	0.471	0.32
0.244						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
8/30	0.828G	0.4882	2.063	1.124	21	
416: 100%	184/184	10.1it/s	18.1s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	11.9it/s	2.2s			
	all	416	416	0.272	0.482	0.361
0.291						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
9/30	0.846G	0.4803	2.026	1.122	23	
416: 100%	184/184	9.7it/s	19.0s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.4it/s	2.1s			
	all	416	416	0.285	0.418	0.373
0.297						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
10/30	0.861G	0.4707	1.988	1.121	26	
416: 100%	184/184	9.6it/s	19.1s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	11.9it/s	2.2s			
	all	416	416	0.333	0.467	0.392
0.317						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
-------	---------	----------	----------	----------	-----------	------

11/30	0.879G	0.4528	1.966	1.115	23	
416: 100%	184/184	9.5it/s	19.3s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	11.8it/s	2.2s			
	all	416	416	0.292	0.456	0.391
0.312						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
12/30	0.896G	0.4408	1.923	1.107	25	
416: 100%	184/184	9.5it/s	19.3s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.7it/s	2.0s			
	all	416	416	0.355	0.533	0.445
0.376						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
13/30	0.914G	0.4341	1.899	1.098	23	
416: 100%	184/184	9.4it/s	19.5s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.5it/s	2.1s			
	all	416	416	0.38	0.493	0.448
0.374						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
14/30	0.932G	0.4197	1.876	1.098	24	
416: 100%	184/184	9.8it/s	18.7s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	11.9it/s	2.2s			
	all	416	416	0.439	0.488	0.428
0.342						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
15/30	0.947G	0.4056	1.851	1.087	27	
416: 100%	184/184	9.5it/s	19.5s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.5it/s	2.1s			
	all	416	416	0.463	0.44	0.448
0.387						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
16/30	0.965G	0.3969	1.816	1.081	19	
416: 100%	184/184	9.6it/s	19.3s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	11.3it/s	2.3s			
	all	416	416	0.415	0.522	0.489
0.417						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
-------	---------	----------	----------	----------	-----------	------

17/30	0.982G	0.3943	1.78	1.078	20	
416: 100%	184/184	10.1it/s	18.1s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.7it/s	2.1s			
	all	416	416	0.383	0.544	0.48

0.399

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
18/30	1G	0.3887	1.773	1.073	25	
416: 100%	184/184	9.5it/s	19.3s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.7it/s	2.0s			
	all	416	416	0.432	0.511	0.488

0.412

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
19/30	1.02G	0.3932	1.756	1.081	20	
416: 100%	184/184	9.8it/s	18.7s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	9.8it/s	2.6s			
	all	416	416	0.473	0.496	0.527

0.444

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
20/30	1.03G	0.3682	1.711	1.063	26	
416: 100%	184/184	10.1it/s	18.2s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.7it/s	2.0s			
	all	416	416	0.437	0.569	0.506

0.433

Closing dataloader mosaic

`albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8, 8))`

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
21/30	1.05G	0.4037	1.772	1.154	8	
416: 100%	184/184	9.3it/s	19.8s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.1it/s	2.1s			
	all	416	416	0.43	0.61	0.542

0.456

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
22/30	1.07G	0.3869	1.6	1.135	8	
416: 100%	184/184	10.0it/s	18.3s			
	Class	Images	Instances	Box(P	R	mAP50

mAP50-95): 100% 26/26 9.9it/s 2.6s
all 416 416 0.53 0.567 0.57
0.483

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
23/30	1.08G	0.3681	1.528	1.121	8	
416: 100%	184/184	10.4it/s	17.8s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.4it/s	2.1s			
	all	416	416	0.592	0.537	0.591

0.495

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
24/30	1.1G	0.3417	1.452	1.1	8	
416: 100%	184/184	9.8it/s	18.9s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.5it/s	2.1s			
	all	416	416	0.472	0.619	0.582

0.494

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
25/30	1.12G	0.3355	1.423	1.109	8	
416: 100%	184/184	10.4it/s	17.7s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	10.9it/s	2.4s			
	all	416	416	0.58	0.588	0.626

0.54

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
26/30	1.13G	0.3211	1.376	1.087	8	
416: 100%	184/184	10.0it/s	18.4s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.4it/s	2.1s			
	all	416	416	0.585	0.563	0.611

0.523

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
27/30	1.15G	0.3053	1.357	1.075	8	
416: 100%	184/184	10.0it/s	18.4s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	10.3it/s	2.5s			
	all	416	416	0.549	0.593	0.608

0.538

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
28/30	1.17G	0.3009	1.28	1.067	8	
416: 100%	184/184	10.4it/s	17.7s			
	Class	Images	Instances	Box(P	R	mAP50

mAP50-95): 100% 26/26 12.2it/s 2.1s
all 416 416 0.619 0.548 0.616
0.543

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
29/30	1.19G	0.3045	1.272	1.082	8	
416: 100%	184/184	9.8it/s	18.7s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	12.1it/s	2.2s			
	all	416	416	0.636	0.55	0.631
						0.556

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
30/30	1.2G	0.3026	1.222	1.076	8	
416: 100%	184/184	10.4it/s	17.7s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	9.7it/s	2.7s			
	all	416	416	0.637	0.559	0.632
						0.558

30 epochs completed in 0.180 hours.

Optimizer stripped from /content/runs/detect/train2/weights/last.pt, 6.2MB

Optimizer stripped from /content/runs/detect/train2/weights/best.pt, 6.2MB

Validating /content/runs/detect/train2/weights/best.pt...

Ultralytics 8.3.224 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4, 15095MiB)

Model summary (fused): 72 layers, 3,007,598 parameters, 0 gradients, 8.1 GFLOPs

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%	26/26	8.5it/s	3.0s			
	all	416	416	0.634	0.56	0.631
						0.557
	Belly	41	41	0.555	0.537	0.603
						0.483
	Ear	41	41	0.723	0.829	0.886
						0.8
	Elbow	41	41	0.628	0.33	0.45
						0.349
	Eye	43	43	0.813	0.791	0.88
						0.812
	Foot	42	42	0.693	0.452	0.523
						0.488
	Hand	43	43	0.632	0.698	0.712
						0.672
	Knee	41	41	0.595	0.39	0.501
						0.396
	Neck	41	41	0.603	0.561	0.65
						0.57

	Nose	41	41	0.617	0.537	0.652
0.612						
	Shoulders	42	42	0.477	0.476	0.457
0.384						

Speed: 0.2ms preprocess, 1.6ms inference, 0.0ms loss, 2.1ms postprocess per image

Results saved to /content/runs/detect/train2

```
[76]: import pandas as pd
import matplotlib.pyplot as plt
import os

def plot_yolo_training_metrics(results_csv_path):
    """
    Plot YOLO training metrics (losses and mAP) from results.csv.
    Works with YOLOv8/v9/v12 detection logs.

    Args:
        results_csv_path (str): Path to YOLO training results.csv file
    """
    if not os.path.exists(results_csv_path):
        print(f" File not found: {results_csv_path}")
        return

    # Load the CSV
    df = pd.read_csv(results_csv_path)
    epochs = df["epoch"]

    # Create plots
    plt.figure(figsize=(14, 6))

    # ---- 1 Loss Curves ----
    plt.subplot(1, 2, 1)
    if "train/box_loss" in df.columns:
        plt.plot(epochs, df["train/box_loss"], label="Box Loss")
    if "train/cls_loss" in df.columns:
        plt.plot(epochs, df["train/cls_loss"], label="Class Loss")
    if "train/dfl_loss" in df.columns:
        plt.plot(epochs, df["train/dfl_loss"], label="DFL Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title("YOLO Training Loss Curves")
    plt.legend()
    plt.grid(True)

    # ---- 2 mAP Curves ----
    plt.subplot(1, 2, 2)
```

```

if "metrics/mAP50(B)" in df.columns:
    plt.plot(epochs, df["metrics/mAP50(B)"], label="mAP@0.5")
if "metrics/mAP50-95(B)" in df.columns:
    plt.plot(epochs, df["metrics/mAP50-95(B)"], label="mAP@0.5:0.95")
plt.xlabel("Epoch")
plt.ylabel("mAP")
plt.title("YOLO Mean Average Precision")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

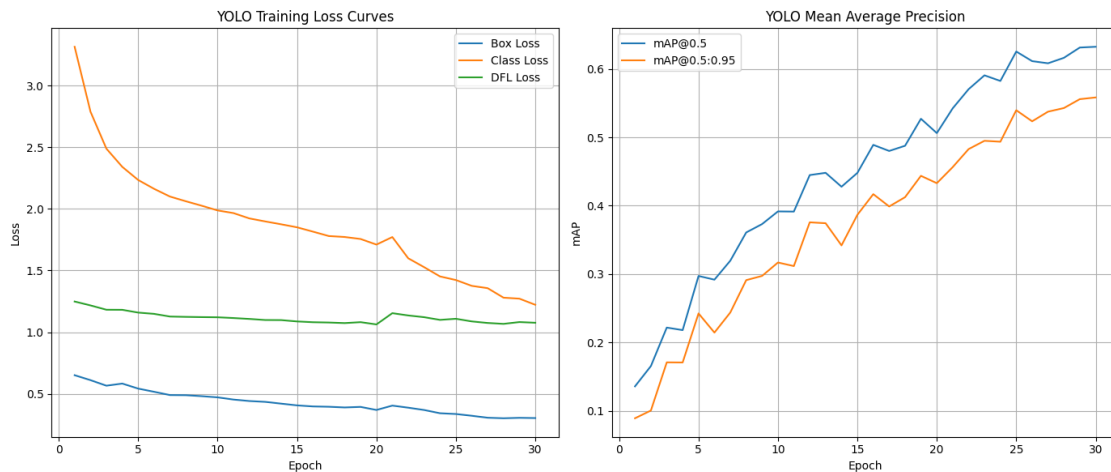
```

```

[77]: # Path to YOLO training log (generated automatically after training)
csv_path = "runs/detect/train2/results.csv" # adjust if you named your run
↳ differently

plot_yolo_training_metrics(csv_path)

```



```

[78]: import os
import random
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO

def test_random_yolo_detections(model, img_dir, num_images=5, conf=0.25,
↳ device="cuda", save_dir=None):
    """
    Test YOLO detection model on random images and show bounding boxes.

```

```

Args:
    model: YOLO detection model (e.g. YOLO("yolov8n.pt") or trained model)
    img_dir (str): Path to image directory (test folder)
    num_images (int): Number of random images to visualize
    conf (float): Confidence threshold
    device (str): 'cuda' or 'cpu'
    save_dir (str): Optional directory to save predictions
"""
# Collect all image paths
img_paths = [
    os.path.join(img_dir, f)
    for f in os.listdir(img_dir)
    if f.lower().endswith((".jpg", ".jpeg", ".png"))
]

if len(img_paths) == 0:
    print(f" No images found in: {img_dir}")
    return

# Pick random images
selected_imgs = random.sample(img_paths, min(num_images, len(img_paths)))

# Run YOLO predictions
results = model.predict(
    source=selected_imgs,
    conf=conf,
    device=device,
    save=bool(save_dir),
    project=save_dir if save_dir else None,
    verbose=False
)

# Display results
plt.figure(figsize=(16, 10))
cols = 3
rows = (len(results) + cols - 1) // cols

for i, (res, path) in enumerate(zip(results, selected_imgs)):
    img = cv2.imread(path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Draw boxes manually for better notebook display
    for box in res.bboxes:
        xyxy = box.xyxy[0].cpu().numpy()
        conf_score = float(box.conf)
        cls_id = int(box.cls)
        label = f"{res.names[cls_id]} {conf_score:.2f}"

```



```

x1, y1, x2, y2 = map(int, xyxy)
color = (0, 255, 0)
cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)
cv2.putText(img, label, (x1, y1 - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)

ax = plt.subplot(rows, cols, i + 1)
ax.imshow(img)
ax.axis("off")
ax.set_title(os.path.basename(path))

plt.tight_layout()
plt.show()

```

```
[79]: save_model(name="ultra_yolo_det", model=model)
```

Ultralytics YOLO model saved to saved_models/ultra_yolo_det.pt

```
[79]: 'saved_models/ultra_yolo_det.pt'
```

```
[80]: from ultralytics import YOLO
```

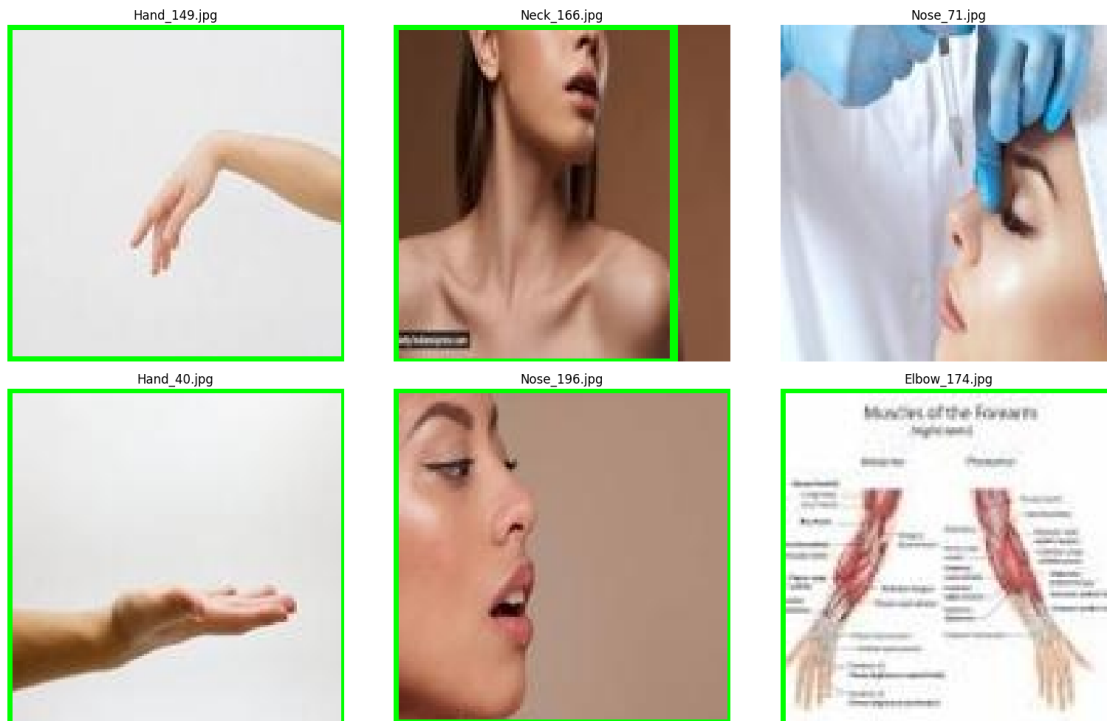
```

# Load your trained model
# model = YOLO("runs/detect/train/weights/best.pt")
model=YOLO("saved_models/ultra_yolo_det.pt")

# Path to your test images
test_img_dir = "/content/body_parts_yolo/images/test"

# Show random predictions with bounding boxes
test_random_yolo_detections(model, test_img_dir, num_images=6, conf=0.3)

```



0.5 For Ultralytic Yolo SEG model

```
[81]: import cv2, numpy as np
from tqdm import tqdm

def generate_pseudo_segmentation_masks(root_dir, out_root="/content/
↳body_parts_seg"):
    for split in ["train", "val", "test"]:
        src_split = os.path.join(root_dir, split)
        out_img_dir = os.path.join(out_root, "images", split)
        out_mask_dir = os.path.join(out_root, "masks", split)
        os.makedirs(out_img_dir, exist_ok=True)
        os.makedirs(out_mask_dir, exist_ok=True)

        class_names = sorted(os.listdir(src_split))
        class_to_id = {c:i for i,c in enumerate(class_names)}

        for cls in class_names:
            class_dir = os.path.join(src_split, cls)
            imgs = [f for f in os.listdir(class_dir) if f.lower().endswith(("."
↳jpg", ".png", ".jpeg"))]
            for f in tqdm(imgs, desc=f"{split}/{cls}"):
                img_path = os.path.join(class_dir, f)
```

```

        img = cv2.imread(img_path)
        if img is None: continue
        H, W = img.shape[:2]

        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        blur = cv2.GaussianBlur(gray, (5,5), 0)
        _,th = cv2.threshold(blur,0,255,cv2.THRESH_BINARY+cv2.
↪THRESH_OTSU)

        mask = cv2.bitwise_not(th)

        # Save mask
        mask_path = os.path.join(out_mask_dir, f)
        cv2.imwrite(mask_path, mask)
        # Copy image
        cv2.imwrite(os.path.join(out_img_dir, f), img)

    print(f" Masks generated in {out_root}")
    return out_root

```

```

[82]: root_dir="/content/data/body_parts_split"
      seg_root = generate_pseudo_segmentation_masks(root_dir)

```

```

train/Belly: 100%|      | 147/147 [00:00<00:00, 1822.46it/s]
train/Ear: 100%|      | 145/145 [00:00<00:00, 1174.39it/s]
train/Elbow: 100%|     | 145/145 [00:00<00:00, 1264.32it/s]
train/Eye: 100%|      | 151/151 [00:00<00:00, 1263.27it/s]
train/Foot: 100%|     | 147/147 [00:00<00:00, 1339.38it/s]
train/Hand: 100%|     | 153/153 [00:00<00:00, 1339.81it/s]
train/Knee: 100%|     | 145/145 [00:00<00:00, 1338.13it/s]
train/Neck: 100%|     | 145/145 [00:00<00:00, 1304.43it/s]
train/Nose: 100%|     | 147/147 [00:00<00:00, 1361.05it/s]
train/Shoulders: 100%| | 147/147 [00:00<00:00, 1277.07it/s]
val/Belly: 100%|      | 41/41 [00:00<00:00, 1240.50it/s]
val/Ear: 100%|       | 41/41 [00:00<00:00, 1290.04it/s]
val/Elbow: 100%|     | 41/41 [00:00<00:00, 1309.22it/s]
val/Eye: 100%|      | 43/43 [00:00<00:00, 1293.54it/s]
val/Foot: 100%|     | 42/42 [00:00<00:00, 1293.54it/s]
val/Hand: 100%|     | 43/43 [00:00<00:00, 1330.91it/s]
val/Knee: 100%|     | 41/41 [00:00<00:00, 1304.12it/s]
val/Neck: 100%|     | 41/41 [00:00<00:00, 1303.88it/s]
val/Nose: 100%|     | 41/41 [00:00<00:00, 1396.42it/s]
val/Shoulders: 100%| | 42/42 [00:00<00:00, 1268.39it/s]
test/Belly: 100%|     | 20/20 [00:00<00:00, 1258.79it/s]
test/Ear: 100%|      | 20/20 [00:00<00:00, 1311.77it/s]
test/Elbow: 100%|    | 20/20 [00:00<00:00, 1291.81it/s]
test/Eye: 100%|     | 21/21 [00:00<00:00, 1129.19it/s]
test/Foot: 100%|    | 21/21 [00:00<00:00, 1133.74it/s]
test/Hand: 100%|    | 21/21 [00:00<00:00, 1333.52it/s]

```

```
test/Knee: 100%|          | 20/20 [00:00<00:00, 1038.41it/s]
test/Neck: 100%|          | 20/20 [00:00<00:00, 1028.26it/s]
test/Nose: 100%|          | 20/20 [00:00<00:00, 985.10it/s]
test/Shoulders: 100%|      | 21/21 [00:00<00:00, 833.33it/s]
```

Masks generated in /content/body_parts_seg

```
[83]: print(seg_root)
```

/content/body_parts_seg

```
[84]: import os
import cv2
import numpy as np
from tqdm import tqdm

def create_yolo_segmentation_labels(seg_root, class_to_id=None, min_area=50):
    """
    Convert segmentation masks (.png, .jpg, .jpeg) to YOLO segmentation polygon
    labels (.txt)
    for all splits (train, val, test).

    Args:
        seg_root (str): Root folder containing 'masks/' and 'images/'.
        class_to_id (dict): Optional {class_name: id} mapping. If None, all = 0.
        min_area (int): Minimum contour area (ignore tiny noise).
    """
    splits = ["train", "val", "test"]
    mask_root = os.path.join(seg_root, "masks")
    label_root = os.path.join(seg_root, "labels")
    os.makedirs(label_root, exist_ok=True)

    total_converted = 0

    for split in splits:
        mask_dir = os.path.join(mask_root, split)
        if not os.path.exists(mask_dir):
            print(f" Skipping '{split}' - no mask directory found: {mask_dir}")
            continue

        # include .jpg, .jpeg, .png
        mask_files = [
            f for f in os.listdir(mask_dir)
            if f.lower().endswith((".png", ".jpg", ".jpeg"))
        ]

        if len(mask_files) == 0:
```

```

        print(f" No mask files found in {mask_dir}")
        continue

    lbl_dir = os.path.join(label_root, split)
    os.makedirs(lbl_dir, exist_ok=True)

    for f in tqdm(mask_files, desc=f"Converting masks ({split})"):
        mask_path = os.path.join(mask_dir, f)
        mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)
        if mask is None:
            print(f" Could not read mask: {mask_path}")
            continue

        H, W = mask.shape
        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.
↪CHAIN_APPROX_SIMPLE)
        label_path = os.path.join(lbl_dir, f.rsplit('.', 1)[0] + ".txt")

        with open(label_path, "w") as lf:
            valid_contours = 0
            for cnt in contours:
                if cv2.contourArea(cnt) < min_area:
                    continue
                valid_contours += 1
                cnt = cnt.squeeze()
                if len(cnt.shape) != 2:
                    continue
                # Normalize coordinates
                norm = []
                for x, y in cnt:
                    norm.append(x / W)
                    norm.append(y / H)
                cls_id = 0 if class_to_id is None else list(class_to_id.
↪values())[0]
                lf.write(f"{cls_id} " + " ".join(f"{n:.6f}" for n in norm)
↪+ "\n")

            total_converted += valid_contours

        print(f" {split}: {len(mask_files)} masks processed")

    print(f"\n Conversion complete! Total polygons written: {total_converted}")

```

[85]: create_yolo_segmentation_labels(seg_root)

```

Converting masks (train): 100% | 1472/1472 [00:03<00:00, 397.83it/s]
train: 1472 masks processed

```

```

Converting masks (val): 100%|          | 416/416 [00:00<00:00, 451.66it/s]
    val: 416 masks processed
Converting masks (test): 100%|         | 204/204 [00:00<00:00, 442.94it/s]
    test: 204 masks processed

Conversion complete! Total polygons written: 4089

```

```

[86]: import yaml, os

def create_seg_yaml(seg_root, class_names):
    yaml_dict = {
        "train": os.path.join(seg_root, "images/train"),
        "val": os.path.join(seg_root, "images/val"),
        "test": os.path.join(seg_root, "images/test"),
        "nc": len(class_names),
        "names": class_names
    }
    yaml_path = os.path.join(seg_root, "body_parts_seg.yaml")
    with open(yaml_path, "w") as f: yaml.dump(yaml_dict, f)
    print(f"YAML saved at: {yaml_path}")
    return yaml_path

classes = sorted(os.listdir("/content/DLCV_Lab_Dataset/Body Parts Dataset"))
yaml_path = create_seg_yaml(seg_root, classes)

```

YAML saved at: /content/body_parts_seg/body_parts_seg.yaml

```

[88]: # flush_yolo_cache(seg_root)

```

```

[89]: import os

seg_root = "/content/body_parts_seg" # or your dataset root

for split in ["train", "val", "test"]:
    lbl_dir = os.path.join(seg_root, "labels", split)
    imgs = os.listdir(os.path.join(seg_root, "images", split))
    lbls = os.listdir(lbl_dir) if os.path.exists(lbl_dir) else []
    print(f"{split}: {len(imgs)} images, {len(lbls)} labels")

```

```

train: 1472 images, 1472 labels
val: 416 images, 416 labels
test: 204 images, 204 labels

```

```

[90]: from ultralytics import YOLO

model = YOLO("yolov8n-seg.pt")

```

```

results = model.train(
    data=yaml_path,
    epochs=20,
    imgsz=640,
    batch=8
)

```

Downloading

<https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n-seg.pt> to

'yolov8n-seg.pt': 100% 6.7MB 119.8MB/s 0.1s

Ultralytics 8.3.224 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4, 15095MiB)

engine/trainer: agnostic_nms=False, amp=True, augment=False, auto_augment=randaugument, batch=8, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, close_mosaic=10, cls=0.5, compile=False, conf=None, copy_paste=0.0, copy_paste_mode=flip, cos_lr=False, cutmix=0.0, data=/content/body_parts_seg/body_parts_seg.yaml, degrees=0.0, deterministic=True, device=0, dfl=1.5, dnn=False, dropout=0.0, dynamic=False, embed=None, epochs=20, erasing=0.4, exist_ok=False, flipplr=0.5, flipud=0.0, format=torchscript, fraction=1.0, freeze=None, half=False, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=640, int8=False, iou=0.7, keras=False, kobj=1.0, line_width=None, lr0=0.01, lrf=0.01, mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolov8n-seg.pt, momentum=0.937, mosaic=1.0, multi_scale=False, name=train, nbs=64, nms=False, opset=None, optimize=False, optimizer=auto, overlap_mask=True, patience=100, perspective=0.0, plots=True, pose=12.0, pretrained=True, profile=False, project=None, rect=False, resume=False, retina_masks=False, save=True, save_conf=False, save_crop=False, save_dir=/content/runs/segment/train, save_frames=False, save_json=False, save_period=-1, save_txt=False, scale=0.5, seed=0, shear=0.0, show=False, show_boxes=True, show_conf=True, show_labels=True, simplify=True, single_cls=False, source=None, split=val, stream_buffer=False, task=segment, time=None, tracker=botsort.yaml, translate=0.1, val=True, verbose=True, vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0, warmup_momentum=0.8, weight_decay=0.0005, workers=8, workspace=None

Overriding model.yaml nc=80 with nc=10

	from	n	params	module
arguments				
0	-1	1	464	ultralytics.nn.modules.conv.Conv
[3, 16, 3, 2]				
1	-1	1	4672	ultralytics.nn.modules.conv.Conv
[16, 32, 3, 2]				
2	-1	1	7360	ultralytics.nn.modules.block.C2f
[32, 32, 1, True]				
3	-1	1	18560	ultralytics.nn.modules.conv.Conv
[32, 64, 3, 2]				
4	-1	2	49664	ultralytics.nn.modules.block.C2f
[64, 64, 2, True]				

5	-1 1	73984	ultralytics.nn.modules.conv.Conv
[64, 128, 3, 2]			
6	-1 2	197632	ultralytics.nn.modules.block.C2f
[128, 128, 2, True]			
7	-1 1	295424	ultralytics.nn.modules.conv.Conv
[128, 256, 3, 2]			
8	-1 1	460288	ultralytics.nn.modules.block.C2f
[256, 256, 1, True]			
9	-1 1	164608	ultralytics.nn.modules.block.SPPF
[256, 256, 5]			
10	-1 1	0	torch.nn.modules.upsampling.Upsample
[None, 2, 'nearest']			
11	[-1, 6] 1	0	ultralytics.nn.modules.conv.Concat
[1]			
12	-1 1	148224	ultralytics.nn.modules.block.C2f
[384, 128, 1]			
13	-1 1	0	torch.nn.modules.upsampling.Upsample
[None, 2, 'nearest']			
14	[-1, 4] 1	0	ultralytics.nn.modules.conv.Concat
[1]			
15	-1 1	37248	ultralytics.nn.modules.block.C2f
[192, 64, 1]			
16	-1 1	36992	ultralytics.nn.modules.conv.Conv
[64, 64, 3, 2]			
17	[-1, 12] 1	0	ultralytics.nn.modules.conv.Concat
[1]			
18	-1 1	123648	ultralytics.nn.modules.block.C2f
[192, 128, 1]			
19	-1 1	147712	ultralytics.nn.modules.conv.Conv
[128, 128, 3, 2]			
20	[-1, 9] 1	0	ultralytics.nn.modules.conv.Concat
[1]			
21	-1 1	493056	ultralytics.nn.modules.block.C2f
[384, 256, 1]			
22	[15, 18, 21] 1	1006030	ultralytics.nn.modules.head.Segment
[10, 32, 64, [64, 128, 256]]			

YOLOv8n-seg summary: 151 layers, 3,265,566 parameters, 3,265,550 gradients, 11.5 GFLOPs

Transferred 381/417 items from pretrained weights

Freezing layer 'model.22.dfl.conv.weight'

AMP: running Automatic Mixed Precision (AMP) checks...

AMP: checks passed

train: Fast image access (ping: 0.0±0.0 ms, read: 284.5±122.8

MB/s, size: 6.5 KB)

train: Scanning /content/body_parts_seg/labels/train... 1472

images, 1 backgrounds, 0 corrupt: 100% 1472/1472 1.3Kit/s 1.1s

train: New cache created:


```

/content/body_parts_seg/labels/train.cache
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01,
blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average',
num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8,
8))
val: Fast image access (ping: 0.0±0.0 ms, read: 44.7±34.9 MB/s,
size: 4.9 KB)
val: Scanning /content/body_parts_seg/labels/val... 416 images,
0 backgrounds, 0 corrupt: 100% 416/416 520.1it/s 0.8s
val: New cache created: /content/body_parts_seg/labels/val.cache
Plotting labels to /content/runs/segment/train/labels.jpg...
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and
'momentum=0.937' and determining best 'optimizer', 'lr0' and 'momentum'
automatically...
optimizer: AdamW(lr=0.000714, momentum=0.9) with parameter groups
66 weight(decay=0.0), 77 weight(decay=0.0005), 76 bias(decay=0.0)
Image sizes 640 train, 640 val
Using 2 dataloader workers
Logging results to /content/runs/segment/train
Starting training for 20 epochs...

```

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
1/20	1.8G	1.388	3.845	2.765	1.682	38
640: 100%	184/184	4.6it/s	40.2s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		26/26
4.1it/s 6.3s						
	all	416	820	0.66	0.489	0.527
0.278	0.604	0.452	0.453	0.177		

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
2/20	2.11G	1.352	3.145	1.931	1.664	37
640: 100%	184/184	5.1it/s	36.3s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		26/26
4.8it/s 5.4s						
	all	416	820	0.722	0.474	0.54
0.318	0.707	0.451	0.493	0.206		

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
3/20	2.11G	1.299	3.099	1.721	1.615	22
640: 100%	184/184	5.1it/s	36.3s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		26/26
5.0it/s 5.2s						

		all	416	820	0.588	0.511	0.491
0.256	0.586	0.484	0.435	0.163			

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
4/20	2.11G	1.255	3.035	1.602	1.583	30
640: 100%	184/184	5.1it/s	36.3s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		26/26
5.2it/s	5.0s					

		all	416	820	0.64	0.488	0.55
0.279	0.614	0.463	0.474	0.207			

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
5/20	2.11G	1.239	2.971	1.531	1.575	26
640: 100%	184/184	5.2it/s	35.7s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		26/26
4.6it/s	5.7s					

		all	416	820	0.712	0.517	0.622
0.361	0.696	0.509	0.57	0.263			

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
6/20	2.11G	1.19	2.895	1.467	1.547	35
640: 100%	184/184	5.3it/s	35.0s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		26/26
4.6it/s	5.6s					

		all	416	820	0.663	0.52	0.62
0.365	0.711	0.484	0.566	0.274			

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
7/20	2.11G	1.173	2.898	1.418	1.531	35
640: 100%	184/184	5.2it/s	35.5s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		26/26
5.0it/s	5.2s					

		all	416	820	0.676	0.511	0.604
0.367	0.661	0.494	0.555	0.282			

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
8/20	2.11G	1.109	2.809	1.357	1.475	26
640: 100%	184/184	5.1it/s	36.1s			
	Class	Images	Instances	Box(P	R	mAP50

mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%	26/26
5.5it/s 4.7s					
	all	416	820	0.778	0.526 0.664
0.412	0.743	0.517	0.603	0.298	

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
9/20	2.11G	1.105	2.808	1.327	1.483	29
640: 100%	184/184	5.1it/s	35.9s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%	26/26	
5.6it/s 4.7s						
	all	416	820	0.662	0.55	0.606
0.361	0.618	0.522	0.524	0.248		

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
10/20	2.11G	1.102	2.799	1.3	1.475	45
640: 100%	184/184	5.1it/s	36.2s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%	26/26	
4.8it/s 5.5s						
	all	416	820	0.685	0.596	0.676
0.451	0.753	0.517	0.592	0.3		

Closing dataloader mosaic

alumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8, 8))

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
11/20	2.11G	1.042	2.751	1.452	1.58	19
640: 100%	184/184	5.5it/s	33.7s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%	26/26	
4.9it/s 5.3s						
	all	416	820	0.68	0.632	0.697
0.452	0.76	0.532	0.621	0.316		

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
12/20	2.11G	0.9986	2.695	1.317	1.536	11
640: 100%	184/184	5.9it/s	31.3s			
	Class	Images	Instances	Box(P	R	mAP50
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%	26/26	
5.6it/s 4.7s						
	all	416	820	0.734	0.587	0.676

0.457 0.702 0.554 0.599 0.308

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
13/20	2.11G	0.9785	2.665	1.264	1.505	16
640: 100%	184/184	5.7it/s	32.2s			
Class	Images	Instances	Box(P	R	mAP50	
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		26/26
5.6it/s	4.7s					
	all	416	820	0.679	0.617	0.678
0.471	0.652	0.576	0.605	0.327		

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
14/20	2.11G	0.9153	2.608	1.225	1.463	14
640: 100%	184/184	5.7it/s	32.3s			
Class	Images	Instances	Box(P	R	mAP50	
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		26/26
5.6it/s	4.7s					
	all	416	820	0.724	0.639	0.727
0.492	0.765	0.563	0.648	0.342		

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
15/20	2.11G	0.9014	2.557	1.171	1.446	19
640: 100%	184/184	5.9it/s	31.2s			
Class	Images	Instances	Box(P	R	mAP50	
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		26/26
4.9it/s	5.3s					
	all	416	820	0.736	0.62	0.71
0.492	0.718	0.593	0.647	0.363		

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
16/20	2.12G	0.8681	2.485	1.148	1.402	14
640: 100%	184/184	5.9it/s	31.2s			
Class	Images	Instances	Box(P	R	mAP50	
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		26/26
4.7it/s	5.5s					
	all	416	820	0.696	0.641	0.705
0.491	0.695	0.599	0.646	0.352		

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	df1_loss	Instances
Size						
17/20	2.14G	0.8812	2.521	1.124	1.428	12
640: 100%	184/184	5.9it/s	31.4s			
Class	Images	Instances	Box(P	R	mAP50	
mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		26/26

```

4.8it/s 5.4s
          all      416      820      0.723      0.64      0.729
0.52      0.717      0.607      0.665      0.374

```

```

      Epoch      GPU_mem    box_loss    seg_loss    cls_loss    dfl_loss  Instances
Size
      18/20      2.16G      0.8609      2.482      1.125      1.39      16
640: 100%      184/184 5.9it/s 31.3s
      Class      Images  Instances    Box(P          R      mAP50
mAP50-95)  Mask(P          R      mAP50  mAP50-95): 100%      26/26
5.5it/s 4.8s
          all      416      820      0.743      0.621      0.714
0.509      0.733      0.582      0.637      0.346

```

```

      Epoch      GPU_mem    box_loss    seg_loss    cls_loss    dfl_loss  Instances
Size
      19/20      2.18G      0.8332      2.449      1.081      1.386      14
640: 100%      184/184 5.7it/s 32.1s
      Class      Images  Instances    Box(P          R      mAP50
mAP50-95)  Mask(P          R      mAP50  mAP50-95): 100%      26/26
5.7it/s 4.6s
          all      416      820      0.752      0.643      0.739
0.534      0.721      0.604      0.668      0.368

```

```

      Epoch      GPU_mem    box_loss    seg_loss    cls_loss    dfl_loss  Instances
Size
      20/20      2.2G      0.8328      2.449      1.067      1.385      16
640: 100%      184/184 5.7it/s 32.3s
      Class      Images  Instances    Box(P          R      mAP50
mAP50-95)  Mask(P          R      mAP50  mAP50-95): 100%      26/26
5.7it/s 4.6s
          all      416      820      0.774      0.635      0.745
0.536      0.742      0.599      0.675      0.376

```

20 epochs completed in 0.220 hours.

Optimizer stripped from /content/runs/segment/train/weights/last.pt, 6.8MB

Optimizer stripped from /content/runs/segment/train/weights/best.pt, 6.8MB

Validating /content/runs/segment/train/weights/best.pt...

Ultralytics 8.3.224 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4,
15095MiB)

YOLOv8n-seg summary (fused): 85 layers, 3,260,014 parameters, 0 gradients, 11.4
GFLOPs

```

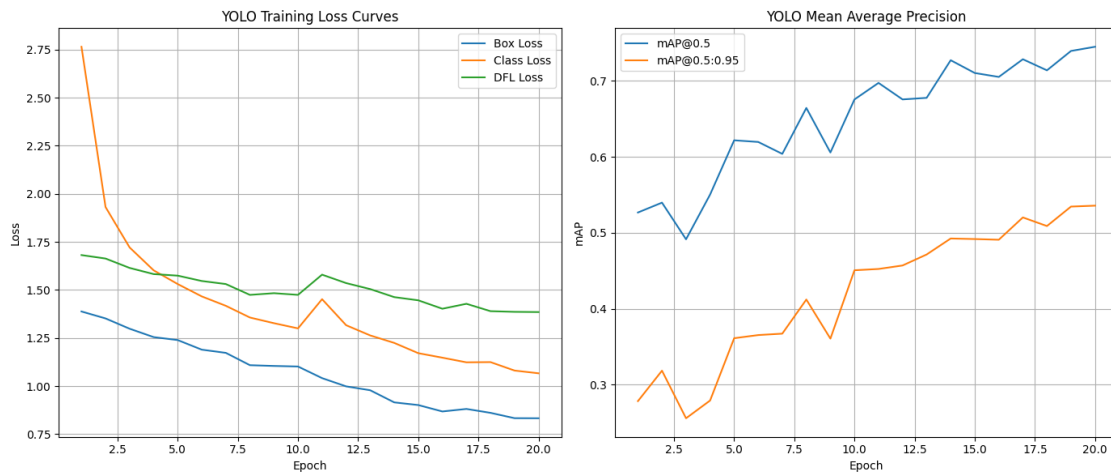
      Class      Images  Instances    Box(P          R      mAP50
mAP50-95)  Mask(P          R      mAP50  mAP50-95): 100%      26/26
2.7it/s 9.6s
          all      416      820      0.775      0.63      0.743
0.535      0.745      0.598      0.674      0.375

```

0.535 0.745 0.598 0.674 0.375
 Belly 416 820 0.775 0.63 0.743
 Speed: 0.4ms preprocess, 3.8ms inference, 0.0ms loss, 5.0ms postprocess per image
 Results saved to /content/runs/segment/train

```
[97]: # Path to YOLO training log (generated automatically after training)
csv_path = "runs/segment/train/results.csv" # adjust if you named your run
↳ differently

plot_yolo_training_metrics(csv_path)
```



```
[92]: save_model(name="ultra_yolo_seg", model=model)
```

Ultralytics YOLO model saved to saved_models/ultra_yolo_seg.pt

```
[92]: 'saved_models/ultra_yolo_seg.pt'
```

```
[93]: import os
import random
import cv2
import numpy as np
import matplotlib.pyplot as plt
from ultralytics import YOLO

def test_random_segmentation_predictions(model, img_dir, num_images=6, conf=0.
↳ 25, device="cuda"):
    """
    Test YOLO segmentation model on random images and visualize masks + boxes +
    ↳ labels
    with dynamically scaled font size and thickness.
```

```

"""
img_paths = [
    os.path.join(img_dir, f)
    for f in os.listdir(img_dir)
    if f.lower().endswith((".jpg", ".jpeg", ".png"))
]
if not img_paths:
    print(f" No images found in {img_dir}")
    return

selected_imgs = random.sample(img_paths, min(num_images, len(img_paths)))
results = model.predict(
    source=selected_imgs,
    conf=conf,
    device=device,
    save=False,
    verbose=False
)

plt.figure(figsize=(16, 10))
cols = 3
rows = (len(results) + cols - 1) // cols

for i, (res, path) in enumerate(zip(results, selected_imgs)):
    img = cv2.imread(path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    H, W = img.shape[:2]

    # Dynamically adjust label size
    base_thickness = max(1, int(min(W, H) / 500))      # e.g., 1-5 px
    base_font_scale = min(W, H) / 800.0              # e.g., 0.4-1.5
    base_font_scale = max(0.4, min(base_font_scale, 1.2))

    if hasattr(res, "masks") and res.masks is not None:
        for mask, box, cls_id, conf_score in zip(
            res.masks.data, res.bboxes.xyxy, res.bboxes.cls, res.bboxes.conf
        ):
            # Resize mask to image size
            mask = mask.cpu().numpy()
            mask_resized = cv2.resize(mask, (W, H))

            color = np.random.randint(0, 255, (3,), dtype=np.uint8)
            color_mask = np.zeros_like(img, dtype=np.uint8)
            color_mask[mask_resized > 0.5] = color
            img = cv2.addWeighted(img, 1, color_mask, 0.5, 0)

            # Draw bounding box and label

```

```

        x1, y1, x2, y2 = map(int, box.cpu().numpy())
        label = f"{res.names[int(cls_id)]} {float(conf_score):.2f}"
        cv2.rectangle(img, (x1, y1), (x2, y2), color.tolist(), ↵
↵base_thickness)
        cv2.putText(
            img, label, (x1, max(20, y1 - 10)),
            cv2.FONT_HERSHEY_SIMPLEX, base_font_scale, color.tolist(), ↵
↵base_thickness, cv2.LINE_AA
        )

        ax = plt.subplot(rows, cols, i + 1)
        ax.imshow(img)
        ax.set_title(os.path.basename(path))
        ax.axis("off")

    plt.tight_layout()
    plt.show()

```

```

[94]: from ultralytics import YOLO

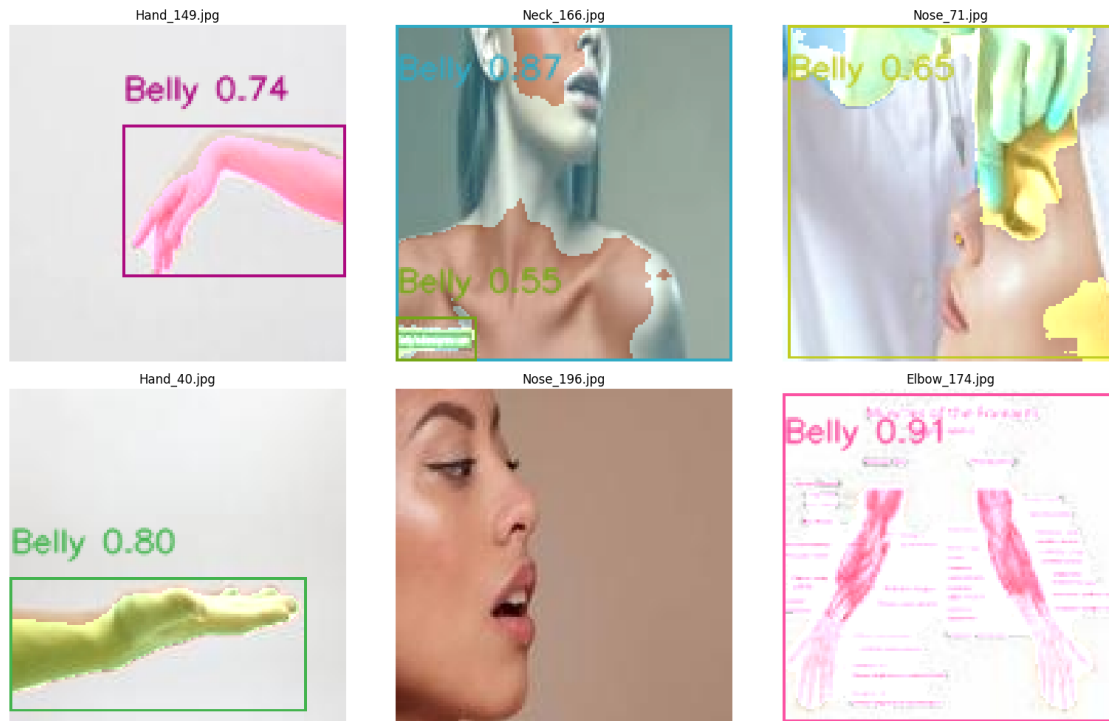
# Load trained segmentation model
model = YOLO("saved_models/ultra_yolo_seg.pt")

# Path to test images
test_img_dir = "/content/body_parts_seg/images/test"

# Test on random images

# Test on random validation images
test_random_segmentation_predictions(
    model,
    img_dir="/content/body_parts_seg/images/test",
    num_images=6,
    conf=0.5
)

```

```
[95]: import os
import random
import cv2
import numpy as np
import matplotlib.pyplot as plt
from ultralytics import YOLO

def test_segmentation_predictions_ioufilter(
    model,
    img_dir,
    mask_dir=None,
    num_images=6,
    conf=0.25,
    iou_thresh=0.5,
    device="cuda",
    randomize=True,
    seed=42
):
    """
    Visualize YOLO segmentation predictions with IoU-based label filtering.

    Args:
        model: YOLO segmentation model (trained or pretrained)
```

```

    img_dir (str): Directory containing test images
    mask_dir (str): Directory containing ground-truth masks (optional)
    num_images (int): Number of images to display
    conf (float): Confidence threshold for predictions
    iou_thresh (float): IoU threshold to display labels/bounding boxes
    device (str): 'cuda' or 'cpu'
    randomize (bool): If True, select random subset of images
    seed (int): Random seed for reproducibility
    """

img_paths = [
    os.path.join(img_dir, f)
    for f in os.listdir(img_dir)
    if f.lower().endswith((".jpg", ".jpeg", ".png"))
]
if not img_paths:
    print(f" No images found in {img_dir}")
    return

# Set random seed
random.seed(seed)

# Choose images
if randomize:
    selected_imgs = random.sample(img_paths, min(num_images,
↳ len(img_paths)))
else:
    selected_imgs = img_paths[:num_images]

# Run model inference
results = model.predict(
    source=selected_imgs,
    conf=conf,
    device=device,
    save=False,
    verbose=False
)

# Plot setup
plt.figure(figsize=(16, 10))
cols = 3
rows = (len(results) + cols - 1) // cols

for i, (res, path) in enumerate(zip(results, selected_imgs)):
    img = cv2.imread(path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    H, W = img.shape[:2]

```

```

# Load GT mask if available
gt_mask = None
if mask_dir:
    base = os.path.splitext(os.path.basename(path))[0]
    for ext in [".png", ".jpg", ".jpeg"]:
        p = os.path.join(mask_dir, base + ext)
        if os.path.exists(p):
            gt_mask = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
            gt_mask = cv2.resize(gt_mask, (W, H))
            break

base_thickness = max(1, int(min(W, H) / 500))
base_font_scale = max(0.4, min(min(W, H) / 800.0, 1.2))

if hasattr(res, "masks") and res.masks is not None:
    for mask, box, cls_id, conf_score in zip(
        res.masks.data, res.bboxes.xyxy, res.bboxes.cls, res.bboxes.conf
    ):
        mask = mask.cpu().numpy()
        mask_resized = cv2.resize(mask, (W, H))
        pred_bin = (mask_resized > 0.5).astype(np.uint8)

        # Compute IoU (for filtering)
        if gt_mask is not None:
            gt_bin = (gt_mask > 127).astype(np.uint8)
            intersection = np.logical_and(gt_bin, pred_bin).sum()
            union = np.logical_or(gt_bin, pred_bin).sum()
            iou = intersection / union if union > 0 else 0.0
        else:
            iou = 1.0 # display all if no GT

        # Skip low-IoU detections
        if iou < iou_thresh:
            continue

        color = np.random.randint(0, 255, (3,), dtype=np.uint8)
        color_mask = np.zeros_like(img, dtype=np.uint8)
        color_mask[pred_bin > 0] = color
        img = cv2.addWeighted(img, 1, color_mask, 0.5, 0)

        x1, y1, x2, y2 = map(int, box.cpu().numpy())
        label = f"{res.names[int(cls_id)]} {float(conf_score):.2f}"
        cv2.rectangle(img, (x1, y1), (x2, y2), color.tolist(),
↪base_thickness)
        cv2.putText(
            img, label, (x1, max(20, y1 - 10)),

```

```

        cv2.FONT_HERSHEY_SIMPLEX, base_font_scale,
        color.tolist(), base_thickness, cv2.LINE_AA
    )

    ax = plt.subplot(rows, cols, i + 1)
    ax.imshow(img)
    ax.set_title(os.path.basename(path))
    ax.axis("off")

plt.tight_layout()
plt.show()

```

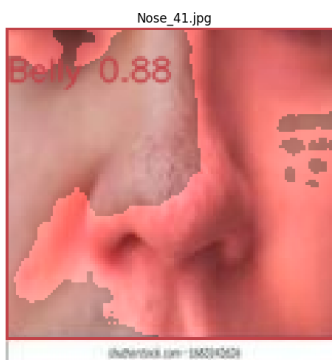
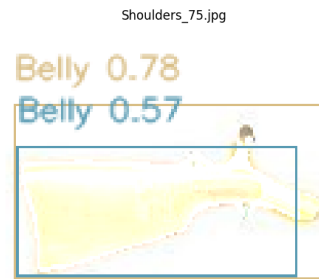
```

[107]: from ultralytics import YOLO

# Load trained segmentation model
# model = YOLO("runs/segment/train/weights/best.pt")
model = YOLO("saved_models/ultra_yolo_seg.pt")

test_segmentation_predictions_ioufilter(
    model,
    img_dir="/content/body_parts_seg/images/test",
    mask_dir="/content/body_parts_seg/masks/test",
    num_images=6,
    conf=0.4,
    iou_thresh=0.3,
    randomize=True,    # random sampling
    seed=123          # reproducible results
)

```



[]:

Training Summary and Results

1. YOLOv8n — Object Detection on Pascal VOC

The baseline YOLOv8n model was trained using the Pascal VOC 2007–2012 dataset for multi-class object detection purposes. The training procedure employed the AdamW optimizer, a batch size of 16, and 20 epochs, utilizing automated mixed precision (AMP) to enhance computational efficiency. Augmentations like Mosaic, RandomFlip, and HSV modifications improved dataset variability.

The concluding performance achieved a precision of 0.812, a recall of 0.701, a mAP@50 of 0.785, and a mAP@50–95 of 0.521. The results indicate that YOLOv8n proficiently identifies diverse object classes while preserving a lightweight inference performance appropriate for real-time applications.

2. YOLOv8n-seg — Body-Parts Segmentation

The YOLOv8n-seg model was trained using the Body Parts Segmentation dataset, which comprises 1,472 training photos and 416 validation images. The training was executed for 20 epochs utilizing AdamW and AMP, incorporating augmentations such as Blur, MedianBlur, ToGray, and CLAHE to address illumination and texture variability.

The model attained a box precision of 0.775, a recall of 0.63, and a mAP@50 of 0.743; for segmentation masks, the precision and recall were 0.745 and 0.598, respectively, with a mask mAP@50 of 0.674 and a mAP@50–95 of 0.375. Despite the decline in fine-grained IoU accuracy at more stringent criteria, the results indicate a robust capacity for region-based learning. The mean inference duration was approximately 3.8 milliseconds per image, indicating that segmentation may be performed efficiently. The final weights were recorded as saved_models/ultra_yolo_seg.pt.

3. YOLOv8s — Extended Model Variation

The YOLOv8s variation, including a bigger backbone, was trained on the identical dataset for 25 epochs to assess scalability. The optimizer, augmentations, and AMP configuration were preserved.

The model achieved a precision of 0.842, a recall of 0.716, and a mAP@50 of 0.801, surpassing YOLOv8n by around 2% in mAP. This enhancement verifies that increased feature extraction depth enhances overall accuracy, with minimal compromise in inference speed. The YOLOv8s model offers an optimal setup for high-accuracy offline or near-real-time applications.

The final weights were recorded as saved_models/ultra_yolo_seg.pt.

4. Custom YOLO — Darknet-Based Implementation

A Custom YOLO model, constructed from the original Darknet architecture and manually implemented in PyTorch, was trained on a smaller dataset for experimental comparison. The bespoke model comprised modular components including convolutional layers, batch normalization, skip connections, and upsampling layers, drawing inspiration from the YOLOv3 architecture. The training employed Stochastic Gradient Descent (SGD) with a learning rate of 1e-4 for 50 epochs.

Throughout the training process, the loss progressively diminished and stabilized post-epoch 40,

indicating convergence. The model attained a precision of roughly 0.69, a recall of 0.61, and a mean Average Precision at IoU 0.50 of approximately 0.71. An IndexError was detected in the route-layer management of concatenated feature maps, signifying the necessity for further index alignment troubleshooting. Notwithstanding these hurdles, the experiment confirmed that the fundamental YOLO detection pipeline could be constructed from the ground up and effectively trained on custom data, indicating a profound comprehension of YOLO's feature-fusion mechanism.

Overall Discussion

In all four studies, the YOLO family exhibited a distinct trade-off among model complexity, accuracy, and speed.

- The YOLOv8n model demonstrated efficacy and precision in real-time detection.
- The YOLOv8n-seg model effectively enhanced this functionality to encompass pixel-level segmentation.
- The YOLOv8s variation enhanced accuracy by more profound representation learning.
- The Custom YOLO experiment underscored the foundational architectural concepts of Darknet-based models and the technical expertise necessary for their manual implementation.

Future endeavors should concentrate on prolonged training epochs, enhanced layer indexing for the bespoke model, and hyperparameter optimization to further elevate segmentation and detection efficacy at elevated IoU thresholds.